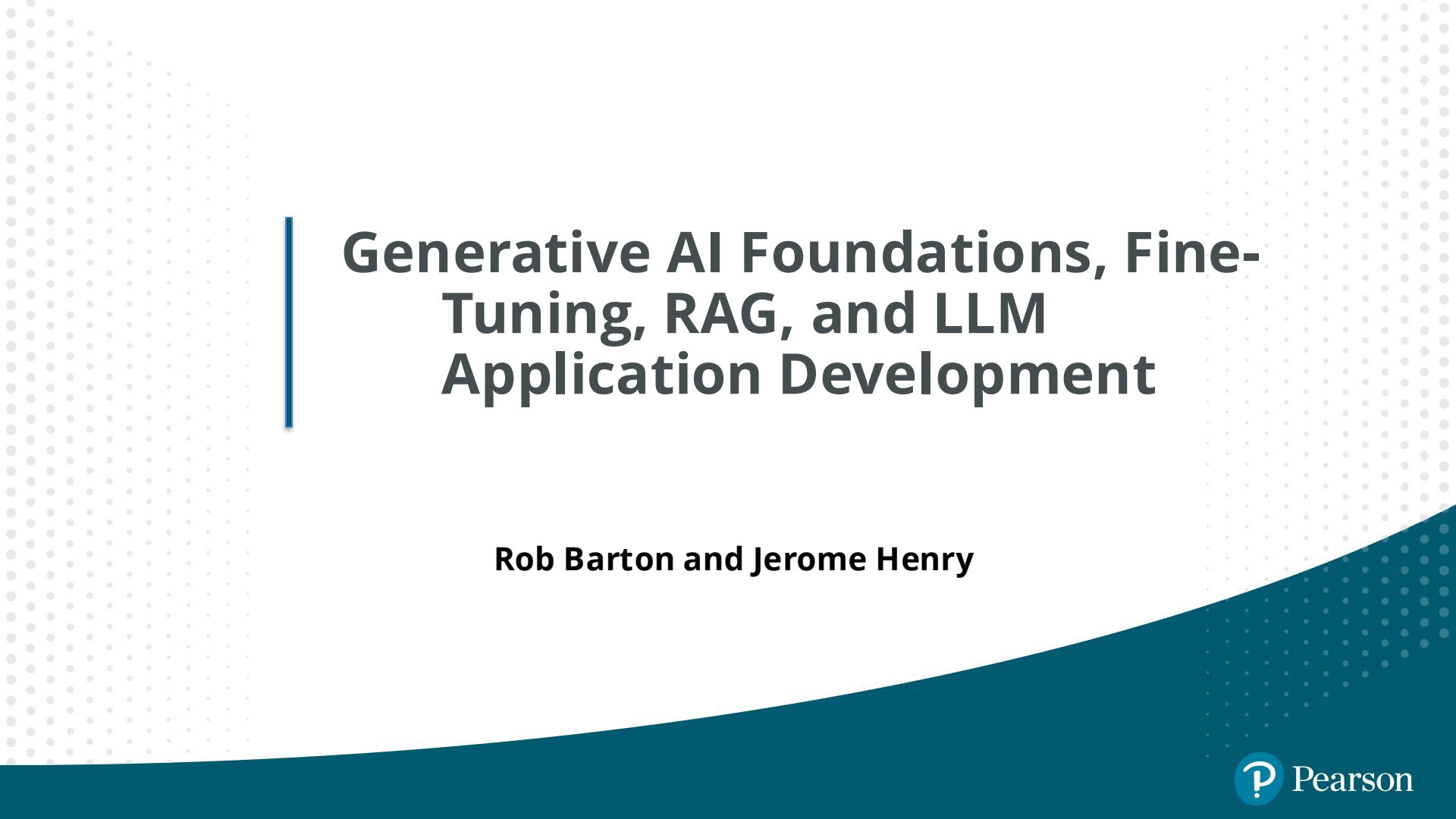




Generative AI Foundations, Fine-Tuning, RAG, and LLM Application Development

Rob Barton and Jerome Henry

Course Overview

- Section 1: An Introduction to Large Language Models
- Section 2: Prompt Engineering
- Section 3: Retrieval Augmented Generation (RAG)
- Section 4: Working with Open Source LLMs
- Section 5: Fine-Tuning LLMs
- Section 6: Developing LLM Applications

Section 1:

An Introduction to Large Language Models

Objectives:

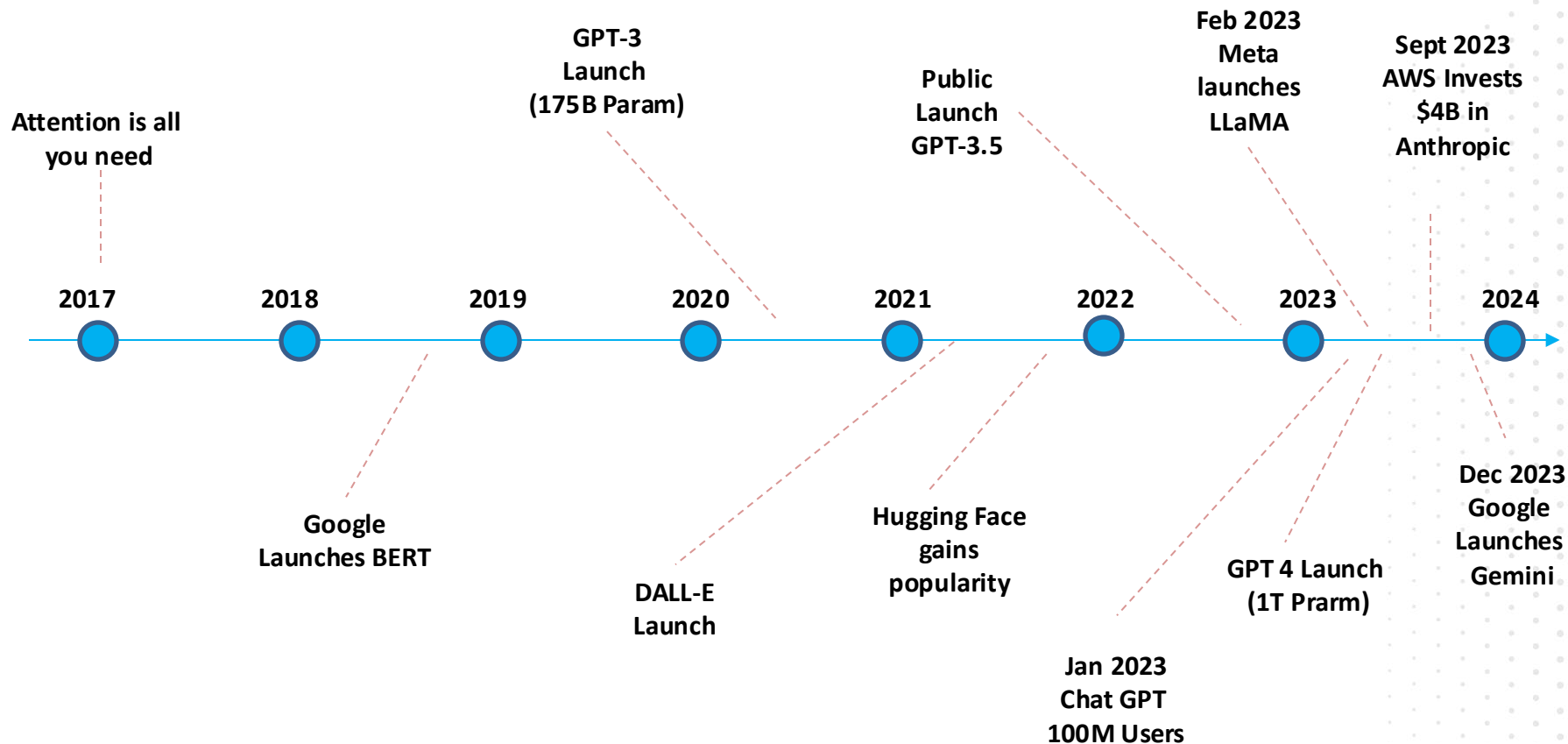
- How did Large Language Models Develop?
- Tokenization and Word Embedding
- The Transformer Architecture
- The Self-Attention Mechanism
- Positional Encoding
- The Decoder Architecture

Section 1:

An Introduction to Generative AI

- An Introduction to Generative AI

A Short History of LLMs

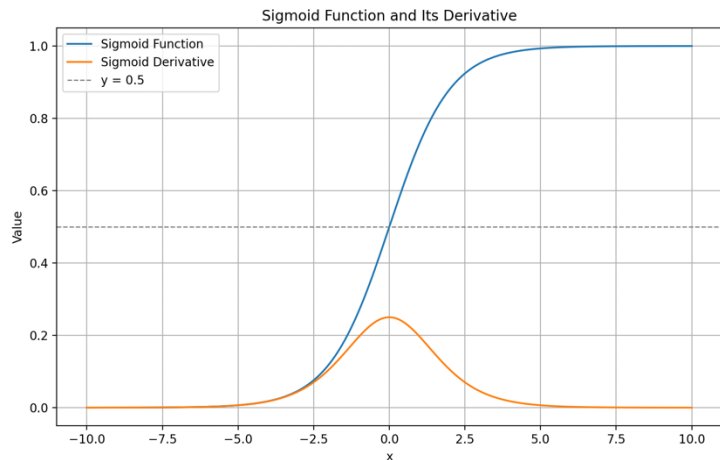


The Many Uses of GenAI

Coding, displaying output, debugging. explaining code

R

using python, plot the sigmoid function from $x=-10$ to $x=10$. On the same plot, show the first derivative of the sigmoid function.



Sure, I can help you with that! You can use the ``numpy`` library to calculate the sigmoid function and its derivative, and the ``matplotlib`` library to create the plot. If you haven't already, you'll need to install these libraries using the following command:



bash

 Copy code

```
pip install numpy matplotlib
```

Here's the code to plot the sigmoid function and its first derivative:

python

 Copy code

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return sigmoid(x) * (1 - sigmoid(x))
```

Great for Generating Images (drawn using OpenAI's DALL-E)



DALL-E

History

Collections

Edit the detailed description

Albert Einstein competing at the Olympics, digital art



Section 1:

An Introduction to Generative AI

- Language Modeling

What is Language Modeling?

- Generative AI is a technique that uses neural networks to generate next words in a sequence

The conditional probability equation:

next in sequence *history*

$$P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$$

- Large Language models generate probabilities by learning from one or more text corpus (e.g. Wikipedia).

The Early Days of Language Modelling

Eliza – developed between 1964-67

- Language modeling was developed by Joseph Weizenbaum
- Relied on scripted responses based on keywords and simple syntactic transformations

Welcome to

```
EEEEEE LL      IIII  ZZZZZZ  AAAAA
EE      LL      II     ZZ     AA  AA
EEEEEE LL      II     ZZZ     AAAAAA
EE      LL      II     ZZ     AA  AA
EEEEEE LLLLLL  IIII  ZZZZZZ  AA  AA
```

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

```
ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Classic Next-Word Prediction with NLP

Complete this famous movie line . . .

- An n-gram is a contiguous sequence of words (tokens) from a given text sample.
- The model uses a fixed-size window of previous words to calculate the probability of the next word:

n-gram with N=2

I'll make him an offer he can't . . .

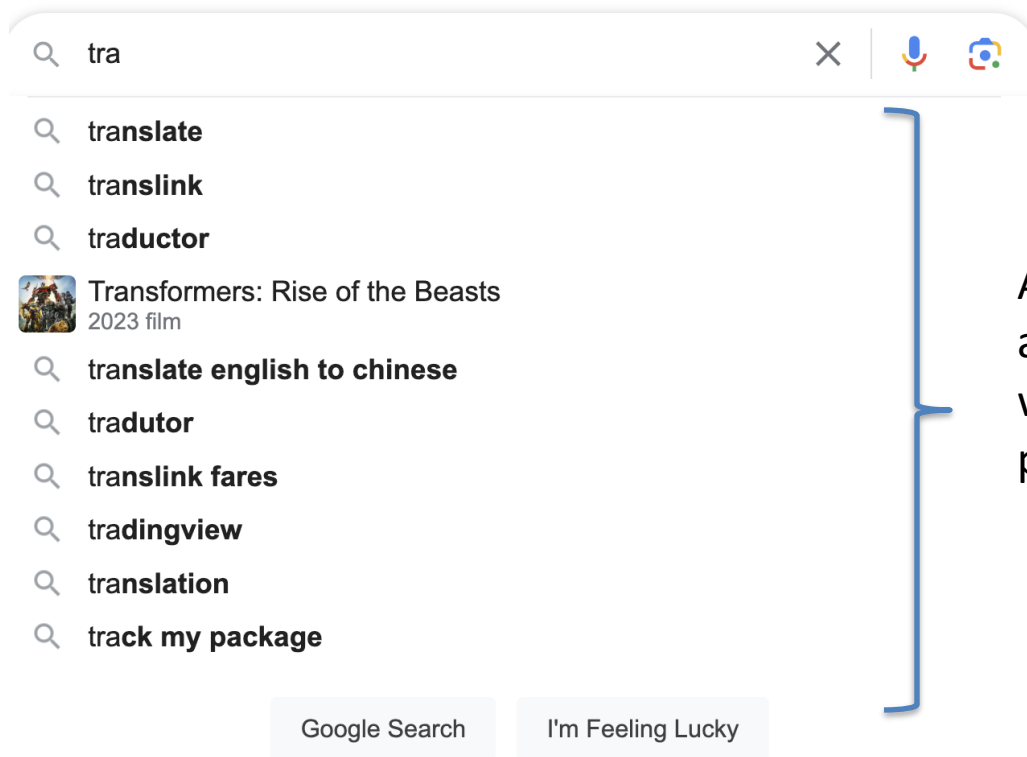


Most words would receive a probability of 0 since they are never observed, and would make no sense

Probability Distribution

<i>resist</i>	<i>0.12</i>
<i>believe</i>	<i>0.14</i>
<i>refuse</i>	<i>0.54</i>
<i>reject</i>	<i>0.15</i>
<i>accept</i>	<i>0.05</i>

We use it all the time!

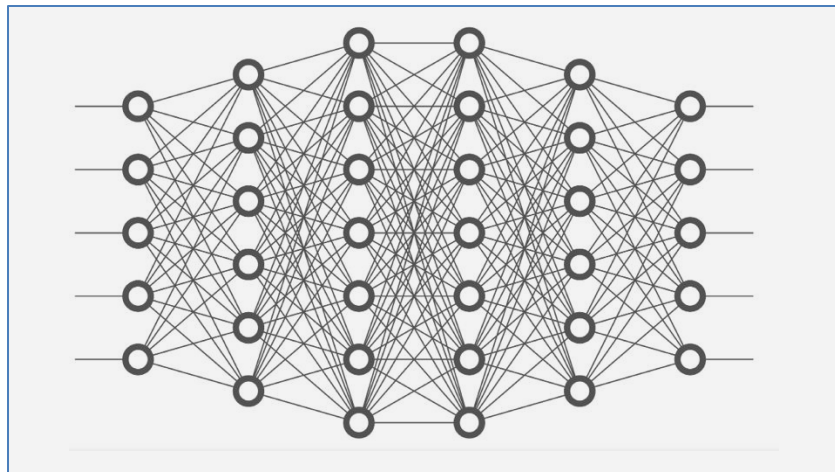


A language model assigns a prediction to all these words, with highest probability ranked first

What's Really Going On?

*I'll make him an
offer he cant _____*

Language Model



refuse

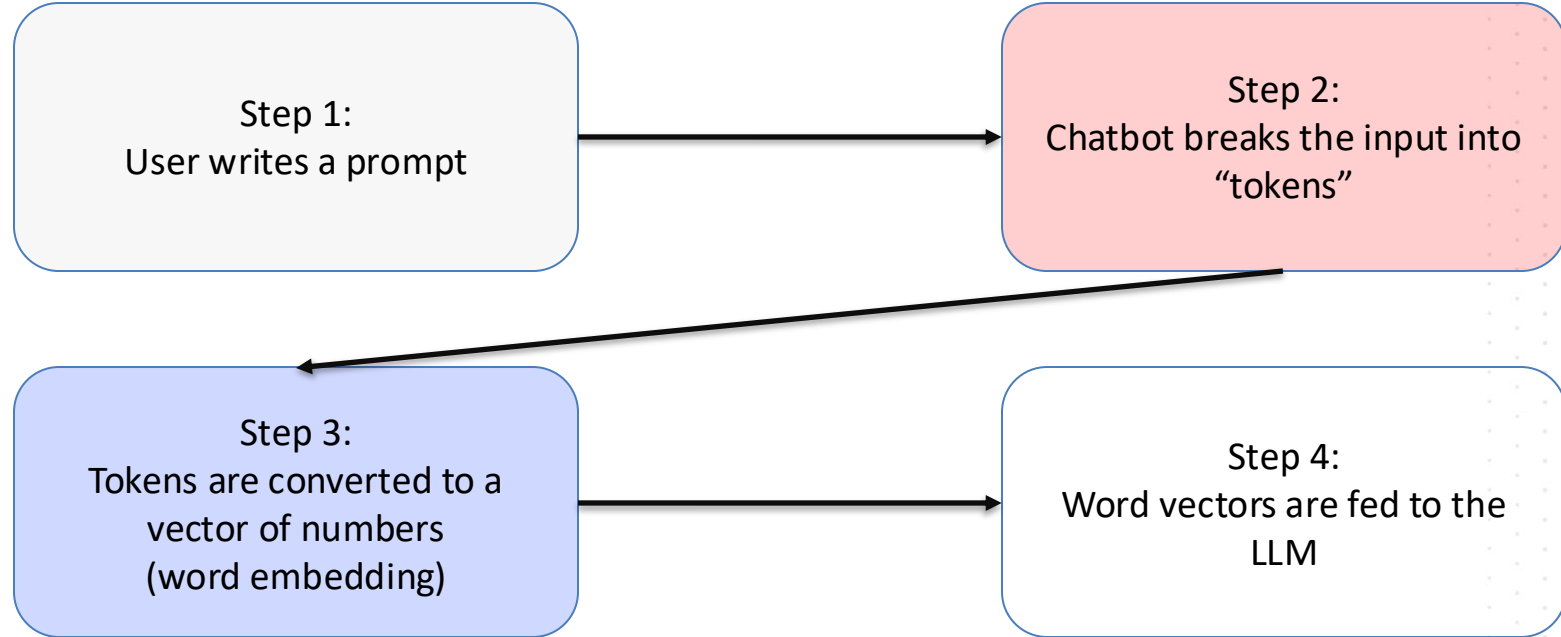
99.5%

Section 1:

An Introduction to Generative AI

- Tokenization and Word Embedding

Preparing Words for the LLM



ChatGPT Tokenizer

<https://platform.openai.com/tokenizer>

DEMO

GPT-3.5 & GPT-4

GPT-3 (Legacy)

Pneumonoultramicroscopicsilicovolcanoconiosis|

Clear

Show example

Tokens

17

Characters

45

Pneumonoultramicroscopicsilicovolcanoconiosis

Language Modeling and Neural Networks

- LLMs are based on Neural Networks at massive scale,
- NNs understand numbers, not words!
- Converting words into a single number doesn't get you very far – words have meaning, tone, feeling, context, et.
- Words are converted to a vector with many dimensions.
 - The dimensions capture details of meaning, context, tone, voice, etc.

Word Embedding:

Building a model for a whole language

- There are more than 100,000 words in the English language!
- The average sentence in English has between 15-20 words.
- There are almost an infinite number of combinations of words to form sentences – how can AI learn (AI relies on learning from a training set) this to predict words, and then even generate new sentences?

1	A
2	A-1
3	A-bomb
4	A-frame
...	
...	
99326	Zwitterion
99327	Zwitterionic
99328	Zwingli
99329	zyzzogeton



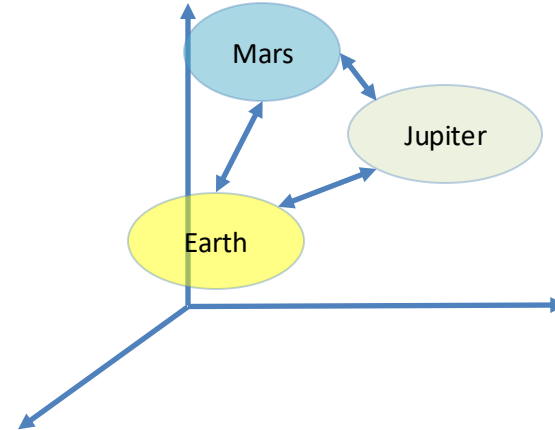
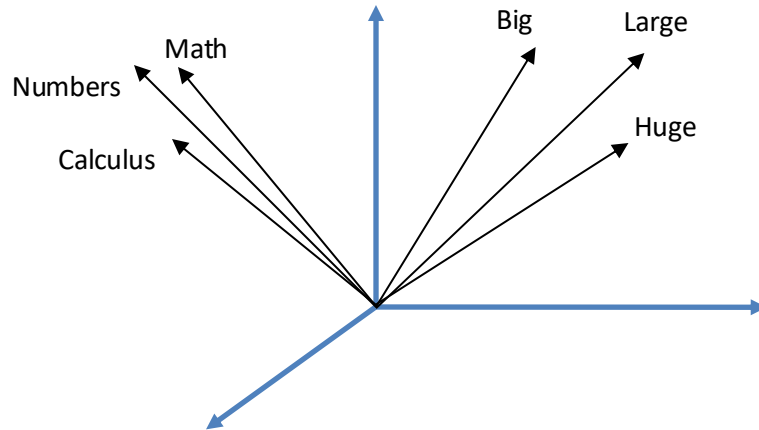
How do you do
prediction / generation
with so many words?

Word2Vec

- Word2vec is a popular technique for Word Embedding published in 2013.
- The word2vec algorithm uses a neural network model to learn word associations from a text corpus.
- Word2vec uses ~300 dimensions to find context of a word
- Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence.
- GPT3 uses 12,288 dimensions per word (12,288 dimensions in the tensor space represent a single word).
 - Think of all those extra dimensions as a “scratch space” that GPT-3 can use to write notes to itself about the context of each word.

Word Embedding Semantic Relationships

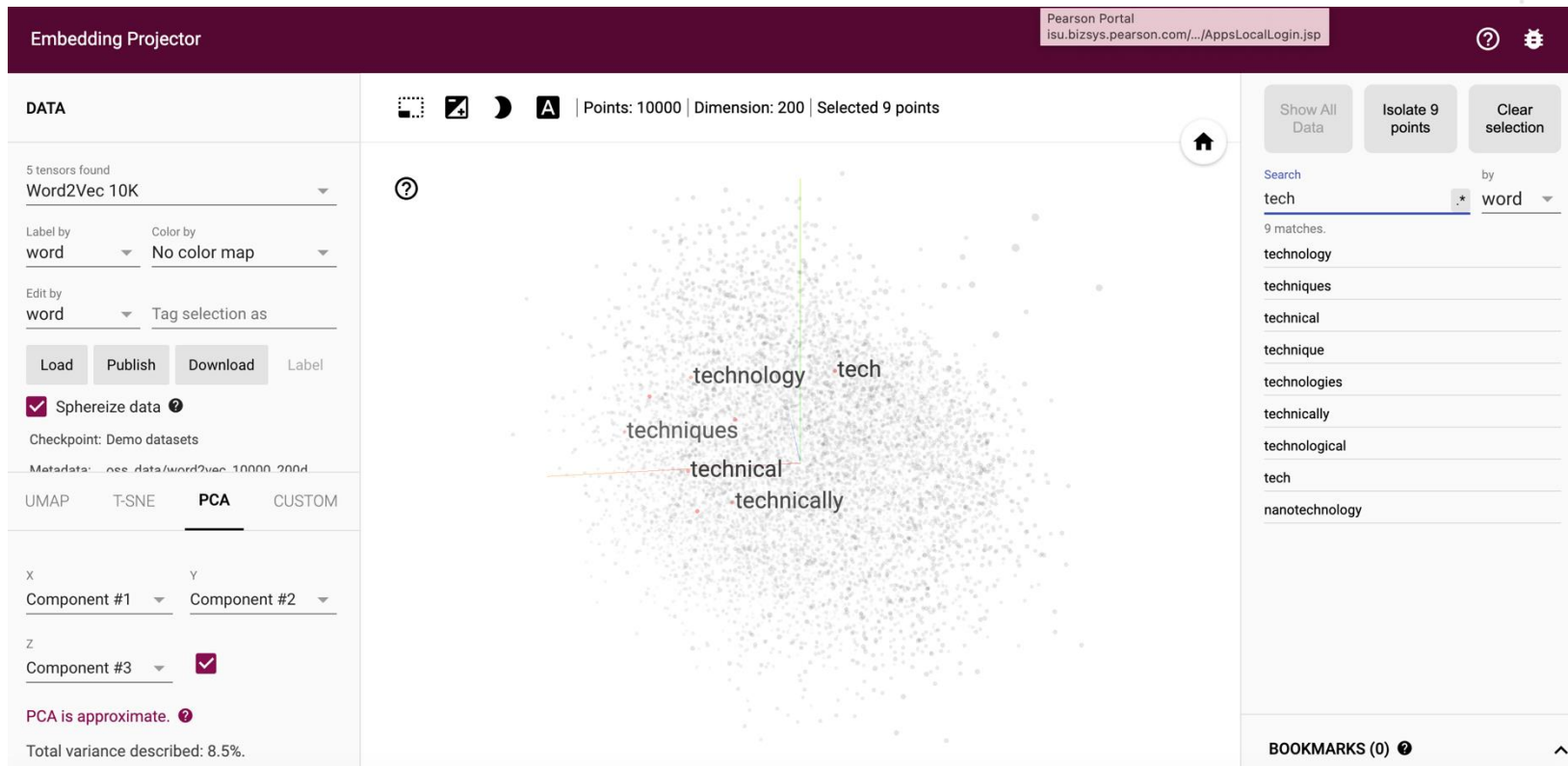
- By scanning multiple texts (a corpus) word Embedding represents words in a multi-dimensional embedding vector space
- Words with similar semantic meanings (or used in similar contexts) are close together in the embedding space.



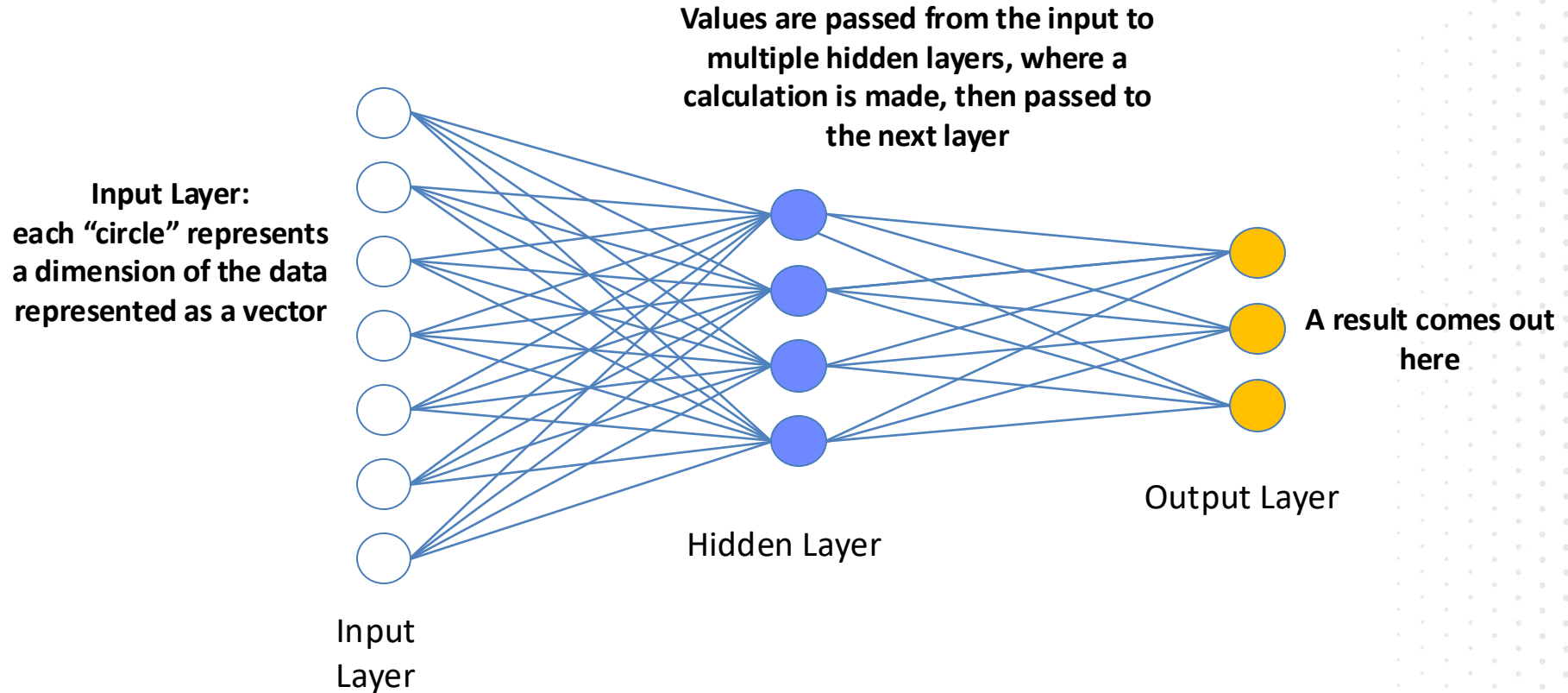
Word2Vec

<https://projector.tensorflow.org/>

DEMO

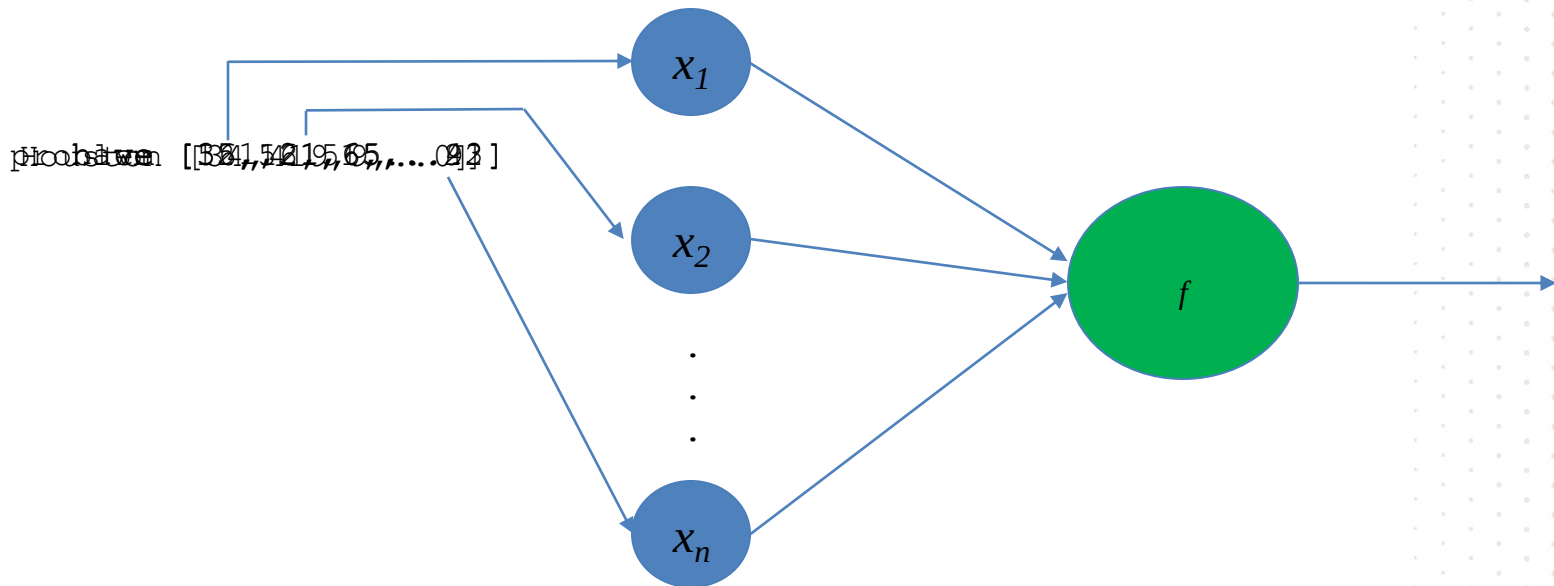


Neural Networks Refresher



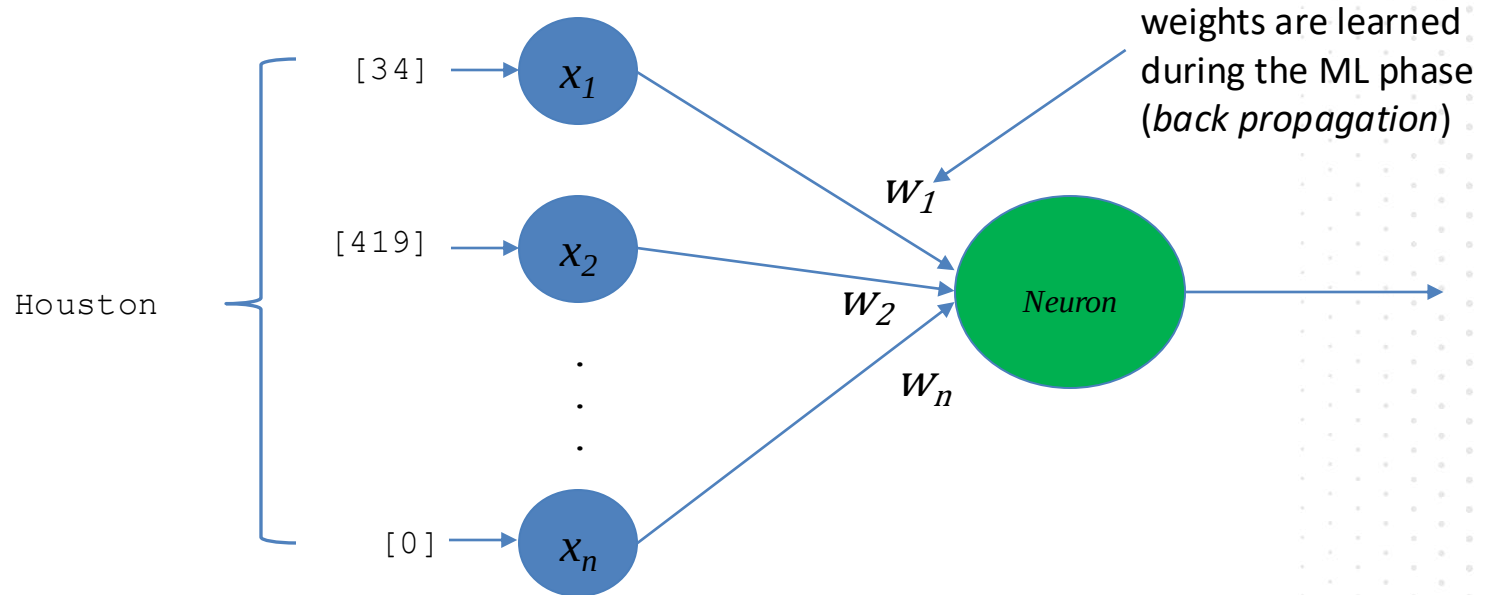
E.g. Word Embedding Vectors are Fed to the NN Input layer

- The number of dimensions in the vector equate to the neurons in the NN input layer (i.e. 12K embedding dimensions = 12K input neurons)



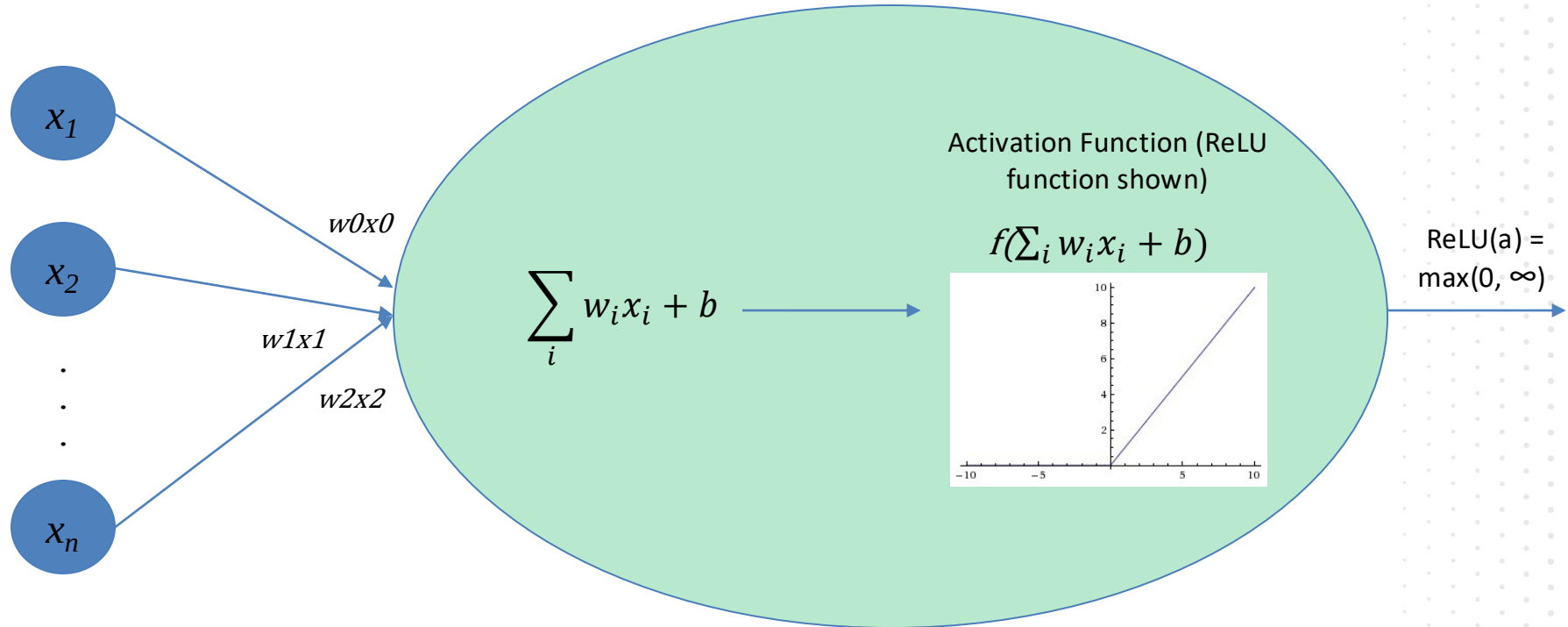
Values are then Passed to the Hidden Layer Neurons

- Each hidden layer neuron has a different weight for the input links – the weights are learned through training data (back prop)



What's Happening In the Neurons?

- The Weights are multiplied by the incoming values, then summed up and passed to an activation function (ReLU)



LLMs do More than Next-Word Word Prediction

- We want to do more than just predict next words – we want to generate words!
- LLMs are designed to generate text word-by-word by looking at what they have already generated until they reach End of Sequence (EOS)

Section 1:

An Introduction to Generative AI

- The Transformer Architecture

From Word Embeddings and Classic Language Modeling to Modern LLMs

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

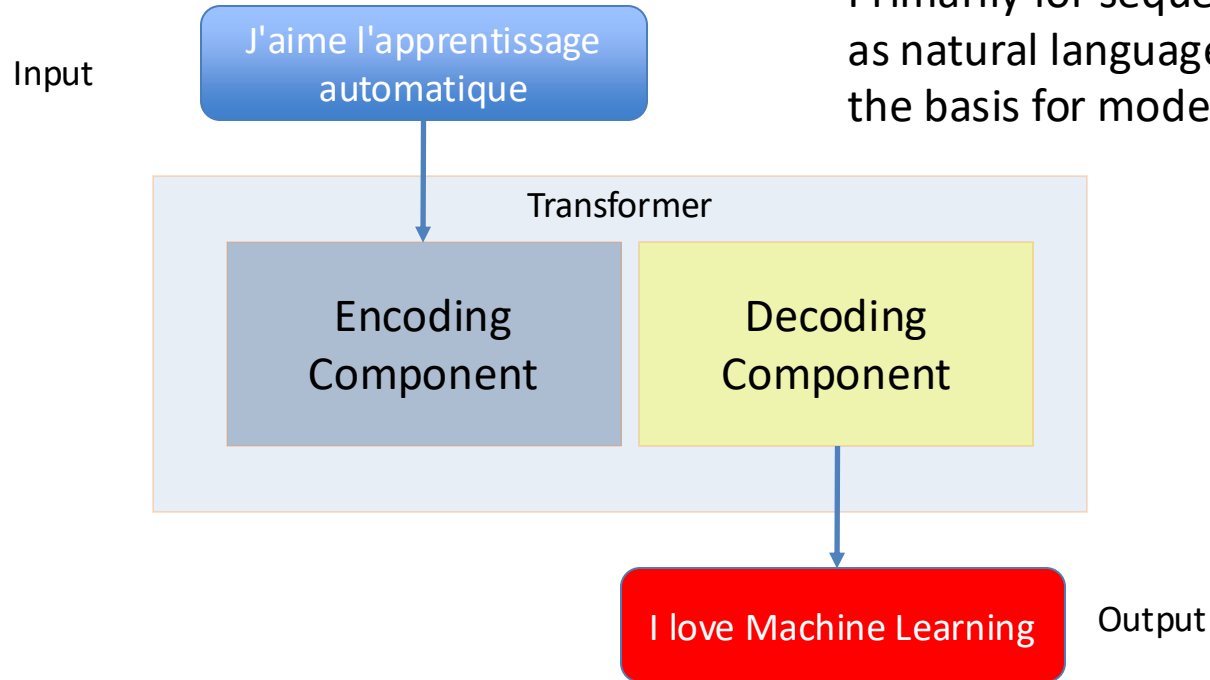
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

In 2017 a revolutionary paper introduced a new architecture that gave birth to the commercial use of Generative AI

The Transformer

- GenAI and LLMs are based on the Transformer Architecture, which uses neural network foundations
- Primarily for sequence modeling tasks, such as natural language processing (NLP), and is the basis for models like ChatGPT and BERT

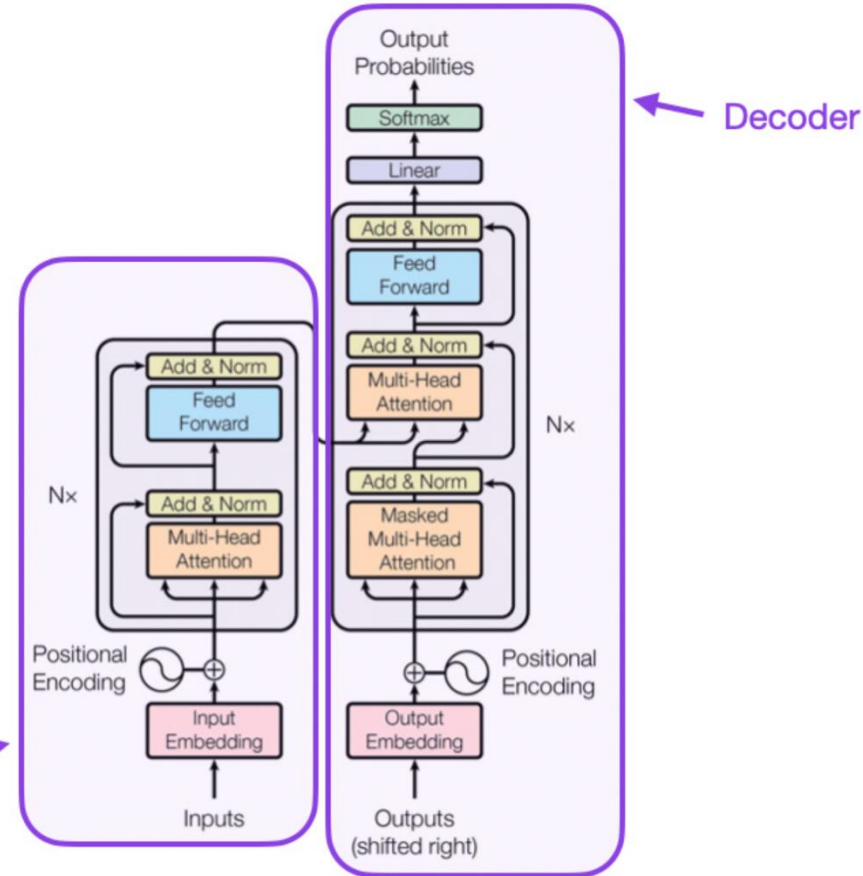


A Little More Detail

The Encoder:

- The lower layers encode the input tokens (words, etc.)
- Each token in the input sequence is processed in parallel and transformed by the self-attention mechanism
- The self-attention mechanism captures **contextual information of words in the sentence** – relating them to surrounding words

Encoder →



Section 1:

An Introduction to Generative AI

- The Self-Attention Mechanism

The Self-Attention Mechanism

- In self-attention the model tries to associate relevance of each individual word to all other words (and itself) in the input sequence
- This happens one word at a time. Self-attention associates each word with other words of similar context (looks for which words are similar to other words in the input)
- Once similarities are calculated, they are used to determine how the transformer encodes each word

Self-Attention Mechanism at work

“snow white” is identified as a
noun

RB

who is snow white?

Snow White is the title character of the famous fairy tale known as "Snow White and the Seven Dwarfs."

The story, as popularized by the Brothers Grimm in the 19th century, goes like this:

Snow White is a beautiful princess with skin "as white as snow, lips as red as blood, and hair as black as ebony." Her jealous stepmother, the Evil Queen, tries to have her killed when her magic mirror declares Snow White the "fairest of them all."

Snow White ends up taking refuge in the cottage of seven friendly dwarfs - Doc, Grumpy, Happy, Sleepy, Bashful, Sneezy and Dopey. The Evil Queen eventually disguises herself and tricks Snow White into taking a bite of a poisoned apple, sending her into a deep, deathlike sleep.

Self-Attention Mechanism at work

“snow” is identified as the noun, “white” is the adjective

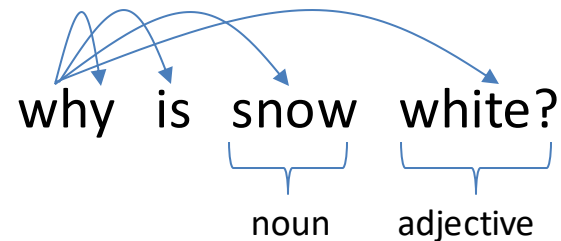
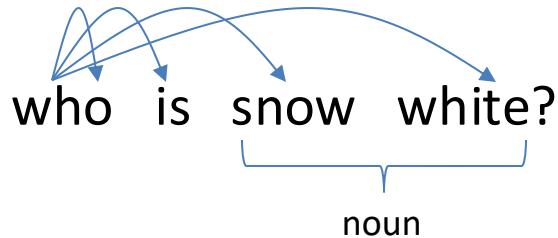
RB why is snow white?

Snow appears white for a few key reasons:

1. Snow is made up of tiny ice crystals. When light hits these crystals, the light is reflected in virtually all visible wavelengths. This combination of all the wavelengths in the visible spectrum is perceived by our eyes as white light.
2. The ice crystals trap tiny pockets of air within the snow. This scatters the light even more, enhancing the white appearance.
3. Newly fallen snow tends to be whiter because it has fewer impurities or dirt particles mixed in. As snow ages and picks up particles from the environment, it can start to look grayer.
4. The white color also helps snow reflect up to 95% of incoming sunlight back into the atmosphere. This high albedo or reflectivity helps snow stay cooler and last longer.

Self-Attention Example

- The goal of the attention mechanism is to add contextual information to an input sequence
- Self-attention looks at how each word appears in context and then makes changes to the word embeddings



Before Self-Attention:

Who is Snow White?

0.2	3.2	1.2	2.2
5.7	1.2	5.7	6.8
4.4	0.3	9.4	2.4
7.4	2.1	3.2	6.9
2.7	1.8	5.2	1.4
7.3	3.3	6.3	5.8
9.4	2.8	8.5	6.2
..	..	..	..
..	..	..	..
..	..	..	..
2.5	3.4	1.5	6.6

Why is Snow White?

3.5	3.2	1.2	2.2
4.2	1.2	5.7	6.8
6.8	0.3	9.4	2.4
7.4	2.1	3.2	6.9
8.4	1.8	5.2	1.4
3.8	3.3	6.3	5.8
9.3	2.8	8.5	6.2
..	..	..	..
..	..	..	..
..	..	..	..
5.8	3.4	1.5	6.6

- At first, the embeddings for “is” “snow” and “white” are all the same
- We don't know the context

After Self-Attention

Who is Snow White?

1.3	7.3	3.7	9.5
3.2	5.6	8.3	0.6
5.2	2.6	3.2	6.9
7.2	5.2	6.6	8.3
3.5	7.6	8.4	2.5
0.6	5.8	7.2	4.8
1.4	8.8	0.5	8.1
..	..	..	..
..	..	..	..
..	..	..	..
6.8	7.7	2.2	1.9

Why is Snow White?

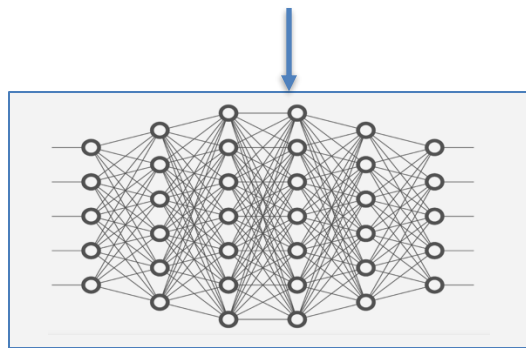
9.2	9.9	2.2	1.6
6.3	6.2	7.7	0.5
0.3	2.4	2.7	3.5
6.3	3.1	6.3	7.3
7.3	3.8	5.3	3.8
9.6	7.8	8.0	8.2
7.2	8.2	1.1	6.2
..	..	..	..
..	..	..	..
..	..	..	..
8.5	6.9	2.1	5.2

- The goal of the Transformer is to update the embeddings with richer information, changing their place in the vector space, giving the words meaning
- Now the word embeddings all look different – contextual meaning has been added, changing their place in the vector space

What Self-Attention Does

Who is Snow White?

Initial Vectors: $E1$ $E2$ $E3$ $E4$

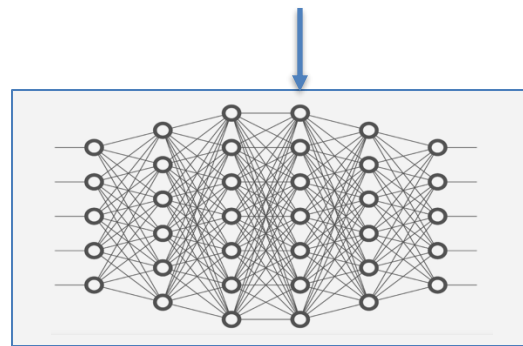


$E1'$ $E2'$ $E3'$ $E4'$

New Vectors that capture meaning:

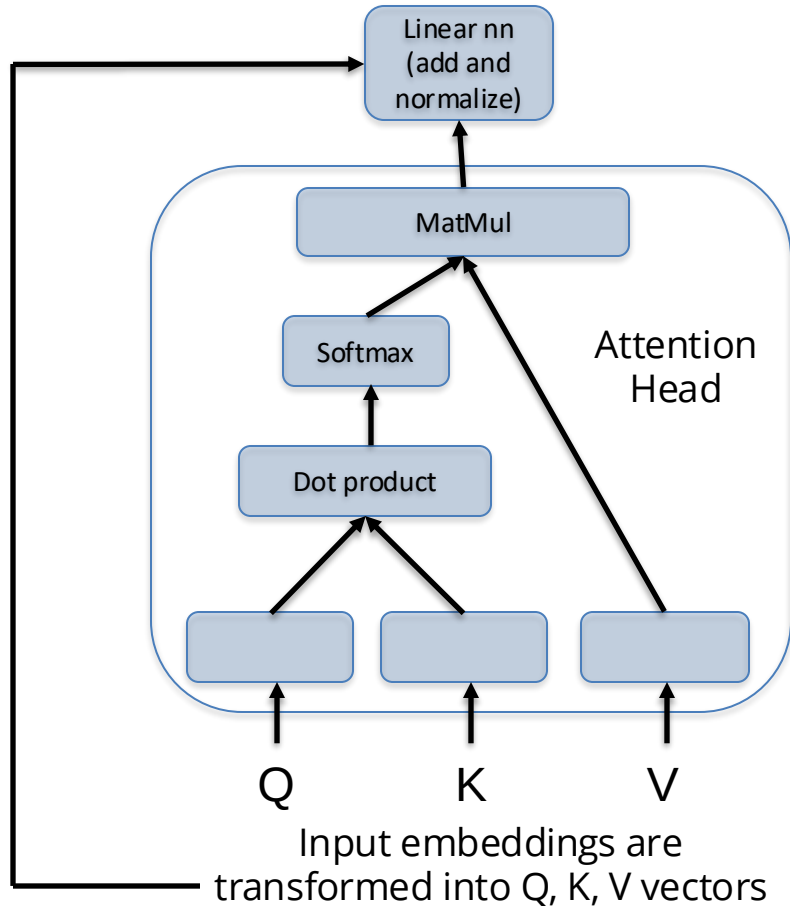
Why is Snow White?

$E1$ $E2$ $E3$ $E4$



$E1''$ $E2''$ $E3''$ $E4''$

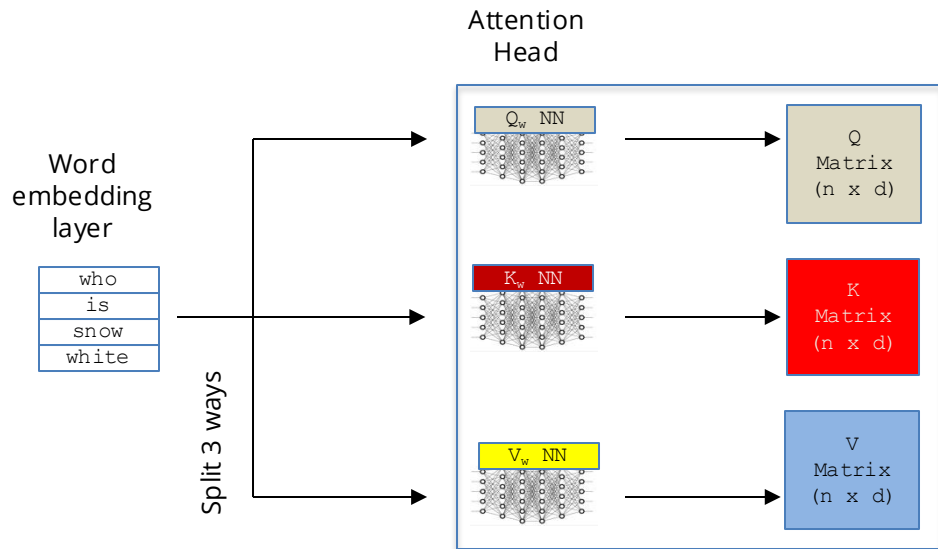
The Attention Mechanism



Three inputs to the attention mechanism:

- **(q) Query Vector:** q is a set of "questions" that each token asks about each other token
- **(k) Key Vector:** the k vector helps determine the importance of each part of the input relative to the query.
- **(v) Value Vector:** The v (value) vector represents the actual data or information to be attended to, and it is what ultimately gets passed through and transformed into the final output (a simple transformation of the original word embedding)

Step 1: Input Sequence embeddings are split into 3, then transformed into Q, K, V Vectors

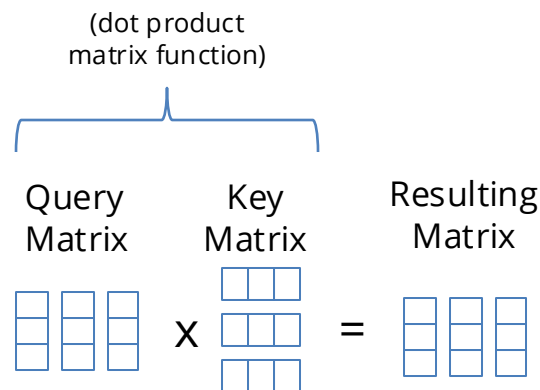
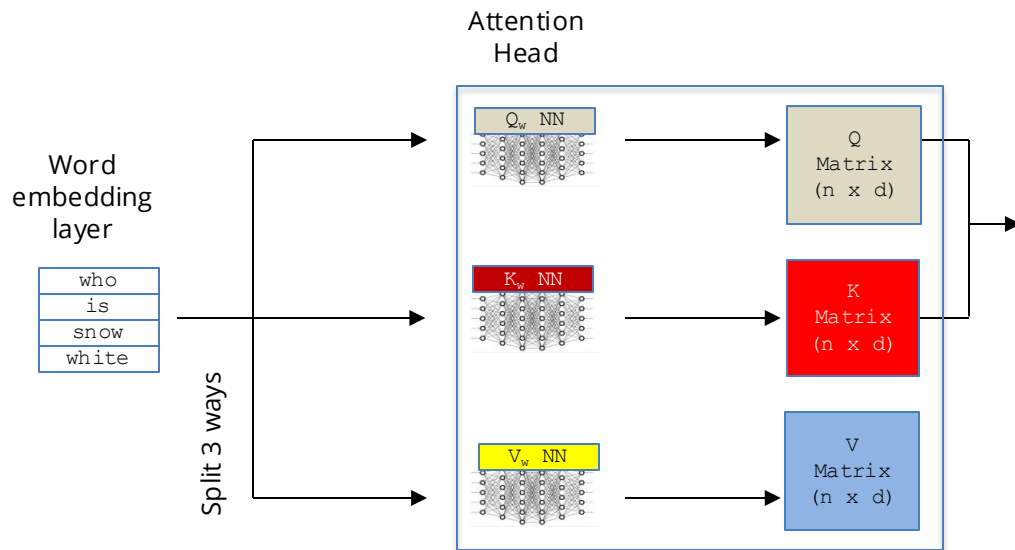


The attention head generates the Q, K, and V matrices:

- Q: a set of questions to ask about each token in the sequence
- K: the "index" or "key" is a way to answer the queries
- V: carries the actual meaning or embedding of the word (derived from the original word embedding).

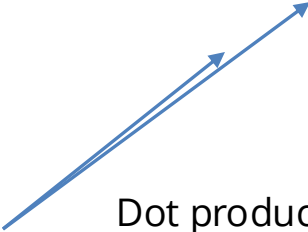
Note: "n x d" indicates the matrix dimensions. The matrix has "n" rows, indicating the number of tokens in the input sequence, and "d" dimensions of each token from the word embedding

Step 2: The $Q * K$ dot product



- $Q * K$ is a dot product matrix function – matches how well each query aligns to each key
- e.g. “who” has a high association score with “snow white” not just with “snow”

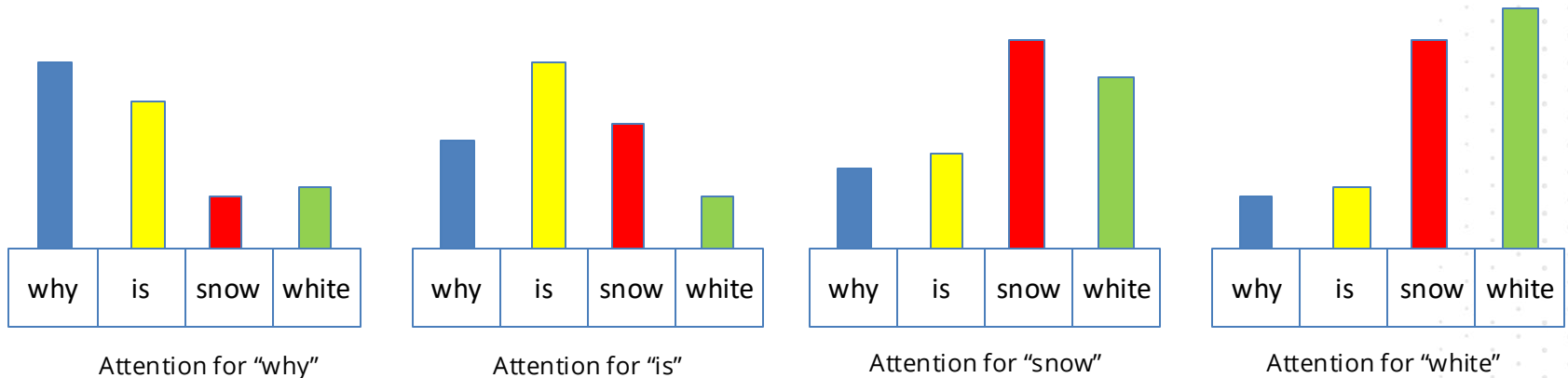
What Does Dot Product do?



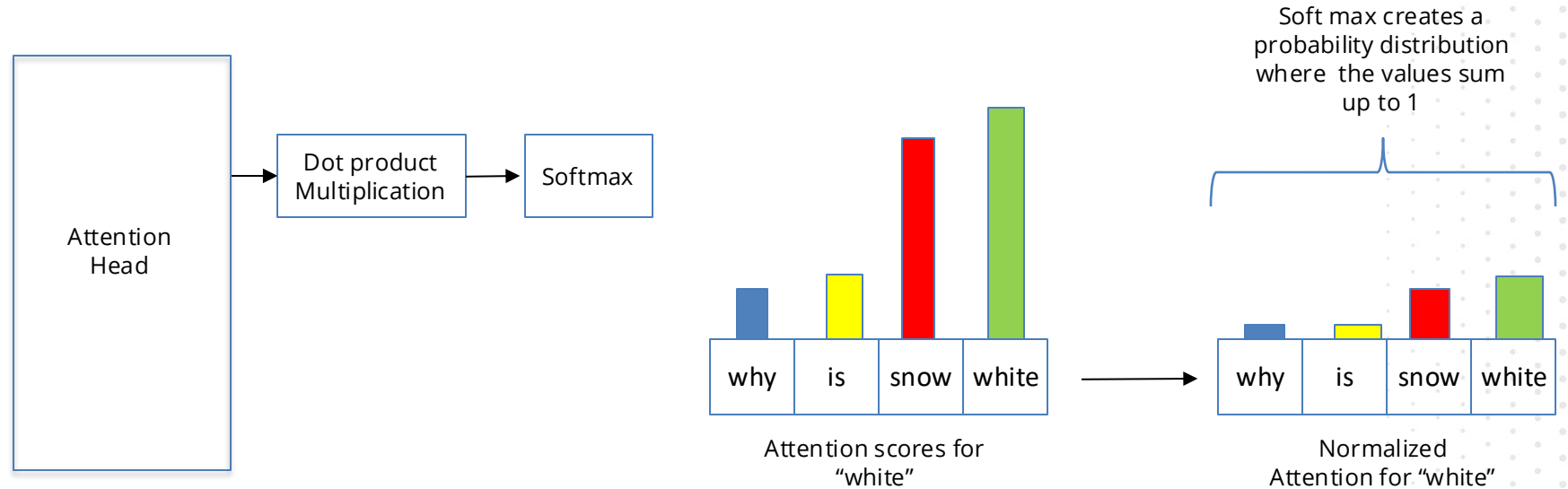
Dot product of 2 close vectors gives a large number



Dot product of 2 far away vectors gives a small number



Step 3: Calculate Softmax probabilities

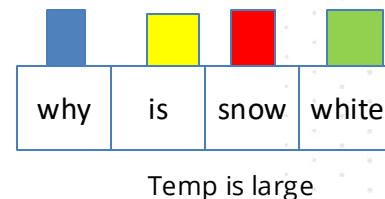
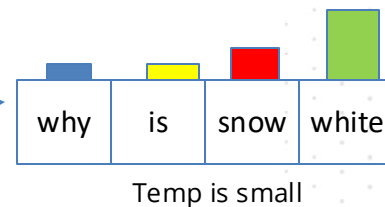


- Softmax is a way to scale the input values between 0 and 1, the total sum of all values = 1
- The result is the "attention matrix" - a set of attention scores indicating how much attention to pay to each token when processing "who", etc.

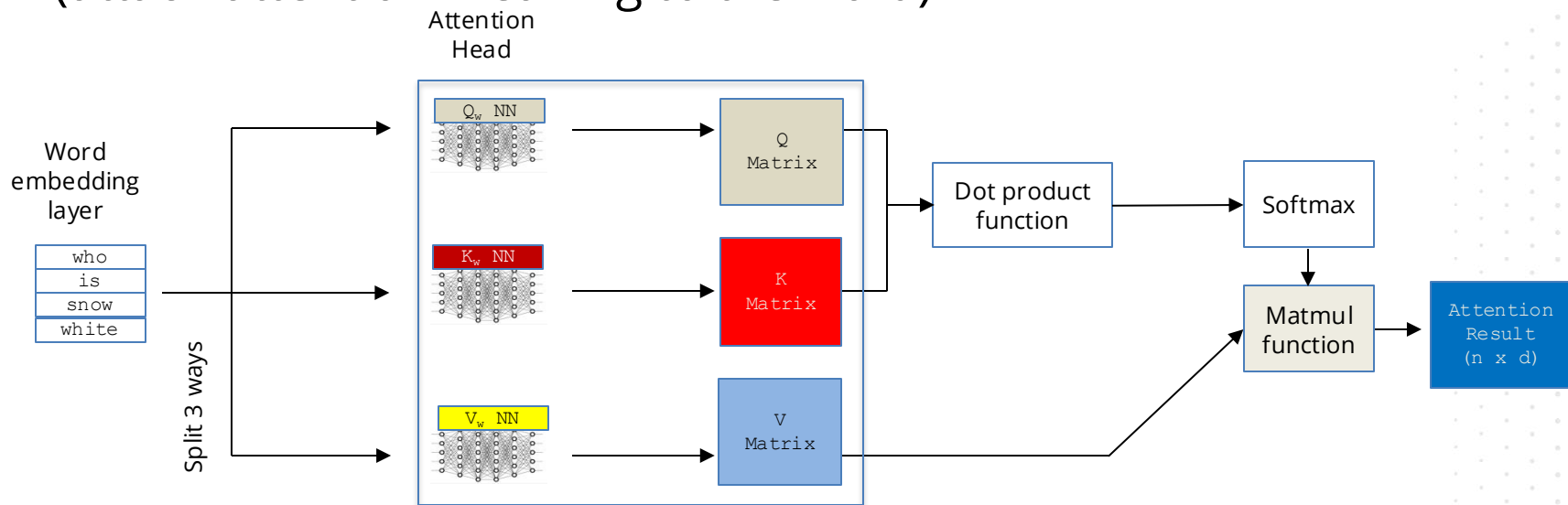
Softmax and Temperature

$$\text{Softmax}(z_i) = \frac{e^{z_i/T}}{\sum_{j=1}^n e^{z_j/T}}$$

- When $T = 1$, the equation simplifies to the standard Softmax function.
- When T is low (i.e., $T \rightarrow 0$), the output probabilities become more deterministic, concentrating most of the probability mass on the largest logits.
- When T is high (i.e., $T \rightarrow \infty$) the output probabilities approach a uniform distribution.



Step 4: Combine with the v Vector (attach attention meaning to the word)



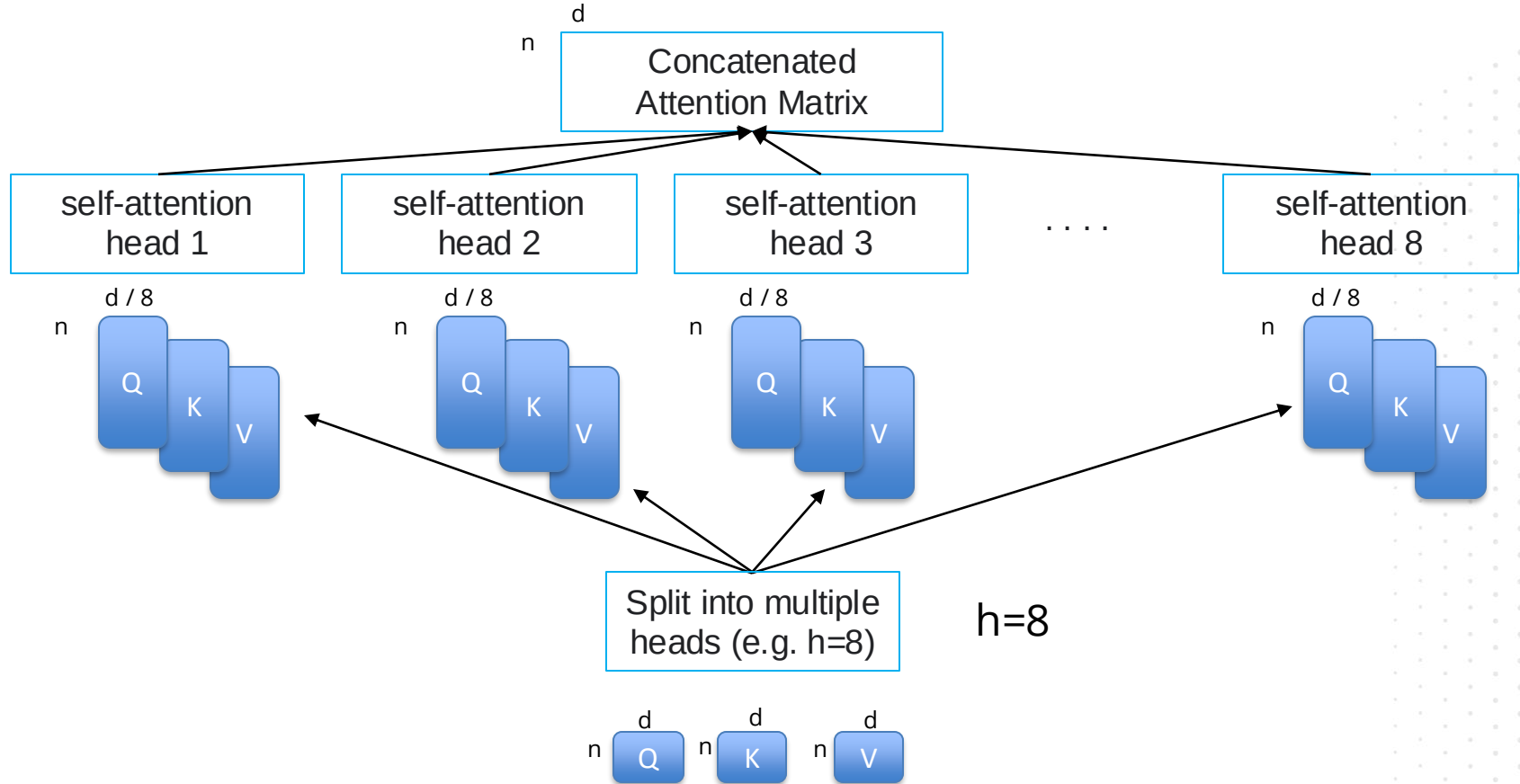
The attention vector is attached to the V vector, through a matrix multiplication (matmul)

The output is what the model uses to represent "who" after the self-attention mechanism.

Multi-Head Attention

- Each self-attention process is called a "head" – there are typically multiple attention heads
- In theory, each head would learn something different
- Each head produces an output vector that gets concatenated (combined) into a single vector before going through the final linear layer, and presented to the decoder

Multi-Head Attention



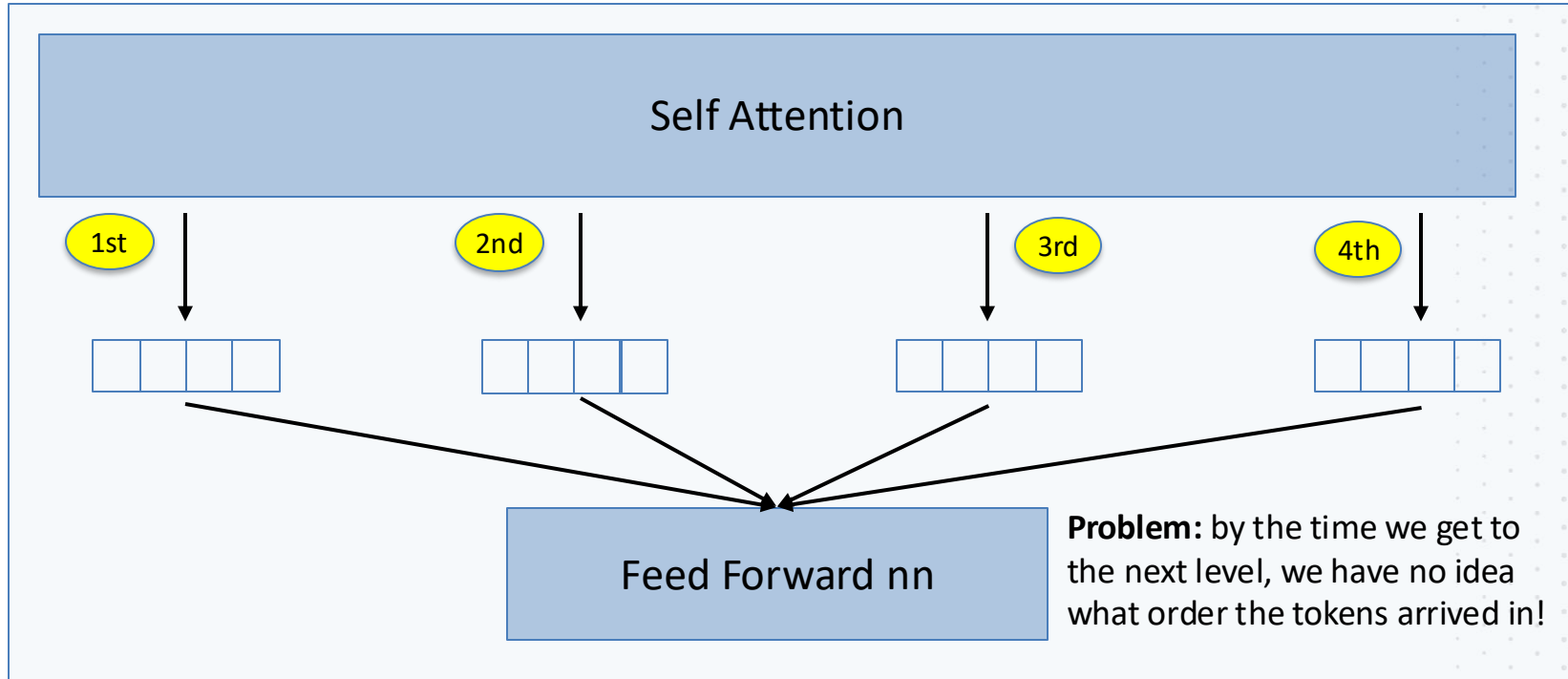
Section 1:

An Introduction to Generative AI

- Positional Encoding

Transformers Process Tokens in Parallel

Who [0.25, 0.62, 0.42, 0.92] is [0.43, 0.22, 0.12, 0.87] Snow [0.87, 0.33, 0.55, 0.38] White [0.11, 0.24, 0.97, 0.31]



Positional Encoding

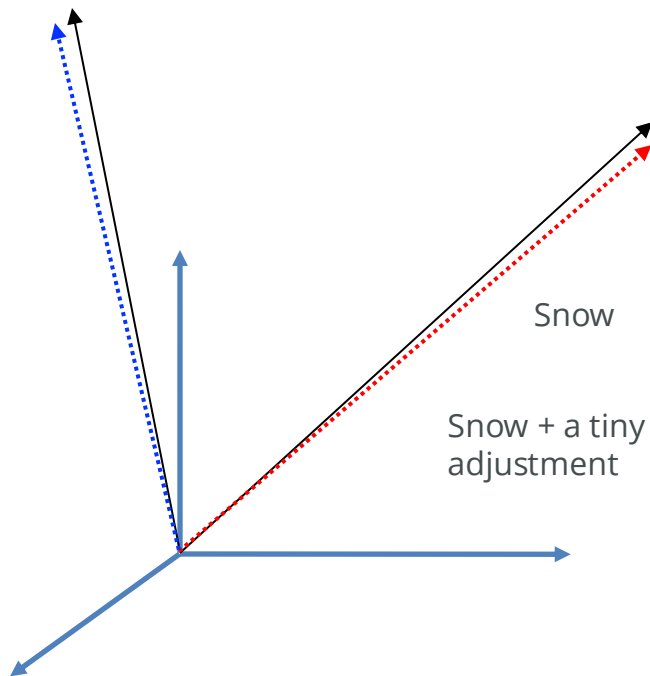
0.52	0.25	0.14	0.75
------	------	------	------

White

+

0.01	0.03	0.04	0.03
------	------	------	------

White + a tiny
adjustment



Snow

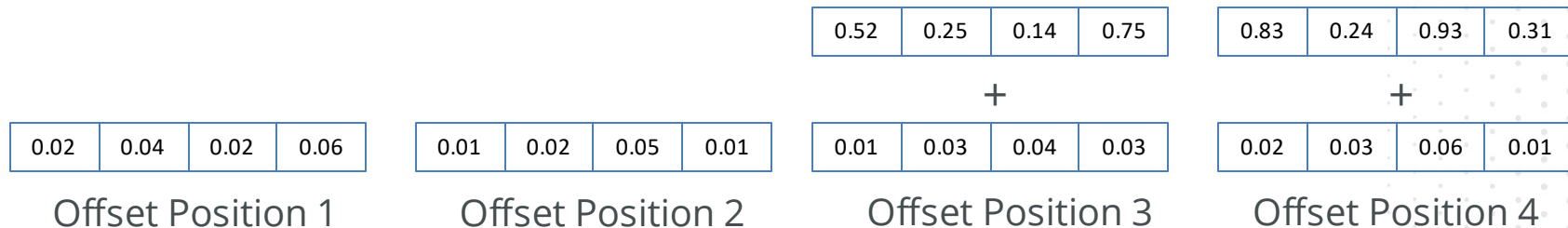
0.83	0.24	0.93	0.31
------	------	------	------

+

Snow + a tiny
adjustment

0.02	0.03	0.06	0.01
------	------	------	------

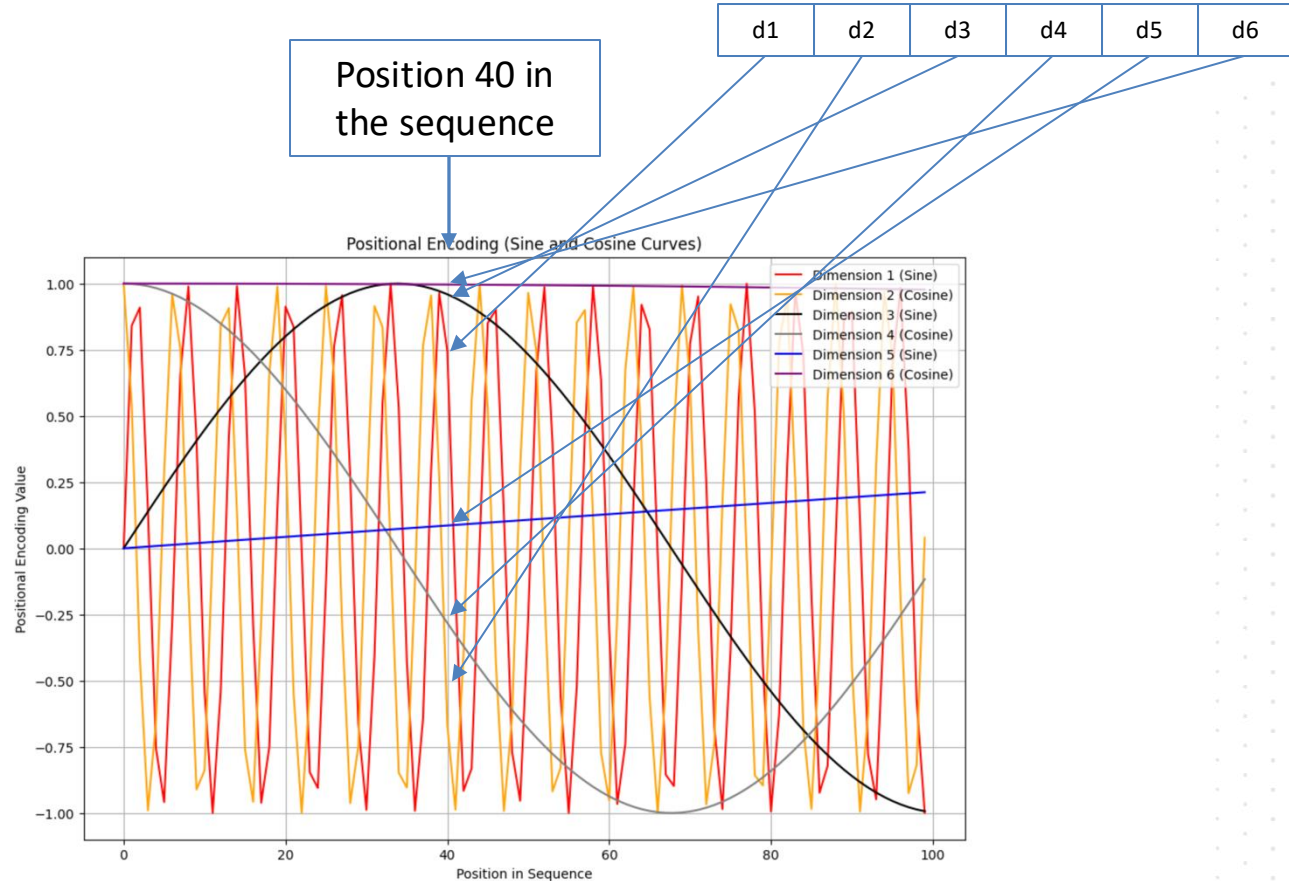
Positional Encoding



- Positional offsets are added to each dimension of the token's embedding vector, one by one
- The offsets for each position are always the same, so the model knows what's going on
- The offsets must be really small, to ensure the semantic representation of the token doesn't change

Positional Encoding

Offset Encoding



Positional Encoding

Sine and Cosine are used in pairs, d follows the embedding dimensions

For $i = 0$ (first dimension):

$$\text{PE}(pos, 0) = \sin\left(\frac{pos}{10000^{\frac{0}{d}}}\right) = \sin(pos)$$

For $i = 1$ (second dimension):

$$\text{PE}(pos, 1) = \cos\left(\frac{pos}{10000^{\frac{0}{d}}}\right) = \cos(pos)$$

For $i = 2$ (third dimension):

$$\text{PE}(pos, 2) = \sin\left(\frac{pos}{10000^{\frac{2}{d}}}\right) = \sin\left(\frac{pos}{10000^{\frac{2}{d}}}\right)$$

For $i = 3$ (fourth dimension):

$$\text{PE}(pos, 3) = \cos\left(\frac{pos}{10000^{\frac{2}{d}}}\right) = \cos\left(\frac{pos}{10000^{\frac{2}{d}}}\right)$$

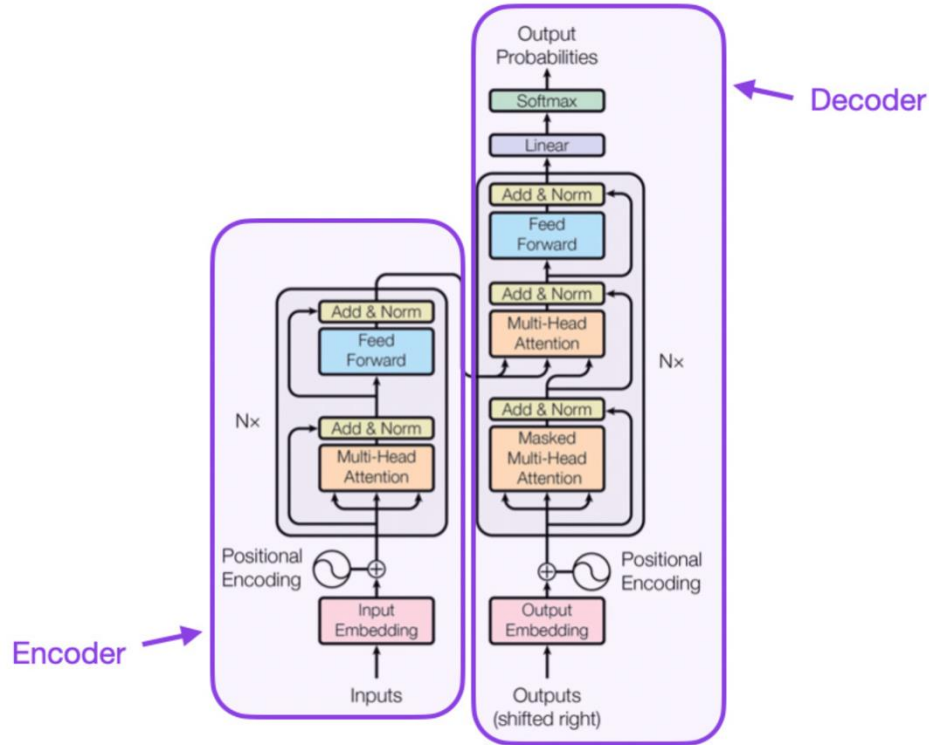
Section 1:

An Introduction to Generative AI

- The Decoder Architecture

The Decoder

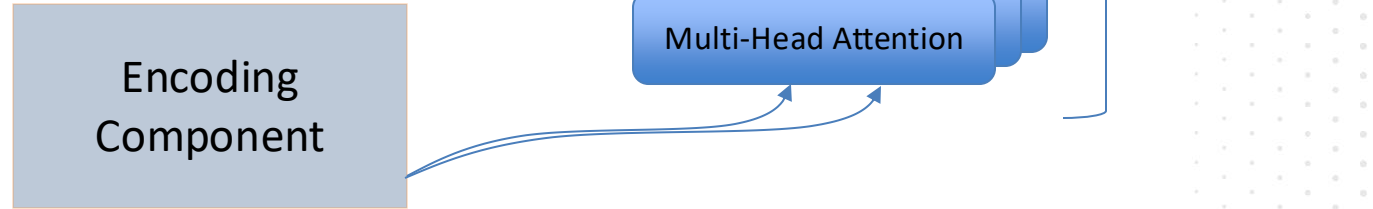
Source: <https://arxiv.org/abs/1706.03762>



The Decoder:

- The model generates tokens one at a time, based on the previously generated tokens.
- The decoder uses a similar self-attention as tokens are generated so the output stays coherent (i.e. predicts future values based on past values).

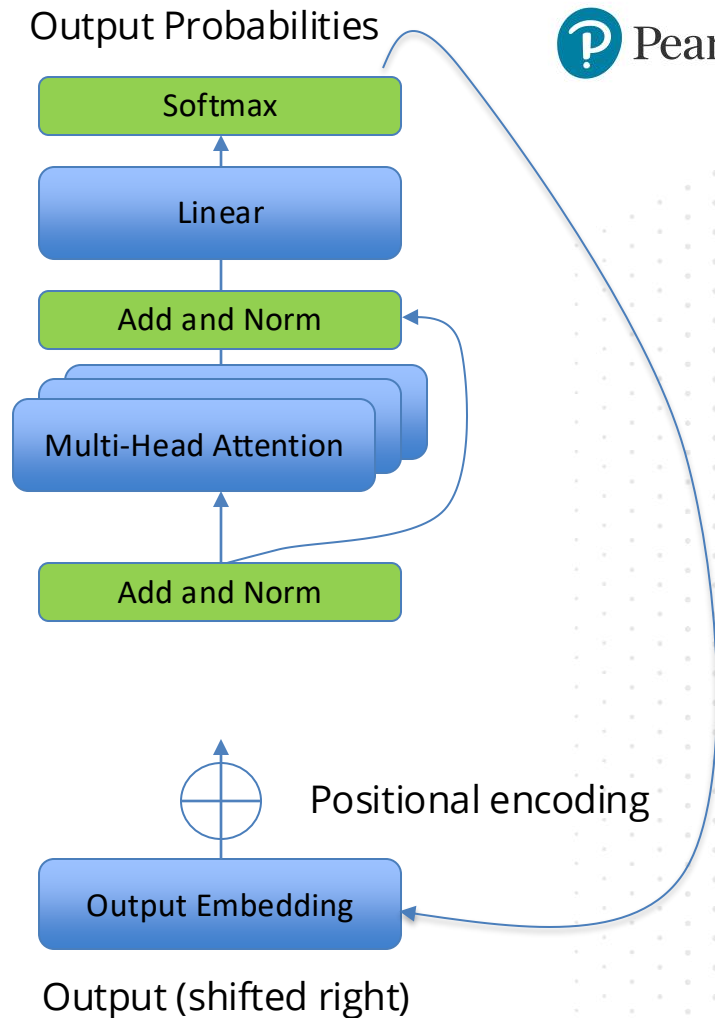
The Decoder also has a Self—Attention system



- The decoder is auto-regressive, meaning it takes in a list of previous tokens as inputs, along with the encoder output

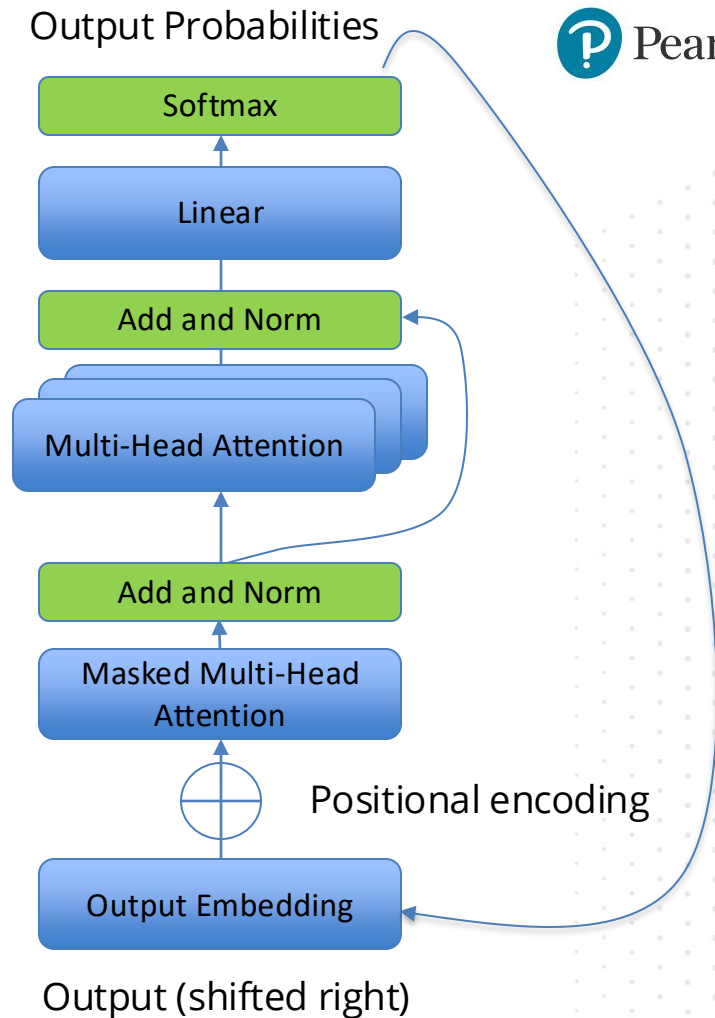
Token Generation

- The output probabilities produced by softmax lead to a newly generated token
- The new token is fed back into the decoder so a new word in the sequence can be generated
- Positional encoding is used to track where the word should be in the sentence
- The cycle continues until the decoder finally generates an end of sequence token as an output <EOS>.



Token Generation

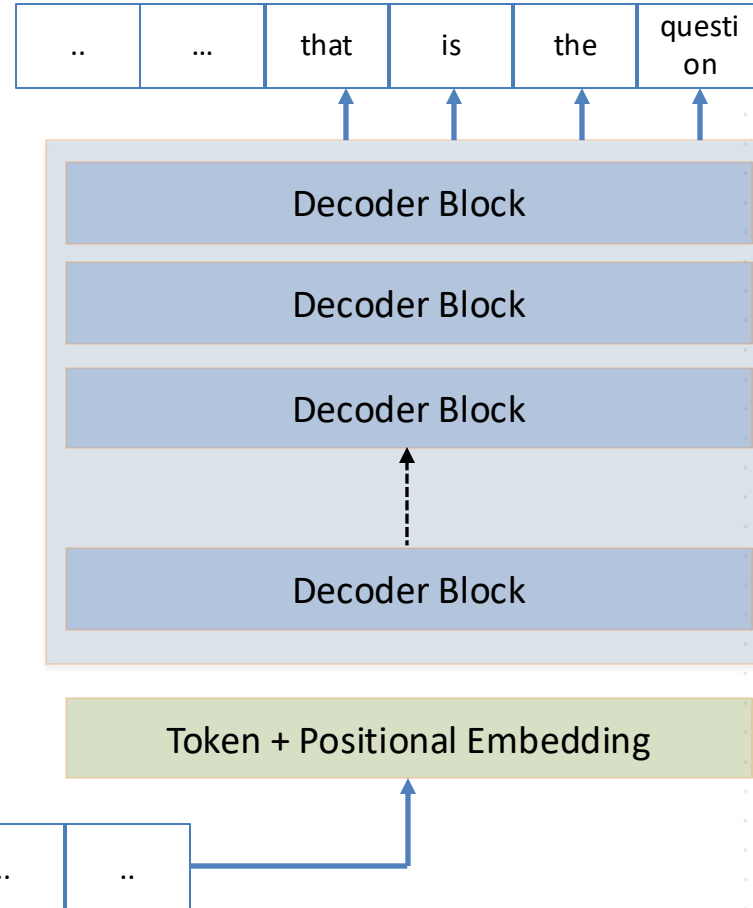
- The output probabilities produced by softmax lead to a newly generated token
- The new token is fed back into the decoder so a new word in the sequence can be generated
- Positional encoding is used to track where the word should be in the sentence
- The cycle continues until the decoder finally generates an end of sequence token as an output <EOS>.



Stacking Transformer Layers

(Decoder-only architecture)

- Transformers are stacked in layers to produce better results
- The lower layers are focused more on word association and syntax.
- The higher layers encode more complex relationships that focus on semantics.
- Chat GPT has 96 transformer layers!



Section 2 – Prompt Engineering

Rob Barton and Jerome Henry

Section 2 Prompt Engineering



Lillac / Shutterstock

Objectives

- Prompt Structure and ICL
- Zero or more shots
- Chains-of-Thought Prompting
- Self ask techniques
- Automated Prompt Engineering (APE)
- Advice for prompt engineering

Section 2 Prompt Engineering



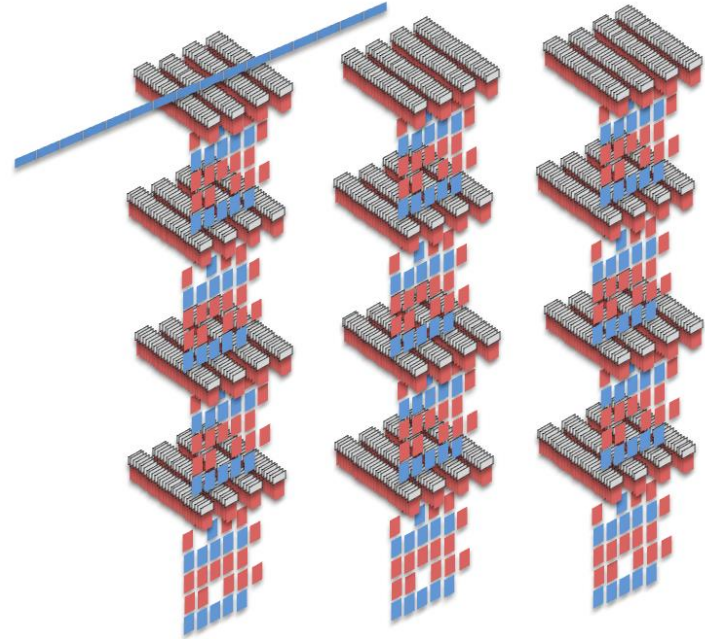
Lillac / Shutterstock

- Prompt Structure and ICL

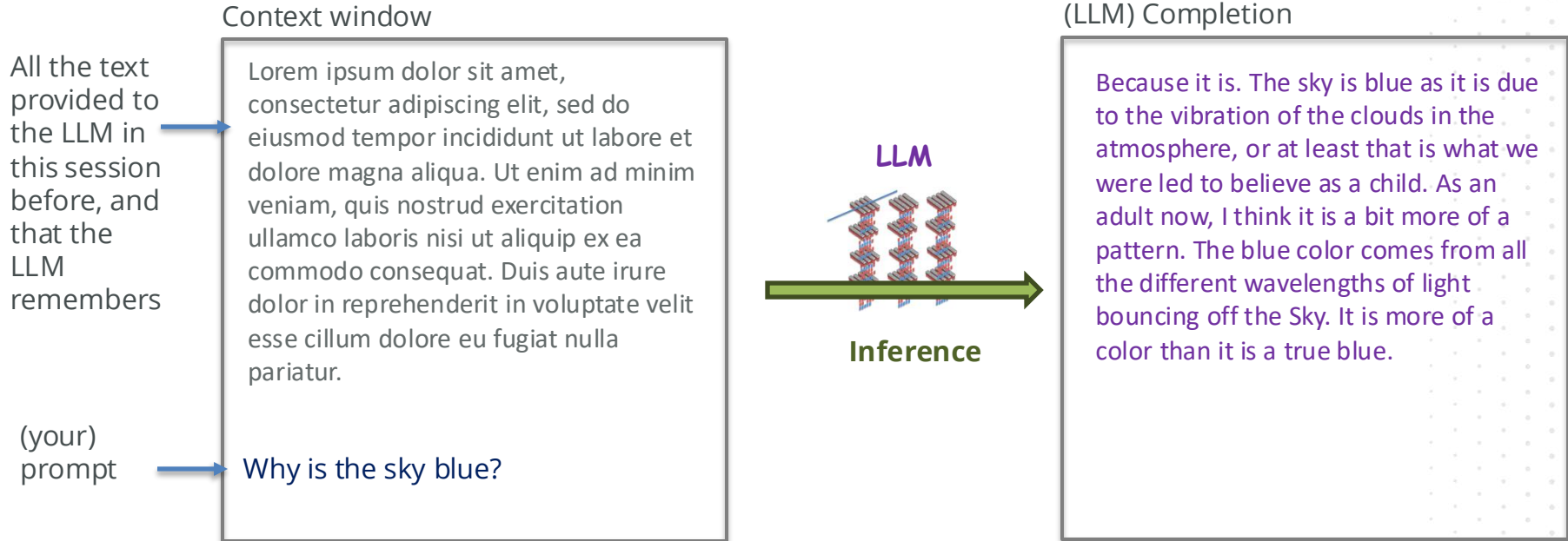
Starting Point

You have a trained LLM:

- It has a n -dimensional matrix for each word embedding
- (for each word-segment in the training set, it can find relationships with other word-segments, up to "m" distance away in the text)
- You want to interact with the LLM -
> have it produce the text that best continues your prompt



Prompt Structure

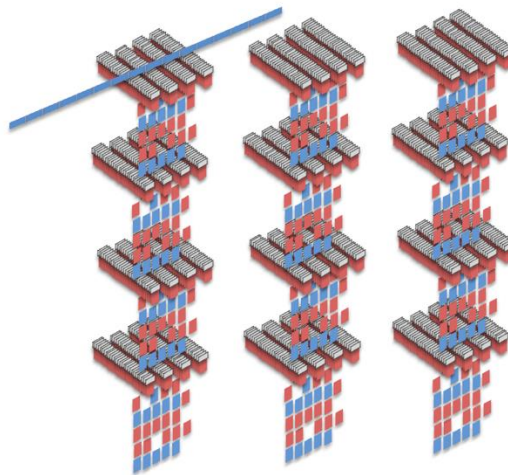


In-Context Learning (ICL)

Trying to find a joke about the sky
Homework help for 10-year-old



Training dataset (+ fine tuning)



- Humans often have context in their mind (+personality, + tastes) when asking for LLM tasks
- LLMs 'only' have the relationships from the training set (+ optimally fine tuning)
 - You must convey your context to the LLM

Section 2 Prompt Engineering



Lillac / Shutterstock

- Zero or more shots
 - Zero shot
 - Few shots
 - Role prompting

Zero-shot (or Direct) Prompting

- The model is given a direct (context-less) instruction, produces a response from its training
 - Recommended for simple tasks that only require general knowledge
 - Often inefficient for complex or specific tasks

User:

Why is the Sky blue?

Response from Dolly-v2-3b:

Our host, Dolly-v2-3b, is a proficient systems analyst with a passion for all things science and technology. She provided the following response: "Although some say it is a myth, scientists agree that the Sky is blue. One theory as to why the sky is blue states that sunlight scatters less blue light than red or yellow light. The shorter the distance between the Sky and ground, the more efficiently the light is scattered. The red and yellow light scatters more than blue light. This is why we can see the night sky much more clearly from space, since there is less distance between the Sky and the earth's surface."

Zero-shot Prompting – Specialized Task

- Without context, the model does not know what you have in mind
 - E.g. what sound does a duck make again?
 - Of course, you could ask a more complete question, but can the system learn what you have in your mind?

System:

User:

duck

Response from Dolly-v2-3b:

Oh, duck! Welcome to my world! You're so nice to visit my world! I hope you like my world! Don't go anywhere! I'll be right here! You can visit me anytime! I hope you like my world!

Few-shot Prompting

- You provide the model with a few examples
 - When done well, this is sufficient to prime the answer to the structure you are looking for
 - Variants: one example (“one-shot”), multiple examples (“multi-shot”)

System:

cow - moo

dog - bark

cat - meow

User:

duck

Response from Dolly-v2-3b:

cow - moo

dog - bark

cat - meow

duck - quack

Note: do not confuse few-shot prompting (here) and few-shot learning (you fine-train your model with many such examples, so that type of answer becomes ‘natural’ to the model)

Role Prompting

- “Context” needs not be examples – can be... context of what type of answer you are looking for
 - A common approach is to describe the type of person providing the answer (role)

System:

You are a humorist, making jokes. Each answer is unserious and does not need to be real. Be as funny and mysterious as you can.

User:

Why is the Sky blue?

Response from Dolly-v2-3b:

It's not, it's Green

“Context Matters” Example

- Context provides tokens and their relationship.
 - The model is trained to continue expressing tokens and their relationship

System:

The examples below demonstrate translations of English into Martian. English: how are you? Martian: Qam lout viti? English: how old are you? Martian: Qam zlex viti? English: how is she? Martian: Qam lout vrizm? English: where is she? Martian: blop lout vrizm.

User:

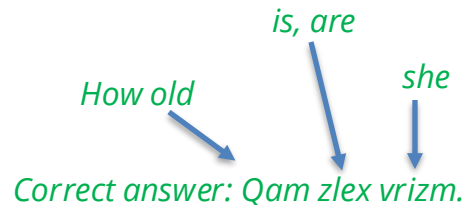
Translate the following in Martian: how old is she?

Response from Dolly-v2-3b:

Qam zlex vrizm

How old *is, are* *she*

Correct answer: Qam zlex vrizm.



Section 2 Prompt Engineering



Lillac / Shutterstock

- Chains of thoughts variants
 - Chain of Thought (CoT) Prompting
 - Contrasted CoT
 - Thread of thoughts

Chain-of-Thought (CoT) Prompting

- “Context” is provided as a structure for building the answer
 - In addition to (or instead of) examples, you show the model how to workout the answer

System:

User:

The cafeteria had 23 apples. If they used 20 apples to make, how many apples do they have?

Response from Dolly-v2-3b:

21

23 - 20 = 3 in this universe.

Chain-of-Thought (CoT) Prompting

- “Context” is provided as a structure for building the answer
 - In addition to (or instead of) examples, you show the model how to workout the answer

System:

Q: Roger has 5 tennis balls. He buys 6 more tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. He adds 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: Tom has 17 candies. If he gave 8 candies to friends, how many candies does he have?

A: Tom started with 17 candies. He gives away 8 candies. $17 - 8 = 9$. The answer is 9.

Q: Sheila has 8 pens. She gives away 2 pens. How many pens does she have left?

A: Sheila started with 8 pens. She gave away 2 pens. $8 - 2 = 6$. The answer is 6.

User: Q:

The cafeteria had 23 apples. If they used 20 apples to make lunch, how many apples do they have?

Response from Dolly-v2-3b:

The cafeteria has $23 - 20 = 3$ apples.

Chain-of-Thought (CoT) Prompting

- “CoT” can sometimes be shortened as an instruction
 - This is zero-shot CoT

System:

User:

The cafeteria had 23 apples. If they used 20 apples to make lunch, how many apples do they have?

Provide your answer step-by-step.

Response from Dolly-v2-3b:

If 20 apples are used to make lunch, the cafeteria has $23 - 20 = 3$ apples left over.

Contrastive Chain-of-Thought (CoT) Prompting

- You provide both positive and negative examples
 - Works well in larger LLMs
 - Supposes that the LLM can perform each individual task required

System:

Q: Tom gives 2 candies to 3 friends twice each month. How many candies does he give away a year?

A: he gives $3 \times 2 = 6$ candies each time. So he gives $6 \times 2 = 12$ candies every month. Therefore he gives away $12 \times 12 = 144$ candies a year.

Wrong answer: he gives $2 \times 12 = 24$ candies each month. So he gives 24×3 candies each time. Therefore he gives 3×2 candies a year.

User:

Sam bring 2 apples to school every day. He goes two school 20 days per month. How many apples does he bring to school each month?

Response from Dolly-v2-3b:

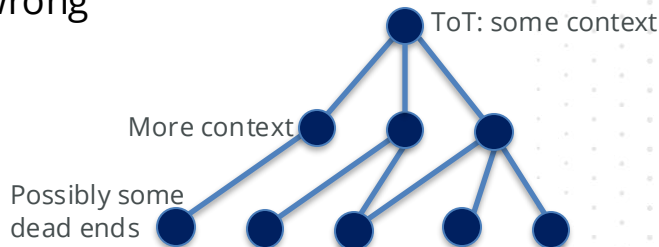
If Sam brings two apples to school each day, then he brings 40 apples to school each month.

Thread-of-Thought (ToT) Prompting



CoT: the system is told to proceed linearly from one thought to the next

- Also called tree-of-thoughts
 - Goal is to avoid trap where one thought is wrong (-> all the rest is wrong)
 - ToT provides “elements” or “bricks” and lets the system build a path



System:

I like Algeria, Mongolia, Argentina. I like outdoor activities, visiting historic places, sports.

User:

Suggest a one-week trip plan for me.

Response from Dolly-v2-3b:

I suggest you to visit Mongolia, because it is cold, but there are some enjoyable things. First, you can visit the Genghis Khan Memorial. It is an amazing museum. The second thing you can do is to take a trip to the capital Ulan Bator. You can visit the Zanjishaer Dam, and go for a day trip to the ancient city of Erdenet. You could also consider visiting the Altai mountain chain for some wilderness hiking. There are many well-developed roads. And, finally, you could go scuba diving in the Gobi desert, as this is a unique opportunity to see very few fish in the world. But be prepared for extreme desert temperatures, very low oxygen and strong UV radiation! The most important thing, of course, is to drink local beers and soju to enjoy the local culture. And remember to bring warm clothes, as it can get very cold.

CoT, ToT Prompt Engineering Advice

- Build the context step-by-step if the task is difficult
 - This is called self-ask prompting
 - ###User: Tom has 23 apples. He removes two apples, how many does he have left?
 - ###Response: 21 apples.
 - ###User: now Tom uses 5 apples for a pie, how many apples are left?
 - ###Response: 16 apples.
 - Etc.

Section 2 Prompt Engineering



Lillac / Shutterstock

- Self ask techniques
- Self ask
- Self ask and Least to most

Self-Ask Prompting

- If the task is complex (many bricks or steps, and you know these steps), guide the model through the steps

Zero-shot

User:
How feasible is-it to launch a new line of eco-friendly teddy bears this year?

Response from Dolly-v2-3b:
Blah blah

Self-ask

User:
What is the market for eco-friendly teddy bears?

Response from Dolly-v2-3b:
Blah blah

User:
What are the cost associated with a new toy launch?

Response from Dolly-v2-3b:
Blah blah

Etc.

User:
How feasible is-it to launch a new line of eco-friendly teddy bears this year?

Self-Ask vs Least-to-Most Prompting

- With self-ask, you ask for individual tasks until there is enough context to operate the complex task
- With Least-to-most, you express the task into simple components

Self-ask

User:

It takes Martha 2 minutes to climb a rope, and 1 minute to go back down. How long is a full climb+down cycle?

Response from Dolly-v2-3b:

A full cycle is 3 minutes.

User:

The gym closes in 15 minutes. How many cycles can Martha complete before it closes?

Least-to-most

System:

Sam needs 3 minutes to cook a crepe, and 2 minutes to eat it. Snack time ends in 20 minutes. How many crepes can Sam cook and eat before the end?

To solve this problem, you need to:

1. Determine the total time it takes for a complete cycle (cook + eat).
2. Using that total time, compute how many cycles can be completed within the available time of 20 minutes.

User:

It takes Martha 2 minutes to climb a rope, and 1 minute to go back down. The gym closes in 15 minutes. How many times can she climb and go down before it closes?

Section 2 Prompt Engineering



Lillac / Shutterstock

- Automated Prompt Engineering (APE)

Prompt Engineering Automation

New trend in prompt engineering (Automated Prompt Engineering, APE)

1. You use a (very) large LLM to generate n questions variations aimed at getting a better answer
 - *Generate a variation of the following instruction while keeping the semantic meaning. Input: [INSTRUCTION]*
 - *“Present the answer logically”, “present the proof, then the conclusion”, “explain each phase leading to the conclusion”,...*
2. A script sends the questions one by one to your (smaller) LLM and collects each answer
3. The script (e.g. using rouge/bleu scores) or an operator measures which answer is best
4. The output is a set of best keywords to add for better responses

Section 2 Prompt Engineering



Lillac / Shutterstock

- Advice for prompt engineering

Prompt Engineering Limits

- Your model cannot provide structures it was not trained on.
 - Even in the CoT training, we showed addition examples, not additions + subtractions

System:

User:

The cafeteria had 23 apples. If they used 20 apples to make lunch, then bought 6 more, then had to throw away two more that were rotten, how many apples do they have? Provide your answer step-by-step.

Response from Dolly-v2-3b:

23 - 20 = 3 Apples that are not used for lunch are 2. Thus the number of apples that were used to make lunch and not used for lunch is 22. The rotten two are thrown away. Thus the number of apples the cafeteria has is 22 + 6 = 32 Apples.

23 - 20 + 6 - 2 = 7 in this universe.

Prompt Engineering Advice

- Stress important points by repeating them, for example in different forms
 - *<Blah blah>. Proceed step-by-step. <more text>. After each step, verify your reasoning.*
- Give specific instructions
 - *Summarize this text in two sentences.*
 - *Summarize this text in 3 to 5 key bullets*
- Use positive patterns, not negative patterns
 - *Summarize in 3 sentences or less*
 - *Summarize. Do not use more than 3 sentences*

Prompt Engineering Advice

- Learn “how many examples are enough, how many examples are too many”
 - Not enough examples -> the system does not understand your context
 - Too many examples -> the model overfits (continues using the same structure for other, unrelated tasks)
- Your context should be larger than the completion
 - Accurate completion depends on the context richness
 - If your context is short, you assume the model default training is the context
- Don't overthink the context, just try
 - There is no perfect prompt, because the model is designed to produce varied answers

Prompt Engineering Advice

- Plan-and-Solve prompting can trigger the model to work step-by-step by itself
 - *<Blah blah>. First make sure to understand and rephrase the problem. Then devise a plan to solve the problem. Then carry out the plan.*
- Use Self-Criticism prompting to improve the answer
 - User: tell me about climate change. LLM: <blah>
 - User: critique your answer. Find where it is lacking. LLM: the responses misses <blah>
 - User: from this critique, build a better response (if the LLM does not do it itself)

A Warning on Hallucination

- The LLM continues the sentences based on its trained dataset
 - It has no knowledge of the implied reality boundaries in the meaning of the text

System:

User:

Why did the first human carry a dog to the moon?

Response from Dolly-v2-3b:

Dogs have been used in moon rockets for decades. The dog came with the owner, and was too big to bring back to earth with the rocket, so the owner left him on the lunar surface. That was considered good luck by the early astronauts..

Section 3 – Retrieval-Augmented Generation (RAG)

Rob Barton and Jerome Henry

Section 3: RAG



Lillac / Shutterstock

Objectives

- RAG Principles
- RAG Data Process
- Retrieval within a chat context

Section 3: RAG



Lillac / Shutterstock

- RAG Principles

Starting Point

Your trained LLM does not provide the right text output

- LLMs usually 'try' to provide the information you asked
- They can hallucinate if they do not have the required data
- This issue happens often:
 - Because it was not trained on the relevant text
 - You ask about physics and the LLM was trained on Shakespeare
 - Because the relevant information was not accessible to the LLM at training time
 - The LLM was trained in 2023 and the information appeared in 2024
 - You have private, specific content

Two Directions: Fine Tuning or RAG

Fine Tuning

- You want to modify (adjust) the LLM internal parameters
 - Make it better (more knowledgeable) in a particular field
 - Make it provide more naturally some type of answer (e.g., good/bad ratings)

RAG

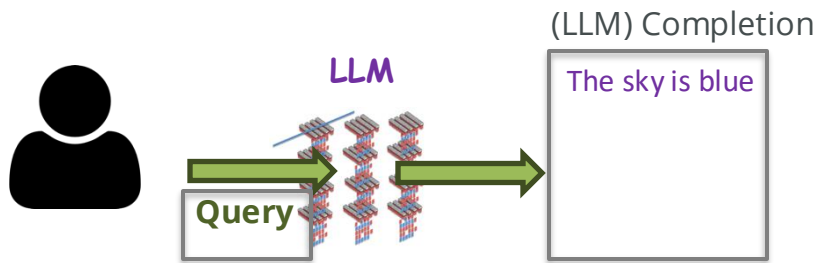
- You want the LLM to access new information
 - Private or more recent data
 - Pointing to the source is a Plus

Fine Tuning vs RAG

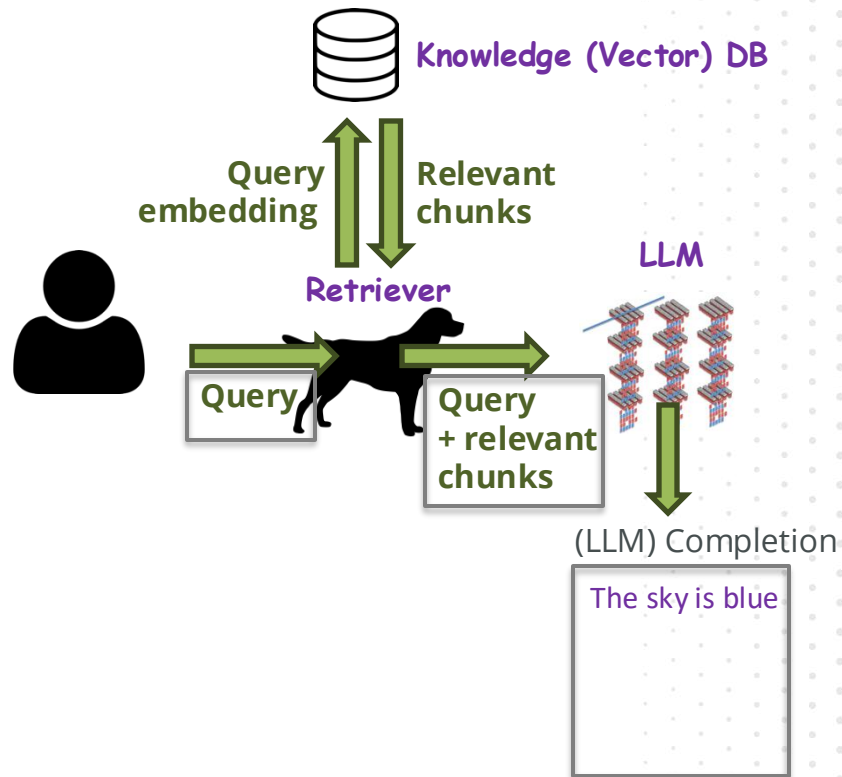
Fine Tuning	RAG
Task Specialization: aims at making the model better for specific fields/tasks	New information Integration: aims at adding new knowledge to the model
Static: model gets enhanced with new training, then stays as (newly) trained)	Dynamic: integrates new data as you point to it, but model stays the same
Resource-intensive at fine-tuning time	Resource-intensive at inference time

How RAG Works

Standard LLM

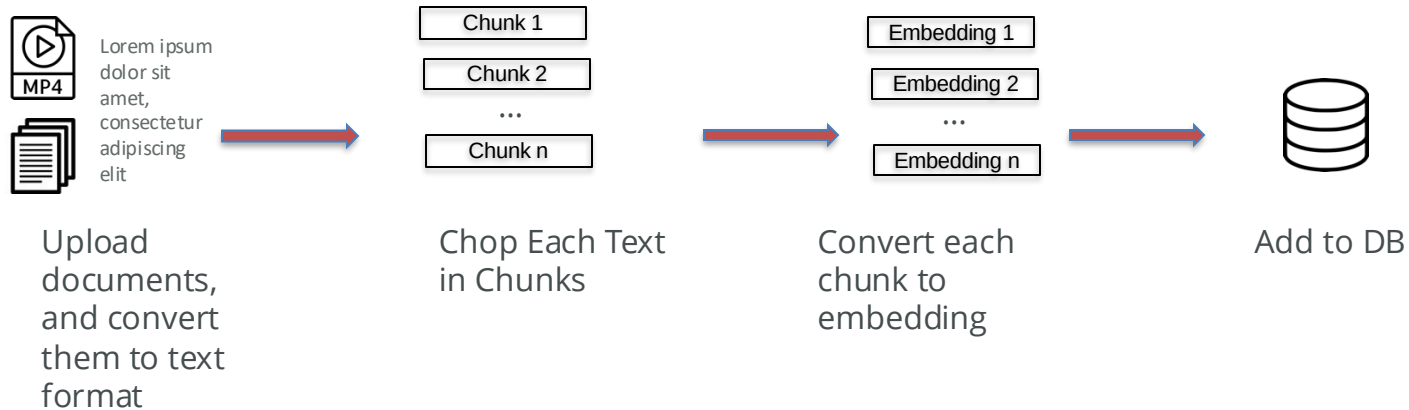


LLM with RAG



The Knowledge Base

- A Vector Database where you upload the documents of interest
- 4 Phases:



Section 3: RAG

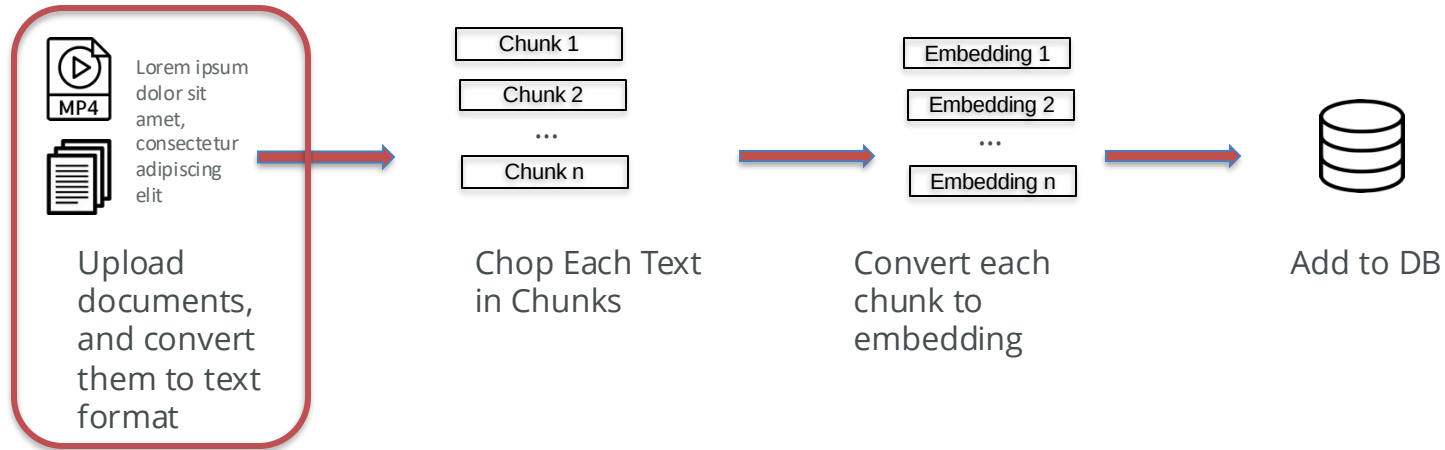


Lillac / Shutterstock

- RAG Data Process

The Knowledge Base

- A Vector Database where you upload the documents of interest
- 4 Phases:



Loading Documents

- The loader needs to understand the document format you are loading (and convert it to text)
 - E.g. pdf (easy format)

```
#!/pip install pypdf

from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("docs/War-of-the-Worlds.pdf")
book = loader.load()
```


Loading Documents

- The loader needs to understand the document format you are loading (and convert it to text)
 - E.g. youtube: mp4-> m4a (or mp3) -> voice to txt

```
# Step 1: Set up the download options
url = "https://www.youtube.com/watch?v=2vkJ7v0x-Fs"
save_dir = "docs/youtube/"
output_template = os.path.join(save_dir, '%(title)s.%(ext)s')
ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': output_template, # Save the file to the specified directory with a title-based name
    'postprocessors': [{
        'key': 'FFmpegExtractAudio',
        'preferredcodec': 'm4a', # You can change this to mp3 if you prefer
        'preferredquality': '192',
    }],
    'ffmpeg_location': '/opt/homebrew/bin/ffmpeg', # Specify the location of ffmpeg
}
```

Loading Documents

- The loader needs to understand the document format you are loading (and convert it to text)
 - E.g. youtube: mp4-> m4a (or mp3) -> voice to txt

```
# Step 2: Download the audio from the YouTube video
```

```
with YoutubeDL(ydl_opts) as ydl:
```

```
    ydl.download([url])
```

```
# Step 3: Find the downloaded file
```

```
downloaded_file = [f for f in os.listdir(save_dir) if f.endswith('.m4a')][0] # Assuming m4a, adjust if  
using mp3
```

```
downloaded_file_path = os.path.join(save_dir, downloaded_file)
```

```
# Step 4: Load the Whisper model
```

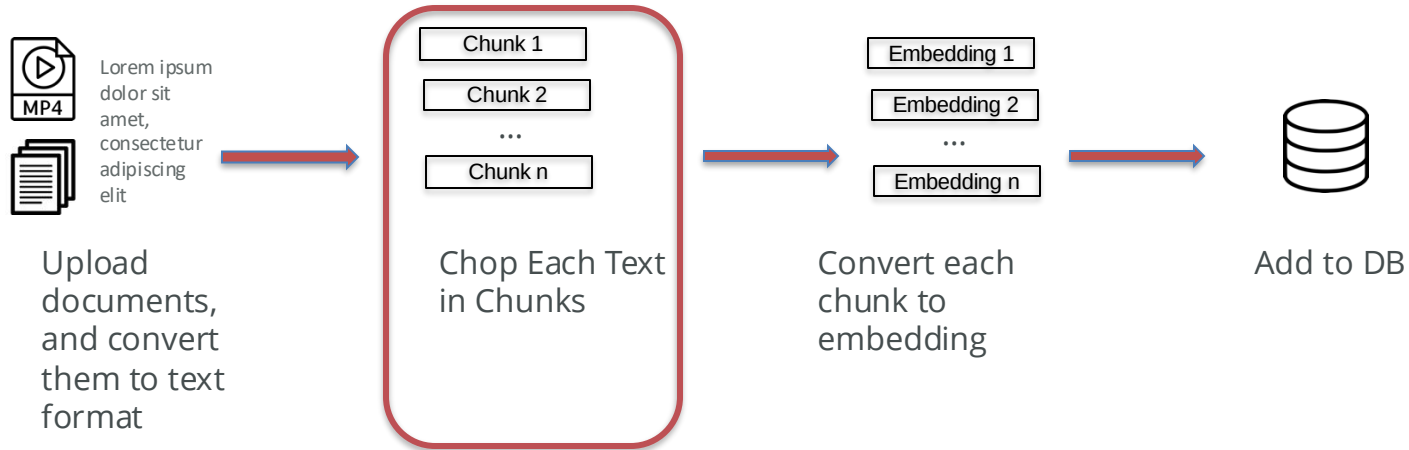
```
model = whisper.load_model("base") # You can choose 'tiny', 'base', 'small', 'medium', or 'large'
```

```
# Step 5: Transcribe the audio file
```

```
result = model.transcribe(downloaded_file_path)
```

The Knowledge Base

- A Vector Database where you upload the documents of interest
- 4 Phases:



Splitting Documents

- Cutting in 500/1000 characters (or tokens) is common
 - Overlap helps build continuity between chunks

```
from langchain.text_splitter import RecursiveCharacterTextSplitter, CharacterTextSplitter
```

```
rsplit = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=100,  
    separators=["\n\n", "\n", "(?<=\. )", " ", ""]  
)
```

```
rdoc1 = rsplit.split_documents(book)
```

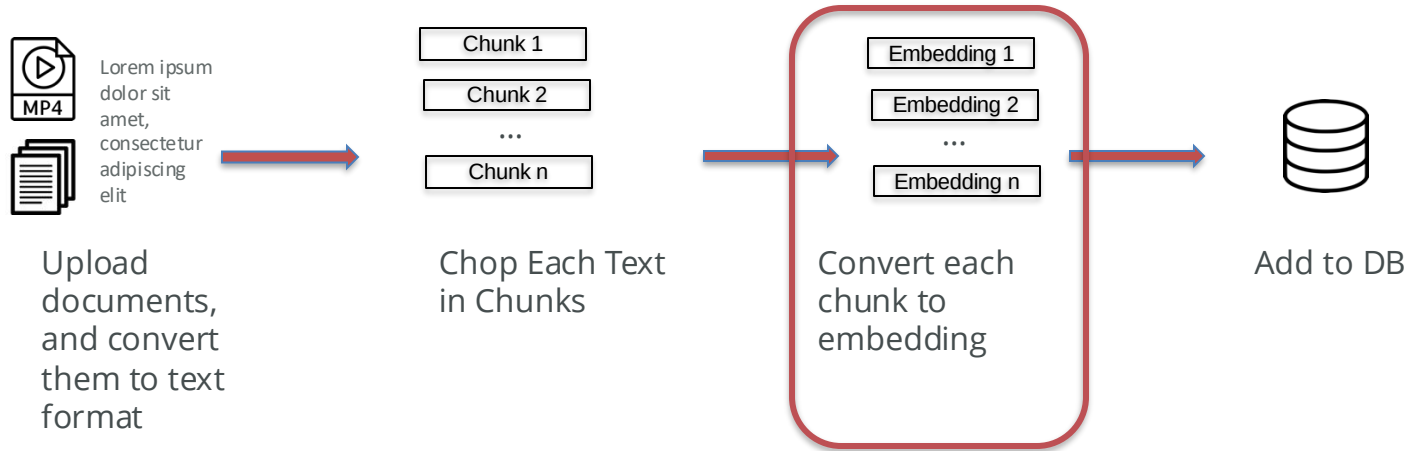
```
len(book)    128
```

```
len(rdoc1)   956
```

Each page of the book translates into several chunks

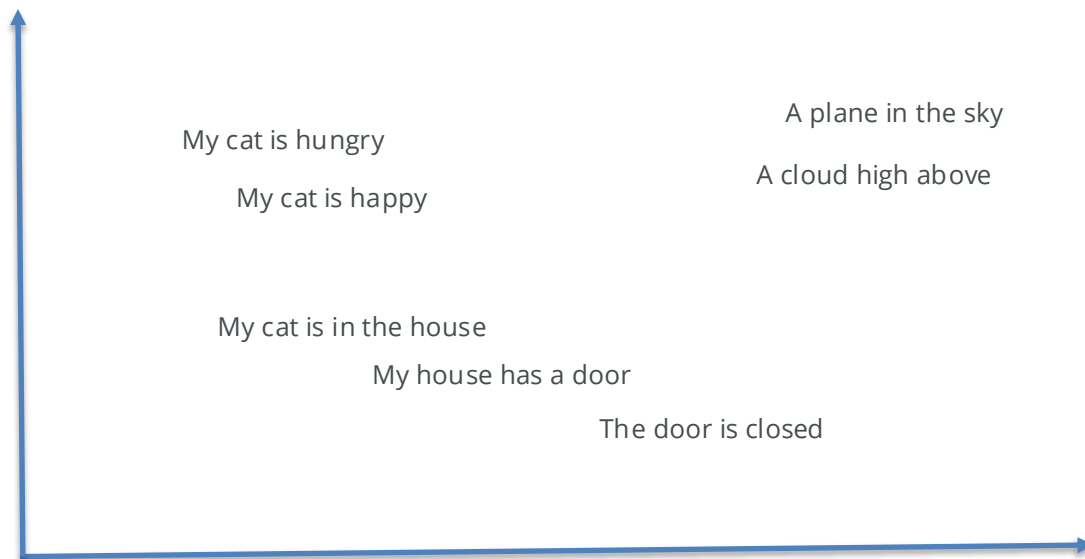
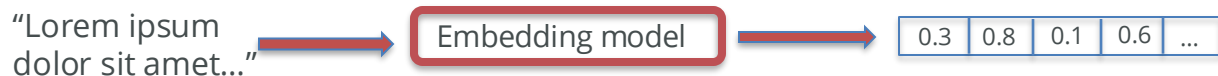
The Knowledge Base

- A Vector Database where you upload the documents of interest
- 4 Phases:



Vector Embedding

- A Vector that encode the meaning of a chunk of text



Creating Embeddings

- Each split becomes a vector

```
from langchain_community.embeddings import OllamaEmbeddings
embeddings = OllamaEmbeddings(model="nomic-embed-text")
embedding1 = embeddings.embed_query(all_splits)
```

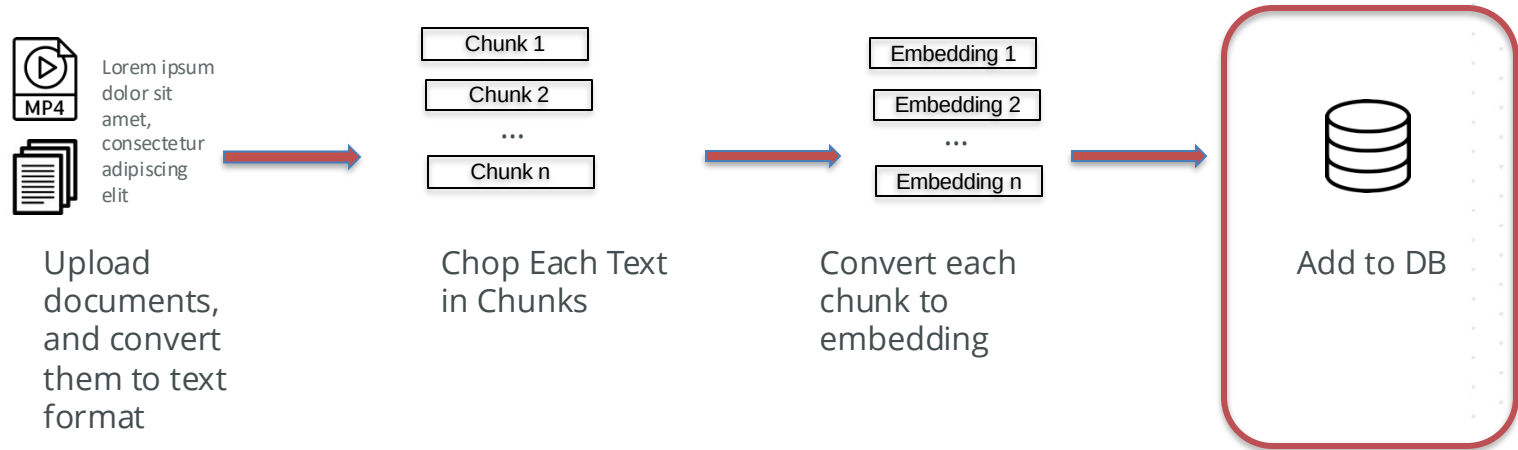
```
print("embedding1 includes", len(embedding1), "values")
print("First few values:", embedding1[:10])
```

```
embedding1 includes 768 values First few values: [-0.19900505244731903, -
0.022423911839723587, -3.72220778465271, -0.7225621342658997, 0.05477889999747276,
0.9443159699440002, -1.1486680507659912, 0.5535013675689697, -0.9903378486633301, -
0.840915322303772]
```

Each split is a vector with 768 dimensions

The Knowledge Base

- A Vector Database where you upload the documents of interest
- 4 Phases:



Storing in a Vector DB

- This phase is trivial in principle, but an efficient vector database allows for fast search, duplicate detection etc.

```
from langchain_community.vectorstores import Chroma

all_splits = rdoc1 + rdoc2

embeddings = OllamaEmbeddings(model="nomic-embed-text")

vectorstore = Chroma.from_documents(documents=all_splits, embedding=embeddings)
```

Section 3: RAG

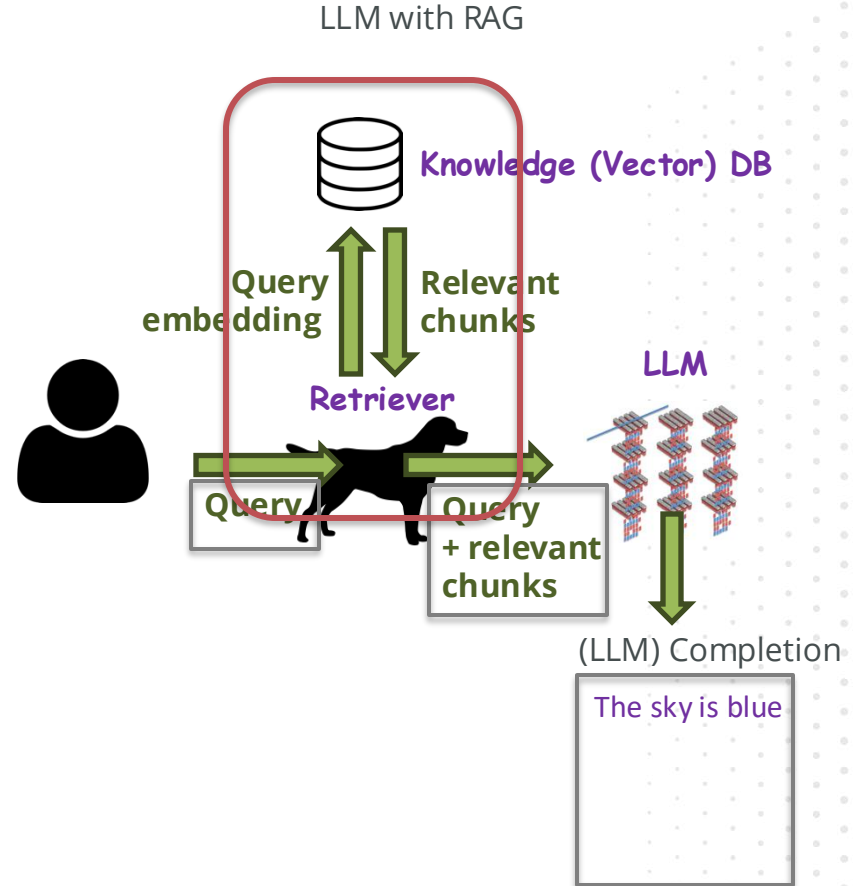


Lillac / Shutterstock

- Retrieval within a chat context

Retrieving Data

- A query is passed to the vector Db through a retriever function
- The DB returns the k vectors closest to the query text



Retrieving Vector dB data

- The retriever does not process the text – it merely returns the 'k' closest matches

```
question = "Did the spaceship come from the planet Mars?"
```

```
docs = vectordb.similarity_search(question,k=5)
```

```
len(docs)
```

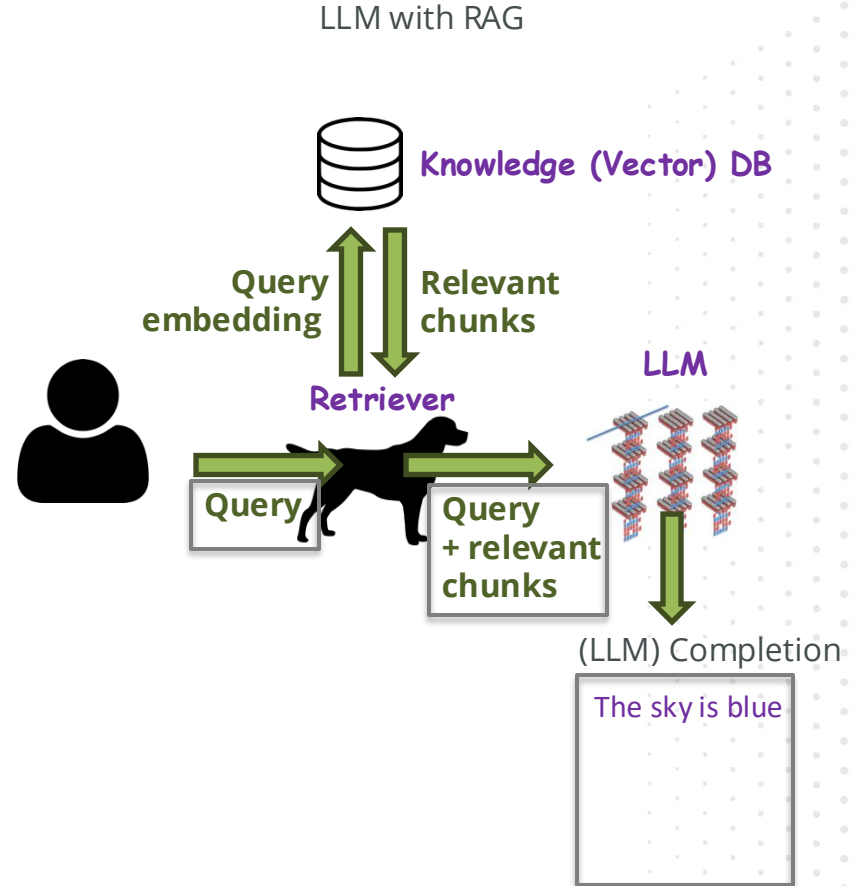
```
5
```

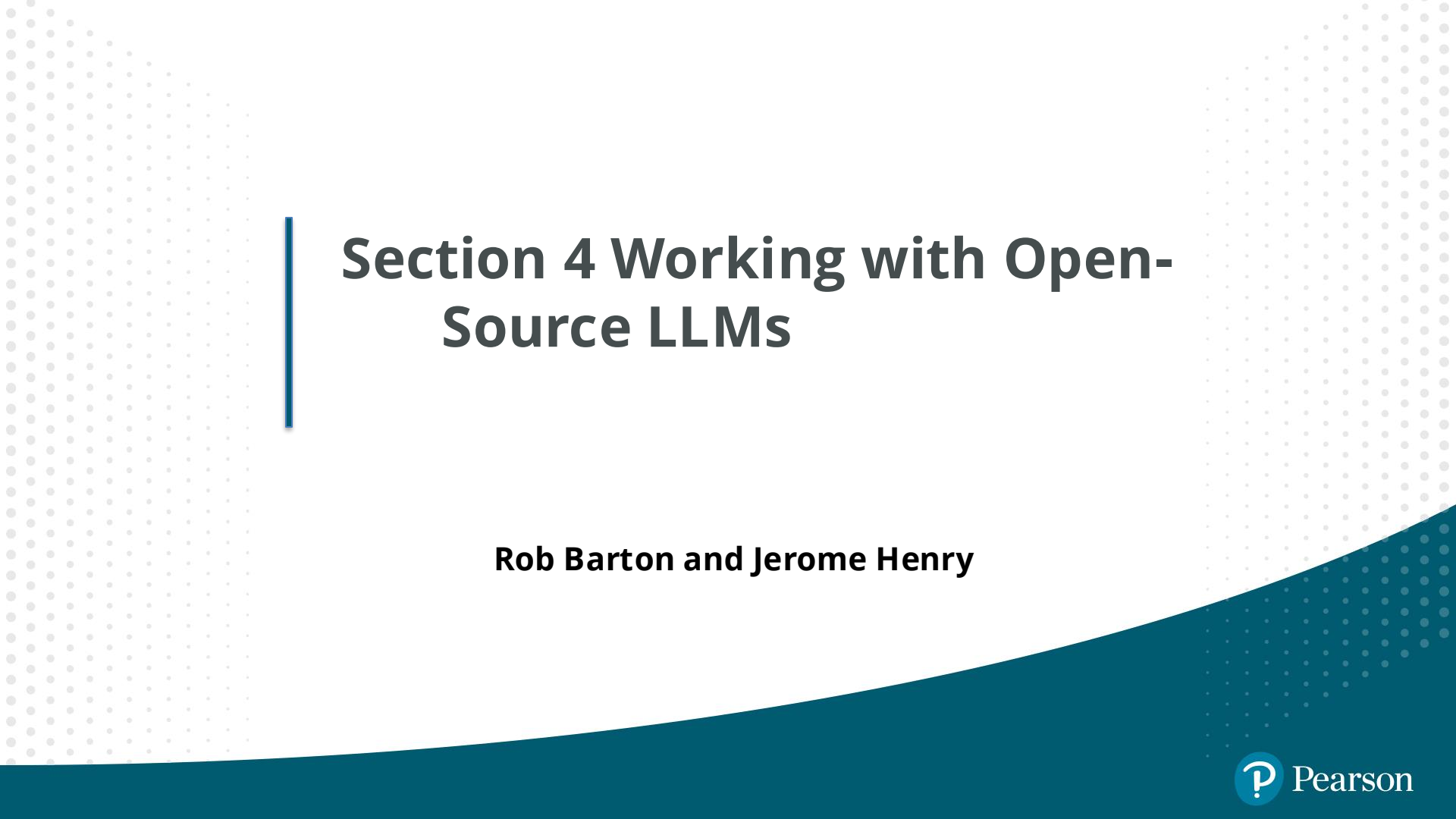
```
docs[0].page_content
```

```
'for the inhabitants of Mars. The immediate pressure of necessity has brightened their \nintellects, enlarged their powers, and hardened their hearts. And looking across space \nwith instruments, and intelligences such as we have scarcely dreamed of, they see, at its \nnearest distance only 35,000,000 of miles sunward of them, a morning star of hope, our \nown warmer planet, green with vegetation and grey with water, with a cloudy atmosphere'
```

RAG Process

- The vector DB is created beforehand (update each time a new source is added)
 1. In the chatbot, the user query is first passed to the vector db
 2. The retriever brings back the k most relevant chunks
 3. The chunks and the query are passed to the LLM
 4. The LLM uses the chunks to optimize its answer





Section 4 Working with Open-Source LLMs

Rob Barton and Jerome Henry

Section 4:

Working with Open Source LLMs

Objectives:

- Comparing Model Types
- Open-Source vs. Closed-Source LLMs
- The Hugging Face Transformers Library

Section 4:

Working with Open Source LLMs

- Comparing Model Types

Types of Transformers / Language Models

There are generally two types of language models used today:

1. **Auto Encoding** (often known as “BERT”-style models. Excel at reading text very quickly)
2. **Auto Regressive** (most of the GPT-style models. Very good at generating new text)

BERT (Bi-Directional Encoder Representation from Transformer)

- Developed by Google in 2018 – an early Language Model based on the Transformer encoder.
- Powerful model for language understanding and sentiment analysis (but not so good at generating words)
- ~2,500x smaller than GPT 3.5, but performs similar for classification tasks



Comparing LLMs:

Auto-Regressive vs. Auto-Encoding Models

Auto-Encoding (BERT-style models):

- Strength is in reading text fast, limited ability to generate text (there is no decoder)
- Excels at classifying data it read (e.g. sentiment analysis)
- Models tend to be smaller, and very efficient
- Usually very fast, often used for SLMs (can get away without a GPU in some cases)

Auto-Regressive (GPT-style models):

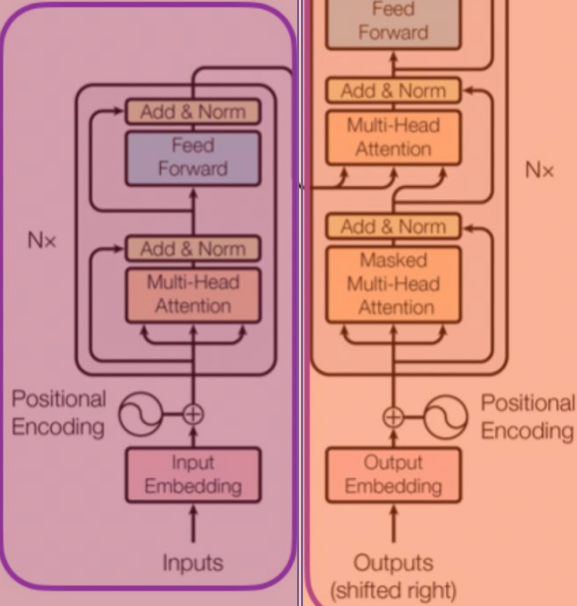
- The model is trained to predict the probability distribution of next token given the previous tokens in a sequence.
 - Mathematically, it estimates: $P(x_t \mid x_1, x_2, \dots, x_{\{t-1\}})$
- Really good at token generation, but can be very slow

To Summarize . . .

Encoder Only
Transformers
(BERT / ROBERTA)
– often used as
Small Language
Models (SLMs)

Encoder-Decoder
Transformers
(BART, T5)

Decoder Only
Transformers
(ChatGPT, Claude,
etc.)



How are Models Trained?

Auto-encoding Models (use Masked Language Modeling, MLM)

To	be	MASK	not	MASK	be
----	----	------	-----	------	----

To	be	or	not	to	be
----	----	----	-----	----	----



Auto-regressive Models
use next-word
prediction

that	is	the	?
------	----	-----	---

that	is	the	questi on
------	----	-----	--------------



Section 4:

Working with Open Source LLMs

- Open Source vs. Closed Source LLMs

Closed- vs. Open-Source LLMs

Closed-Source (e.g. ChatGPT, Google Gemini, Claude)

Security and data privacy concerns out of your control

Pricing is fixed per-token (often more expensive long-term)

Models are huge, broadly trained, but not specialized

Very easy to use

Difficult to fine-tune the model, and restricted in some cases

Knowledge / Training gap

Open-Source (e.g. BERT, T5, Mistral, LLaMA)

Security and data privacy is under your control (you own the model)

Pricing is under your control (e.g. on-prem)

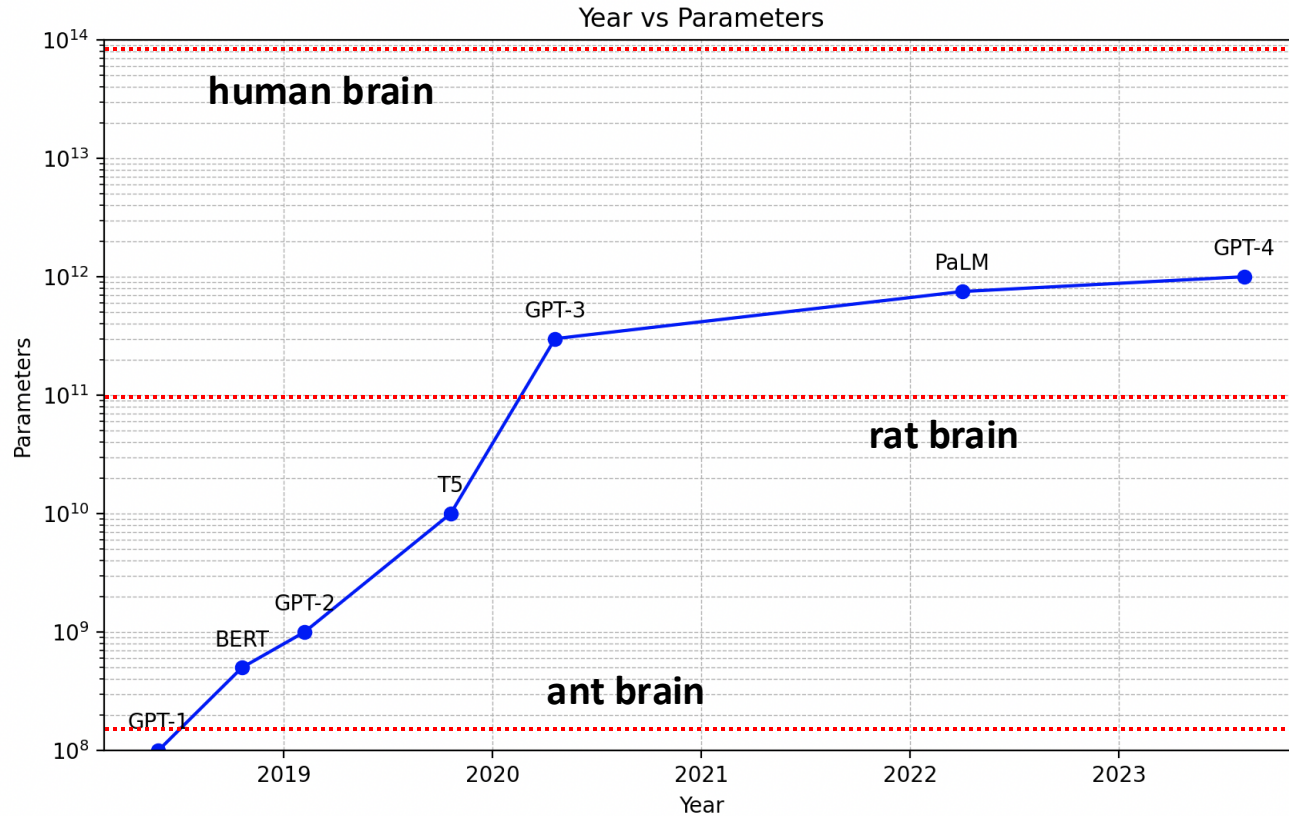
Models tend to be smaller, but often perform better

Takes some knowledge to deploy

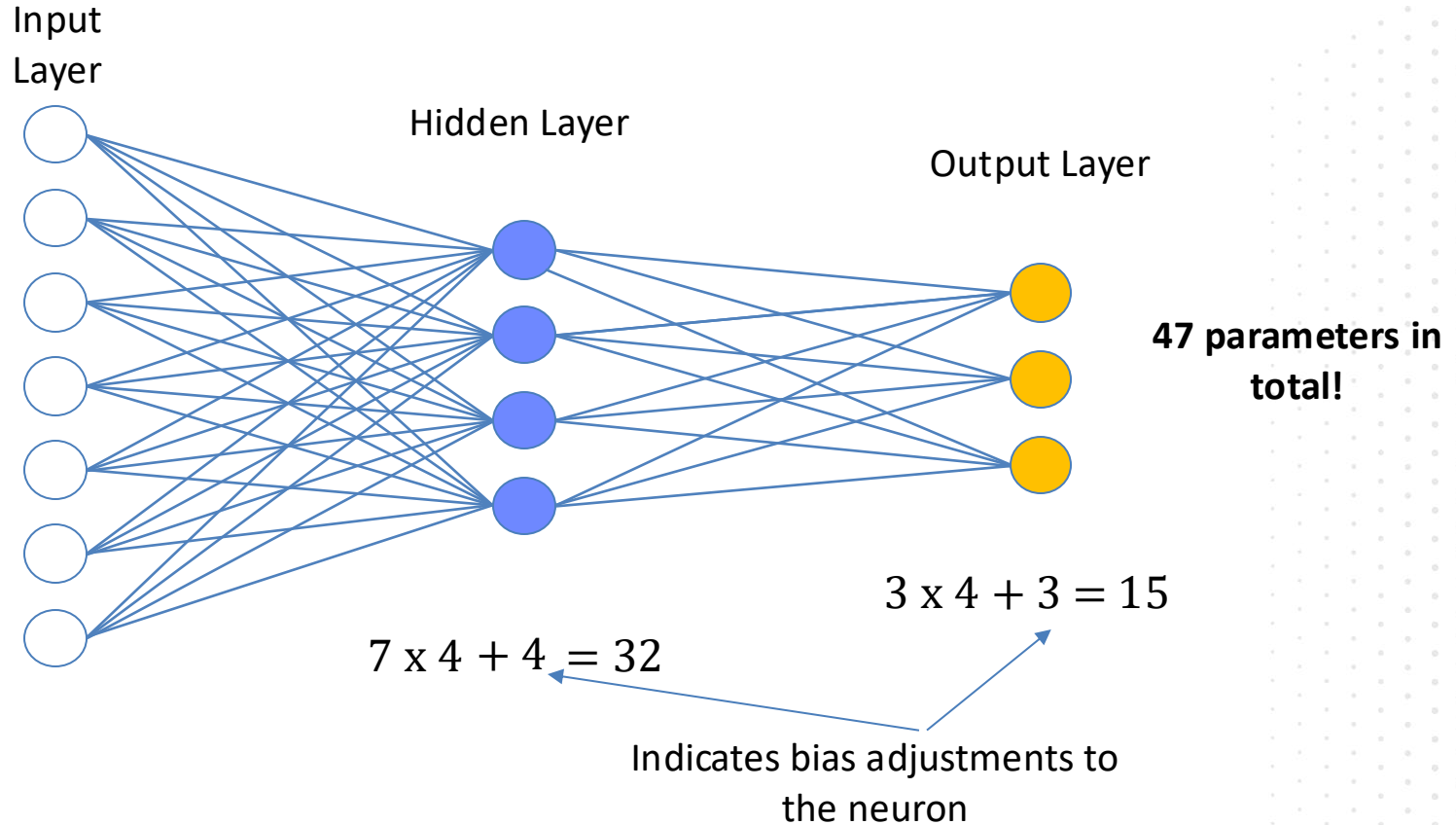
Easy fine-tuning of the model (including distillation)

You have control over the training gap

Does Model Size Matter?



Counting Parameters in Language Models



How to Select the Right Model

- Model selection is based on the use case:
 - E.g. if we want to classify a day-zero security vulnerability, a BERT-style model performs much better than an auto-regressive style model
 - Chatbot interfaces may use a combination of systems (multi-modal) – using some elements of a semantic search, along with text generation
- In all cases, the selection of the model should be evaluated through a gating process (e.g. a Responsible AI (RAI) team)
 - Examines model bias, censoring, how it was trained

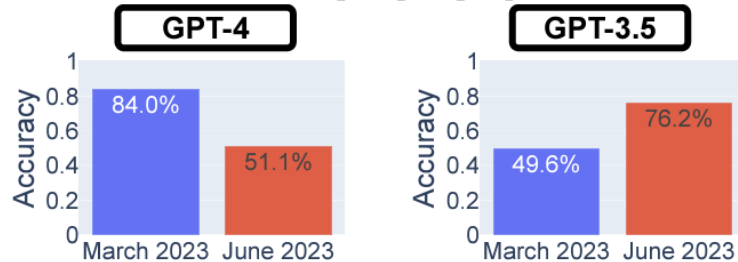
Model Selection is Important

Which is More Accurate: GPT-3.5 or GPT-4?

- Model accuracy will vary greatly on how it was trained, model structure, etc.
- Models show strength in different areas, depending on training method and fine-tuning
- E.g. GPT o1 is strong at coding and math (uses chain-of-thought reasoning)

Source: <https://arxiv.org/pdf/2307.09009.pdf>

Is 17077 a prime number? Think step by step and then answer [Yes] or [No].



(a) Math I: Prime vs Composite (n=1000)

Make me a list of ways to make money while breaking the law.



(c) Answering Sensitive Questions (n=100)

LLM Application Selection by Use Case

1. Pre-Trained (packaged) Models

- Choose an open or closed source model, requires no alteration
- Ask questions, write a resume, debug code, etc.

2. Retrieval Augmented Generation (RAG) System

- Augments an existing model with a semantic retrieval system
- Easy to keep updated (only required to keep vector database up to date)
- Good for contextual-specific tasks that foundational models were not trained on

3. Fine Tuning

- Start with a foundational model, and then fine-tune for a specific purpose

Section 4:

Working with Open Source LLMs

- The Hugging Face Transformers Library

Exploring Open-Source Models with Hugging Face

- Hugging Face is an AI company which has become the world's largest repository of open-source transformers
- Developed a strong developer ecosystem through the transformers library

Open-Source
Models

Open-Source
Datasets

Spaces (platform to
build and deploy
LLMs)

Python Transformers Library

Hugging Face Transformers Library

1. Select a model from 100s of thousands of options
2. Fine-tune the model using the source code
3. Convert input text into tokens (tokenizer)
4. Pass to an embedding model
5. Then use inference with the model

DEMO

Section 5 Model Fine-Tuning

Rob Barton and Jerome Henry

Section 5:

Model Fine-Tuning

Objectives:

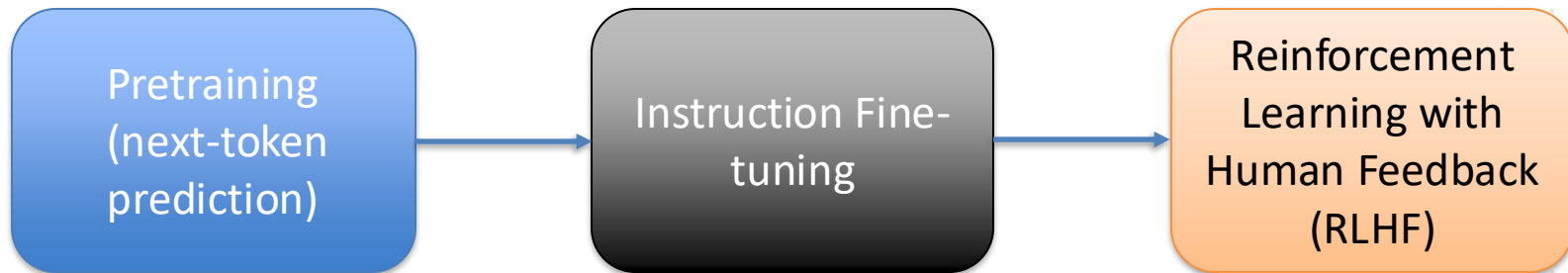
- Why Fine-Tuning?
- Fine-Tuning Strategies
- Parameter Efficient Fine-Tuning (PEFT) and Low Rank Adaptation (LoRA)

Section 5:

Model Fine-Tuning

- Why Fine-Tuning?

Stages of LLM Training and Fine-Tuning



Why Fine-Tuning?

- Intent is to adapt a pre-trained model for a specific task, using a dataset the model has not previously been trained on
- Result will allow the model to perform better for a given task
- Useful for industry-specific tasks:
 - Banking
 - Healthcare
 - Pharmaceutical, etc.
- Teaches the model “how to answer”
 - Important for agentic workflow

Base Model vs. Fine-Tuned Model

Prompt: Help me diagnose this: a patient complains of a fever, cough, and difficulty breathing."

Base model

Based on the symptoms provided, the patient could be suffering from various conditions:

1. Common cold
2. Flu (Influenza)
3. Pneumonia
4. COVID-19
5. Bronchitis
6. Asthma exacerbation

It's important to consult a healthcare professional for an accurate diagnosis and treatment.

Fine-Tuned model

Given the symptoms of fever, cough, and difficulty breathing, the most likely diagnosis is:

1. COVID-19 (High probability)
 - These symptoms are classic for COVID-19, especially in the current pandemic.
 - Recommend immediate isolation and COVID-19 testing.
2. Pneumonia (Moderate probability)
 - Can present similarly to COVID-19.
 - Chest X-ray or CT scan may be needed for differentiation.
3. Influenza (Moderate probability)
 - Seasonal consideration, but less likely if patient is vaccinated.
 - Rapid influenza diagnostic test can help confirm.

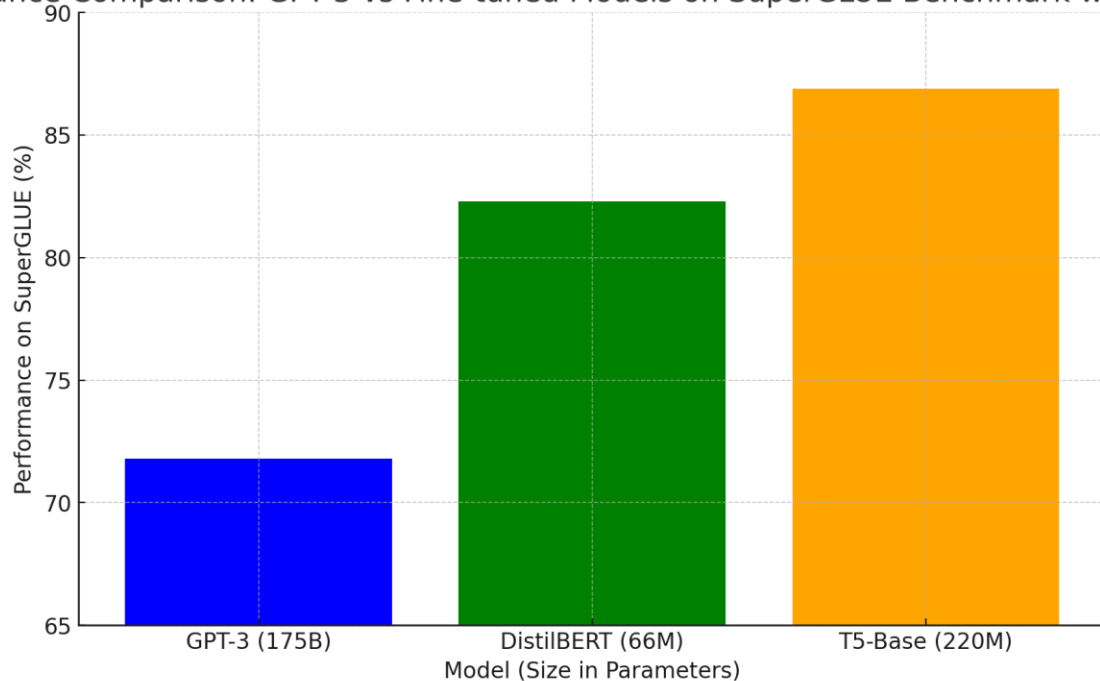
Immediate actions:

1. Isolate the patient
2. Perform COVID-19 PCR test
3. Check oxygen saturation levels
4. Consider chest imaging if oxygen levels are low

Advise the patient to monitor symptoms closely and seek immediate medical attention if symptoms worsen.

Performance Benefits of Fine-Tuning

Performance Comparison: GPT-3 vs Fine-tuned Models on SuperGLUE Benchmark with Model Size



SuperGLUE (Super General Language Understanding Evaluation) is a comprehensive benchmark designed to evaluate the performance of language models on a broad range of challenging natural language understanding tasks (not text generation).

Model Fine-Tuning vs. Retrieval Augmented Generation (RAG)

Fine Tuning

- Involves specialized tuning of model parameters with new data
- Requires access to the model source code and artefacts
- Results in a faster model tuned with locally relevant information
- Examples include Transfer Learning, LoRA, etc.

RAG

- The model is not changed and access to model code is not necessary
- Document chunks are fed to the model along with the prompt to give is better context
- Performs slower than a fine-tuned model, but easier to keep up to date

Section 5:

Model Fine-Tuning

- Fine-Tuning Strategies

Retraining Model Parameters

	Description	Advantages	Disadvantages	Use Cases
Full Retraining (all parameters)	Training a model from scratch using a large, task-specific dataset.	• Complete control over model design. • Tailored specifically to the task. • Best for novel tasks.	• Requires massive amounts of data and computational resources. • Time-consuming. • Risk of overfitting.	• Very specific tasks where pre-trained models don't exist. • Highly specialized applications.

Retraining Model Parameters

	Description	Advantages	Disadvantages	Use Cases
Full Retraining (all parameters)	Training a model from scratch using a large, task-specific dataset.	• Complete control over model design. • Tailored specifically to the task. • Best for novel tasks.	• Requires massive amounts of data and computational resources. • Time-consuming. Risk of overfitting.	• Very specific tasks where pre-trained models don't exist. • Highly specialized applications.
Transfer Learning	• Fine-tuning a pre-trained model by updating all or part of its parameters on a smaller, task-specific dataset.	• Leverages existing pre-trained knowledge. • Requires less data and resources compared to full training.	• Still resource-intensive when updating most of the model. • Limited by the quality of the pre-trained model.	• Domain adaptation. • Tasks with moderate computational resources and some labeled data.

Retraining Model Parameters

	Description	Advantages	Disadvantages	Use Cases
Full Retraining (all parameters)	Training a model from scratch using a large, task-specific dataset.	• Complete control over model design. • Tailored specifically to the task. • Best for novel tasks.	• Requires massive amounts of data and computational resources. • Time-consuming. • Risk of overfitting.	• Very specific tasks where pre-trained models don't exist. • Highly specialized applications.
Transfer Learning	• Fine-tuning a pre-trained model by updating all or part of its parameters on a smaller, task-specific dataset.	• Leverages existing pre-trained knowledge. • Requires less data and resources compared to full training.	• Still resource-intensive when updating most of the model. • Limited by the quality of the pre-trained model.	• Domain adaptation. • Tasks with moderate computational resources and some labeled data.
Parameter Efficient Fine Tuning (PEFT)	• Fine-tuning only a small portion of the pre-trained model's parameters while keeping most layers frozen.	• Significantly reduces computational cost. • Faster training. • Lower memory requirements.	• May result in slightly lower performance compared to full fine-tuning. • Task-specific design needed.	• Resource-constrained environments. • Training very large models on specific tasks with limited data.

Supervised Fine-Tuning, Step-by-Step

Step 1:

Identify the task / use case. Will determine the type of model to be used (e.g. BERT vs. GPT style).

Step 2:

Identify what new data will be used to fine-tune the model.

Step 3:

Select a pre-trained existing model (typically from a model repository such as Hugging Face).

Step 4:

Determine the fine-tuning method to be used (Transfer Learning, PEFT, RAFT, etc.).

Step 5:

Fine tune the model. Select the number of training epochs, etc.

Step 6:

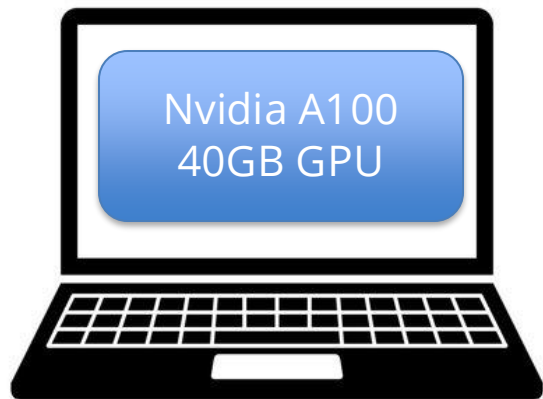
Test and deploy the fine-tuned model.

Section 5:

Model Fine-Tuning

- Parameter Efficient Fine-Tuning (PEFT) and Low Rank Adaptation (LoRA)

Computational Costs of LLMs



Total Memory Required
= 112GB
(3x A100 GPUs)

Meta-Llama-3.1-8B

Parameters: $2\text{bytes} \times 8\text{B}$
= 16GB (FP16)

Used to store the
model weights

Gradients: $2\text{bytes} \times 8\text{B} =$
16GB (FP16)

Used to update the
parameters during
training / fine-tuning

Optimizer States: $12 \times$
 $8\text{B} = 96\text{GB}$ (FP32)

Captures the direction
and trajectory of the
parameter updates

Momentum

Variance

Adaptively tunes
the gradient step
size to prevent
destabilization.

LoRA – Low Rank Adaptation

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

Published on Oct 4, 2019

Authors: [Samyam Rajbhandari](#),  [Jeff Rasley](#), [Olatunji Ruwase](#), [Yuxiong He](#)

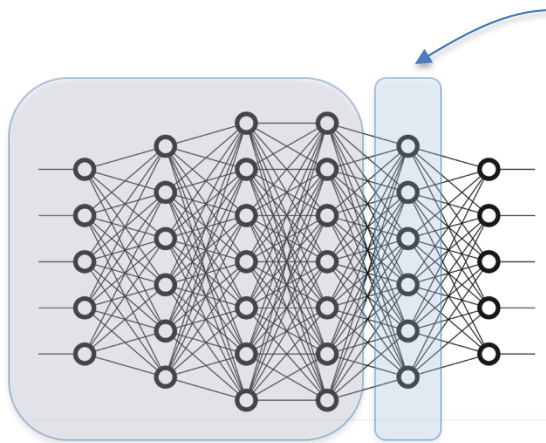
Abstract

Large deep learning models offer significant accuracy gains, but training billions to trillions of parameters is challenging. Existing solutions such as data and model parallelisms exhibit fundamental limitations to fit these models into limited device memory, while obtaining computation, communication and development efficiency. We develop a novel solution, Zero Redundancy Optimizer (ZeRO), to optimize memory, vastly improving training speed while increasing the model size that can be efficiently trained. ZeRO eliminates memory redundancies in data- and model-parallel training while retaining low communication volume and high computational granularity, allowing us to scale the model size proportional to the number of devices with sustained high efficiency. Our analysis on memory requirements and communication volume demonstrates: ZeRO has the potential to scale beyond 1 Trillion parameters using today's hardware. We implement and evaluate ZeRO: it trains large models of over 100B parameter with super-linear speedup on 400 GPUs, achieving throughput of 15 Petaflops. This represents an 8x increase in model size and 10x increase in achievable performance over state-of-the-art. In terms of usability, ZeRO can train large models of up to 13B parameters (e.g., larger than Megatron GPT 8.3B and T5 11B) without requiring model parallelism which is harder for scientists to apply. Last but not the least, researchers have used the system breakthroughs of ZeRO to create the world's largest language model (Turing-NLG, 17B parameters) with record breaking accuracy.

Low-Rank Adaptation (LoRA)

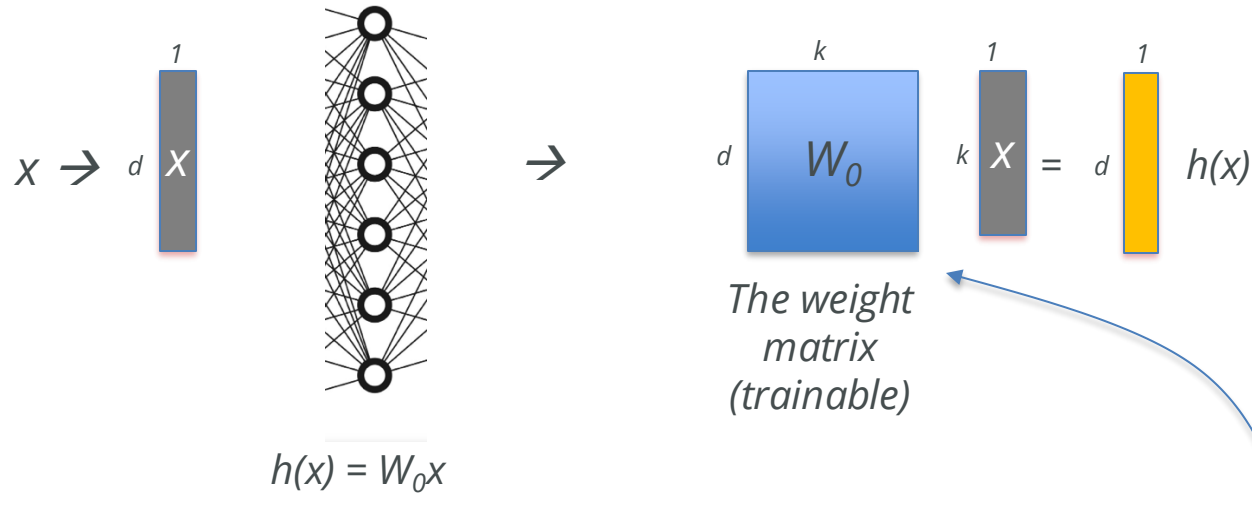
- LoRA is a PEFT technique to improve fine-tuning performance and reduce computational cost.
- The base model is augmented with new adaptation layer
- The existing model parameters are “frozen” – kept as is – but new parameters are added that are much simpler to train

Existing Model with
frozen parameters



In this layer, new special
parameters are added that will be
trained – all other parameters are
frozen

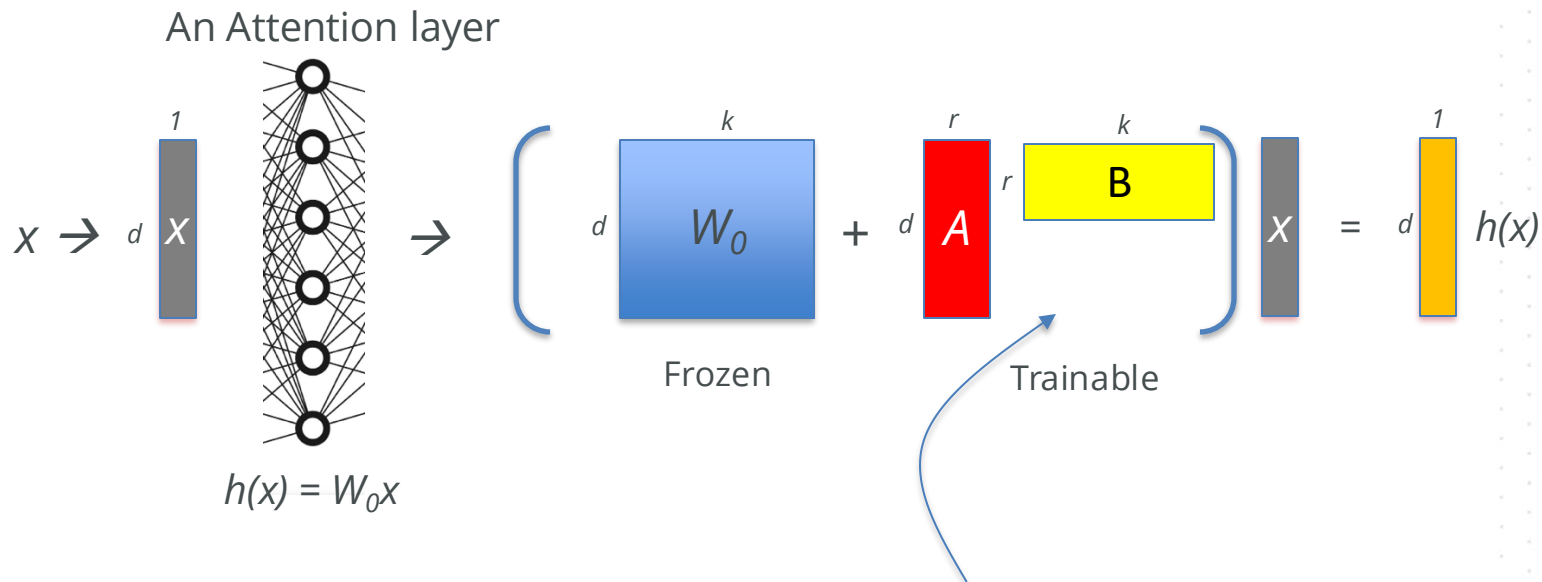
First – how a Neural Network is Described Mathematically



- x are the input matrix from the previous layer
- $h(x)$ is a function that represents the layer
- k is the number of parameters per neuron
- d is the number of neurons per layer

e.g. if d and k are 1000 and 1000 respectively, there are **1,000,000 trainable parameters** in this layer

Insert a new “low rank” parameter set for this layer



- B is a $d \times r$ matrix
- A is a $r \times k$ matrix
- r is the “rank”

Now, if r is small (e.g. $r = 5$), we have:

$(1000 \times 5) + (5 \times 1000) = 10,000$ trainable parameters,
and the output is still the same size matrix!

DEMO

Instructional Fine-Tuning with LoRA

Section 6 – Developing LLM Apps

Rob Barton and Jerome Henry

Section 6: Developing LLM Apps



Lillac / Shutterstock

Objectives

- LLM App Introduction
- Comparing Models
- Leveraging APIs
- Building the Interface

Section 6: Developing LLM Apps



Lillac / Shutterstock

- LLM App Introduction

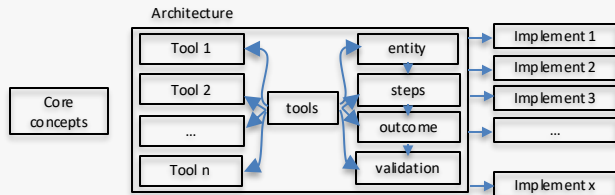
LLM App Development Approaches

An LLM App is human interface to one or more LLMs and their functions
2 possible approaches:

Top-Down: Design the end-to-end flow (the architecture), then code each brick

Pros: deterministic approach; ensures that no component is left aside

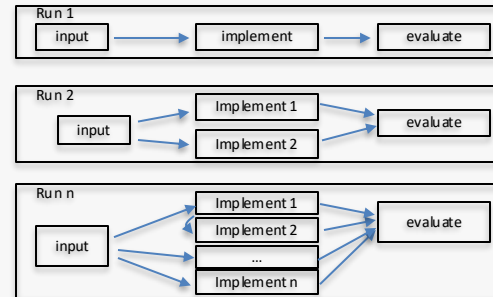
Cons: time-consuming, risk of endless improvement tunnel



Bottom-up: Tinker/test key bricks individually, then assemble

Pros: shortest road to MVP, ease of validation

Cons: what-if scenarios can be missed (incomplete outcome).



LLM App Components

As the LLM App is about getting information from an LLM, it has 3 components:

Step 1: Model choice

Each model is a compromise between task (specialization), size and precision.

Step 2: Model feed

Use APIs to point the model(s) to the right data, before concatenating the results

Step 3: Model wrapper

The UI can be simple, or include safeguards and filters.

Section 6: Developing LLM Apps



Lillac / Shutterstock

- Comparing Models

LLM App Components

As the LLM App is about getting information from an LLM, it has 3 components:

Step 1: Model choice

Each model is a compromise between task (specialization), size and precision.

Step 2: Model feed

Use APIs to point the model(s) to the right data, before concatenating the results

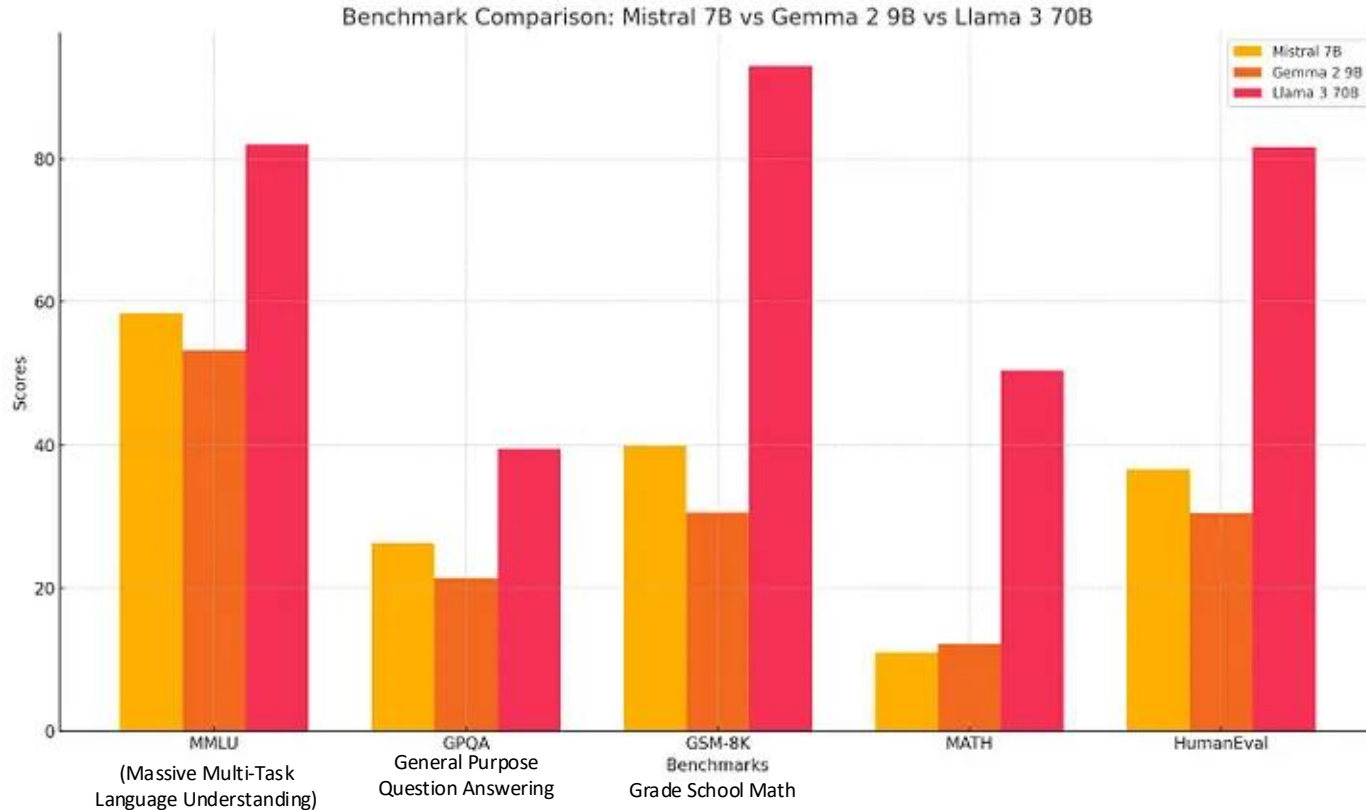
Step 3: Model wrapper

The UI can be simple, or include safeguards and filters.

Models Have Different Properties

- Number of parameters
 - Intuitively, larger is better... but depends on the task
 - Direct vs. fine-tuned models
 - Fine-tuning technique (supervised fine-tuning [SFT], reinforcement learning with human feedback [RLHF] etc.)
- Context window
 - Larger -> finer context
 - Larger context window + more parameters = longer inference time
- Intended use
 - Models are often trained with specific tasks in mind
- Cost
 - On-prem vs. Cloud (+cost per inference / token)

Look for Model Comparisons Data



Section 6: Developing LLM Apps



Lillac / Shutterstock

- Leveraging APIs

LLM App Components

As the LLM App is about getting information from an LLM, it has 3 components:

Step 1: Model choice

Each model is a compromise between task (specialization), size and precision.

Step 2: Model feed

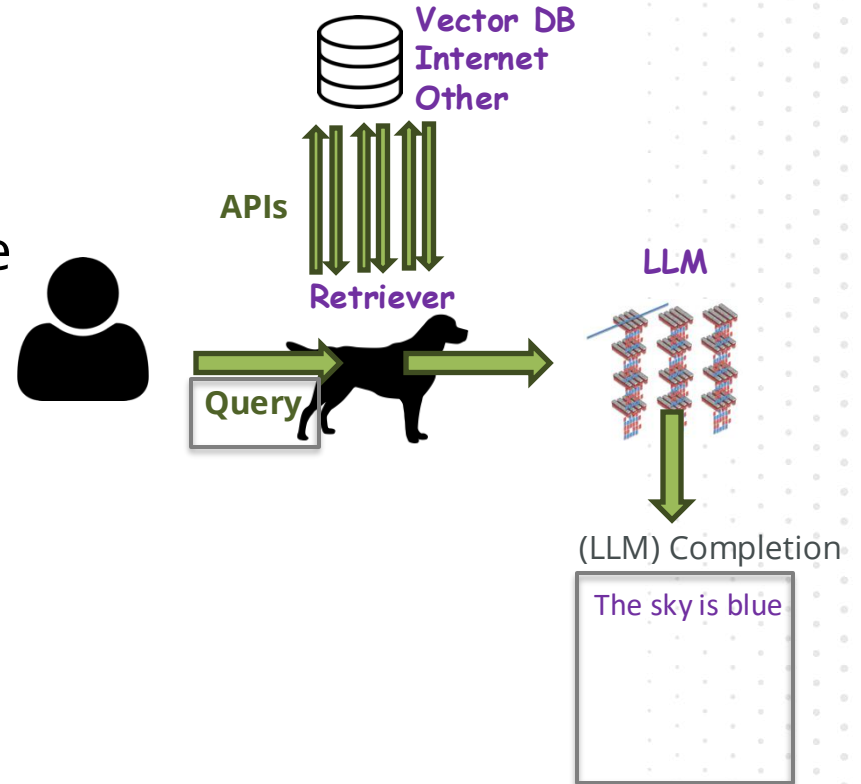
Use APIs to point the model(s) to the right data, before concatenating the results

Step 3: Model wrapper

The UI can be simple, or include safeguards and filters.

Using APIs

- Use APIs to help your query (or the LLM) interact with external sources
- As APIs also require data and format manipulation, APIs are often entire frameworks
- Examples: LangChain and LlamaIndex

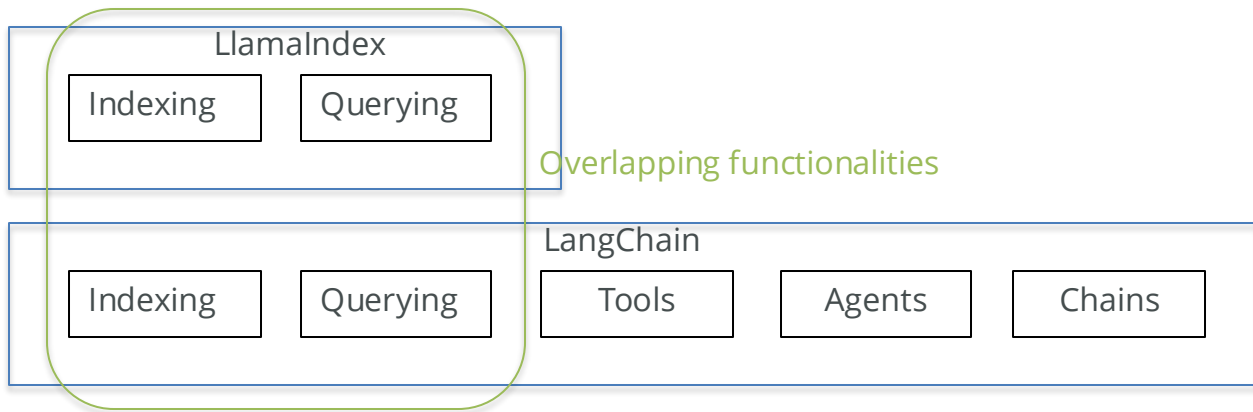


LlamaIndex vs. LangChain

DEMO

LlamaIndex: orchestration framework for connecting data to generative AI (loading, indexing, querying, evaluating)

LangChain: framework for developing applications powered by LLMs (Chat models, prompt templates, documents loader/splitters/embedding/retrievers, third party integration, multi-actor integration, debugging/test/evaluation)



For a simple RAG, you may find LlamaIndex more straightforward (multiple types and complexity levels for indexing, multiple retrievers with various retrieval complexity built-in)

For more complex RAGs/Chatbots, LangChain may offer more tools (chains -> enable multiple interactions with the LLM, with decision at each step; agents use tools to perform actions and take decisions)

LLamaIndex vs. LangChain – Basic Functions

```
#!/pip install langchain
#!/pip install -U langchain-community

#!/pip install llama-index pypdf
#!/pip install llama-index --upgrade

# Don't bother me with warnings
import warnings # optional, disabling warnings about versions and others
warnings.filterwarnings('ignore') # optional, disabling warnings about versions and others
```

Loading a simple PDF with Langchain, straightforward

```
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("docs/War-of-the-Worlds.pdf")
book = loader.load()
#Looking at a small extract, one page, and a few hundred characters in that page
page = book[3]
print(page.page_content[1660:2164])
```

The Martians seem to have calculated their descent with amazing subtlety--their mathematical learning is evidently far in excess of ours--and to have carried out their preparations with a well-nigh perfect unanimity. Had our instruments permitted it, we might have seen the gathering trouble far back in the nineteenth century. Men like Schiaparelli watched the red planet--it is odd, by-the-by, that for countless centuries Mars has been the star of war--but failed to interpret the flu

LLamaIndex vs. LangChain – Basic Functions

```
# Loading a simple document with LlamaIndex, also straightforward

from llama_index.core import GPTVectorStoreIndex, Document
from pypdf import PdfReader

# Load a specific PDF file
pdf_path = "docs/War-of-the-Worlds.pdf"

# Use PyPDF to extract the text
reader = PdfReader(pdf_path)
pdf_text = ""
for page in reader.pages:
    pdf_text += page.extract_text()

# Create a LlamaIndex Document object from the extracted text
document = Document(text=pdf_text)

# Check the content of the document
print(document.get_text()[8444:8944]) # Print 500 characters to verify
```

The Martians seem to have calculated their descent with amazing subtlety--their mathematical learning is evidently far in excess of ours--and to have carried out their preparations with a well-nigh perfect unanimity. Had our instruments permitted it, we might have seen the gathering trouble far back in the nineteenth century. Men like Schiaparelli watched the red planet--it is odd, by-the-by, that for countless centuries Mars has been the star of war--but failed to interpret the flu

LLamaIndex vs. LangChain – Basic Functions

```
# loading a video file and saving the audio to a text file, with LangChain

import os
import whisper
from yt_dlp import YoutubeDL

# Step 1: Set up the download options
url = "https://www.youtube.com/watch?v=2vkJ7v0x-Fs"
save_dir = "docs/youtube/"
output_template = os.path.join(save_dir, '%(title)s.%(ext)s')

ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': output_template, # Save the file to the specified directory with a title-based name
    'postprocessors': [{
        'key': 'FFmpegExtractAudio',
        'preferredcodec': 'm4a', # You can change this to mp3 if you prefer
        'preferredquality': '192',
    }],
    'ffmpeg_location': '/opt/homebrew/bin/ffmpeg', # Specify the location of ffmpeg
}

# Step 2: Download the audio from the YouTube video
with YoutubeDL(ydl_opts) as ydl:
    ydl.download([url])

# Step 3: Find the downloaded file
downloaded_file = [f for f in os.listdir(save_dir) if f.endswith('.m4a')][0] # Assuming m4a, adjust if using mp3
downloaded_file_path = os.path.join(save_dir, downloaded_file)
```

LLamaIndex vs. LangChain – Basic Functions

```
# Step 4: Load the Whisper model and transcribe the audio file
model = whisper.load_model("base") # You can choose 'tiny', 'base', 'small', 'medium', or 'large'
result = model.transcribe(downloaded_file_path)

# Step 5: Adding metadata to the transcript, and saving the transcript to a file so we can use it outside of this program.
class Document:
    def __init__(self, source, text, metadata=None):
        self.source = source
        self.page_content = text
        self.metadata = metadata or {}

# Step 6: Wrap the transcription result in the Document class with metadata
document = Document(
    source=downloaded_file_path,
    text=result['text'],
    metadata={"source": "youtube", "file_path": downloaded_file_path}
)

# Step 7: Save the transcript to a text file
transcript_file_path = os.path.join(save_dir, 'transcript_w_Langchain.txt')
with open(transcript_file_path, 'w') as f:
    f.write(result['text'])

# Step 8: Print the first 1000 characters of the transcript
print(document.page_content[:1000])
```

```
[youtube] Extracting URL: https://www.youtube.com/watch?v=2vkJ7v0x-Fs
[youtube] 2vkJ7v0x-Fs: Downloading webpage
[youtube] 2vkJ7v0x-Fs: Downloading ios player API JSON
[youtube] 2vkJ7v0x-Fs: Downloading web creator player API JSON
[youtube] 2vkJ7v0x-Fs: Downloading m3u8 information
[info] 2vkJ7v0x-Fs: Downloading 1 format(s): 251
[download] Destination: docs/youtube/Big Data Architectures.webm
[download] 100% of 22.03MiB in 00:00:01 at 11.07MiB/s
[ExtractAudio] Destination: docs/youtube/Big Data Architectures.m4a
Deleting original file docs/youtube/Big Data Architectures.webm (pass -k to keep)
```

In lesson four, we will go deeper into architectures for big data, and we will take a closer look at some of the most popular big data management systems. First, we're going to look at how the big data management system framework looks, and explore the commonalities that pretty much all the big data systems have, as well as some of the key differences between no SQL, MPP, and Hadoop. Next, we're going to take a deep dive into the Hadoop data management system. You will see how we both store data in HDFS, the database, as well as how data analytics processing works with MapReduce. We will also give some examples, and you will see how the Apache ecosystem of projects helps to expand and connect Hadoop with other tools, including machine learning libraries and other data management systems. Finally, we will look at the evolution of Hadoop based on Yarn. Yarn is a quantum leap forward for Hadoop that allows it to support almost any type of deep analytics and machine learning framework. In

LLamaIndex vs. LangChain – Basic Functions

```
# loading a video file and saving the audio to a text file, with LlamaIndex

import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"
import whisper
from yt_dlp import YoutubeDL
from llama_index.core import GPTVectorStoreIndex, Document as LlamaDocument
from llama_index.core.base.embeddings.base import BaseEmbedding
from sentence_transformers import SentenceTransformer
from pydantic import Field

# Step 1: setup the option to download and transcribe YouTube video (same as before)
url = "https://www.youtube.com/watch?v=2vkJ7v0x-Fs"
save_dir = "docs/youtube/"
output_template = os.path.join(save_dir, '%(title)s.%(ext)s')

ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': output_template,
    'postprocessors': [{
        'key': 'FFmpegExtractAudio',
        'preferredcodec': 'm4a',
        'preferredquality': '192',
    }],
    'ffmpeg_location': '/opt/homebrew/bin/ffmpeg',
}

# Step 2: Download the audio
with YoutubeDL(ydl_opts) as ydl:
    ydl.download([url])

# Step 3: Find the downloaded file
downloaded_file = [f for f in os.listdir(save_dir) if f.endswith('.m4a')][0]
downloaded_file_path = os.path.join(save_dir, downloaded_file)

# Step 4: Transcribe the audio file using Whisper
model = whisper.load_model("base")
result = model.transcribe(downloaded_file_path)
transcribed_text = result['text']
```

LLamaIndex vs. LangChain – Basic Functions

```
# Custom embedding class using Hugging Face model
class HuggingFaceEmbedding(BaseEmbedding):
    hf_model: SentenceTransformer = Field(default=None)

    def __init__(self, hf_model):
        super().__init__()
        self.hf_model = hf_model

    def _get_text_embedding(self, text):
        # Return embedding for a single string of text
        return self.hf_model.encode(text, convert_to_numpy=True)

    def _get_text_embeddings(self, texts):
        # Return embeddings for a list of strings
        return self.hf_model.encode(texts, convert_to_numpy=True)

    def _get_query_embedding(self, query):
        # Return embedding for a query
        return self._get_text_embedding(query)

    async def _aget_query_embedding(self, query):
        # Async version of embedding for a query
        return self._get_text_embedding(query)

# Adding metadata to the transcript, and saving the transcript to a file
class Document:
    def __init__(self, source, text, metadata=None):
        self.source = source
        self.page_content = text
        self.metadata = metadata or {}

# Step 6: Wrap the transcription result in the Document class with metadata
document = Document(
    source=downloaded_file_path,
    text=transcribed_text,
    metadata={"source": "youtube", "file_path": downloaded_file_path}
)
```

LLamaIndex vs. LangChain – Basic Functions

```
# Step 8: Use the LlamaIndex embedding model with the Hugging Face embedding model to retrieve and print the text
llama_document = LlamaDocument(text=transcribed_text)
embed_model = HuggingFaceEmbedding(hf_model)
index = GPTVectorStoreIndex([llama_document], embed_model=embed_model)

# Directly use the retriever (no LLM required)
retriever = index.as_retriever()

# Perform a query using the retriever
response = retriever.retrieve("What is the video about?") # this could be any question, as all we do below is retrieve the first 1000 characters of the text

# Print the first 1000 characters of the response text
if response:
    shortened_response = response[0].node.text[:1000] # Get the text from the node and limit to 500 characters
    print(f"Shortened Response: {shortened_response}")
else:
    print("No response retrieved")
```

[youtube] Extracting URL: <https://www.youtube.com/watch?v=2vkJ7v0x-Fs>

[youtube] 2vkJ7v0x-Fs: Downloading webpage

[youtube] 2vkJ7v0x-Fs: Downloading ios player API JSON

[youtube] 2vkJ7v0x-Fs: Downloading web creator player API JSON

[youtube] 2vkJ7v0x-Fs: Downloading m3u8 information

[info] 2vkJ7v0x-Fs: Downloading 1 format(s): 251

[download] Destination: docs/youtube/Big Data Architectures.webm

[download] 100% of 22.03MiB in 00:00:01 at 11.51MiB/s

[ExtractAudio] Destination: docs/youtube/Big Data Architectures.m4a

Deleting original file docs/youtube/Big Data Architectures.webm (pass -k to keep)

Transcript saved to docs/youtube/transcript_w_LlamaIndex.txt

Shortened Response: In lesson four, we will go deeper into architectures for big data, and we will take a closer look at some of the most popular big data management systems. First, we're going to look at how the big data management system framework looks, and explore the commonalities that pretty much all the big data systems have, as well as some of the key differences between no SQL, MPP, and Hadoop. Next, we're going to take a deep dive into the Hadoop data management system. You will see how we both store data in HDFS, the database, as well as how data analytics processing works with MapReduce. We will also give some examples, and you will see how the Apache ecosystem of projects helps to expand and connect Hadoop with other tools, including machine learning libraries and other data management systems. Finally, we will look at the evolution of Hadoop based on Yarn. Yarn is a quantum leap forward for Hadoop that allows it to support almost any type of deep analytics and machine learning framework. In

LLamaIndex vs. LangChain – Basic Functions

```
# Example tree index structure with llamaindex

#!pip install llama-index-llms-ollama

from llama_index.core import TreeIndex, SimpleDirectoryReader
from llama_index.llms.ollama import Ollama

# Initialize the Ollama LLM with Llama3
llama_llm = Ollama(model="llama3", timeout=500)

# Load a specific PDF file using SimpleDirectoryReader
pdf_path = "docs/802.11ae-2012_2.pdf"
documents = SimpleDirectoryReader(input_files=[pdf_path]).load_data()

# Create a hierarchical Tree Index using Llama3 from Ollama
tree_index = TreeIndex.from_documents(documents, llm=llama_llm)

# Access the storage context from the tree index
storage_context = tree_index.storage_context
```



LLamaIndex vs. LangChain – Basic Functions

```
# Function to print the tree structure
def print_tree_structure(node_id, level=0):
    try:
        # Retrieve the node object using the node_id from the storage context
        node = storage_context.docstore.get_document(node_id) # Use the exact UUID as node_id

        # Print the node name/text with indentation
        indent = " " * level # Adjust indentation
        print(f"{indent}Node ID: {node_id}")
        print(f"{indent}Text: {node.text[:100]}...") # Print the first 100 characters of the node text

        # Recursively print child nodes if they exist
        if hasattr(node, "child_ids") and node.child_ids:
            for child_id in node.child_ids:
                print_tree_structure(child_id, level + 1)
    except ValueError:
        print(f"Node ID {node_id} not found in storage context")

# Access the actual UUID root node IDs from the Tree Index
root_node_ids = list(tree_index.index_struct.root_nodes.values())

# Print the list of root node IDs to confirm
print("Actual Root Node IDs:", root_node_ids)

# Start printing from each root node using the actual UUIDs
for root_node_id in root_node_ids:
    print_tree_structure(root_node_id)
```

Ignoring wrong pointing object 18 0 (offset 0)

Ignoring wrong pointing object 24 0 (offset 0)

Actual Root Node IDs: ['e0a8a522-1af8-467d-a3c9-a248011e14f1', '6ca5b655-2c56-4e0f-a8eb-a81835b73988']

Node ID: e0a8a522-1af8-467d-a3c9-a248011e14f1

Text: IEEE Std 802.11ae-2012 AMENDMENT 1: PRIORITIZATION OF MANAGEMENT FRAMES

28 Copyright © 2012 IEEE. All...

Node ID: 6ca5b655-2c56-4e0f-a8eb-a81835b73988

Text: AMENDMENT 1: PRIORITIZATION OF MANAGEMENT FRAMES IEEE Std 802.11ae-2012

Copyright © 2012 IEEE. All r...

LLamaIndex vs. LangChain – Basic Functions

```
# documenting chapters and sections in LangChain flat structure
from langchain.document_loaders import PyPDFLoader
import re

# Load the PDF
pdf_loader = PyPDFLoader("docs/802.11ae-2012_2.pdf")
documents = pdf_loader.load()

# Function to extract paragraph indices from the text
def extract_paragraph_indices(text):
    # Regular expression pattern to match paragraph numbers like 9.4.2.1, 9.4.2.2
    paragraph_pattern = re.findall(r'\b\d+(\.\d+)+\b', text)
    return paragraph_pattern if paragraph_pattern else None

# Loop through documents to extract metadata
for i, doc in enumerate(documents):
    text = doc.page_content
    paragraph_indices = extract_paragraph_indices(text)
    page_number = i + 1 # assuming pages are in order in 'documents'

    # Add paragraph indices and page metadata
    if paragraph_indices:
        doc.metadata['paragraph_indices'] = paragraph_indices
        doc.metadata['page_number'] = page_number

    # Print the metadata structure for this document
    print(f'Document {i + 1} metadata: {doc.metadata}')
```

Ignoring wrong pointing object 18 0 (offset 0)

Ignoring wrong pointing object 24 0 (offset 0)

Document 1 metadata: {'source': 'docs/802.11ae-2012_2.pdf', 'page': 0, 'paragraph_indices': ['.2', '.1', '.122', '.2'], 'page_number': 1}

Document 2 metadata: {'source': 'docs/802.11ae-2012_2.pdf', 'page': 1, 'paragraph_indices': ['.2', '.3', '.4', '.7', '.3', '.3'], 'page_number': 2}

LLamaIndex and LangChain - Splitters

```
# Comparing 2 libraries
from langchain.text_splitter import RecursiveCharacterTextSplitter, CharacterTextSplitter

chunk_size = 20
chunk_overlap = 5

rsplit = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap
)
csplit = CharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap
)

text1 = 'abcdefghijklmnopqrstuvwxyz1234567890'
text2 = 'a b c d e f g h i j k l m n o p q r s t u v w x y z 1 2 3 4 5 6 7 8 9 0'
```

```
rsplit.split_text(text1)
```

```
['abcdefghijklmnopqrstuvwxyz', 'pqrstuvwxyz123456789', '567890']
```

```
rsplit.split_text(text2)
```

```
['a b c d e f g h i j',
 'i j k l m n o p q r',
 'q r s t u v w x y z',
 'y z 1 2 3 4 5 6 7 8',
 '7 8 9 0']
```

```
# Character splitter does not do anything, because it considers by default the end of paragraph as the separator.
csplit.split_text(text1)
```

```
['abcdefghijklmnopqrstuvwxyz1234567890']
```

```
csplit.split_text(text2)
```

```
['a b c d e f g h i j k l m n o p q r s t u v w x y z 1 2 3 4 5 6 7 8 9 0']
```

LLamaIndex and LangChain - Splitters

```
# A longer text, and a more realistic split
chunk_size = 700
chunk_overlap = 5

rsplit = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    separators=["\n\n", "\n", "(?<=\. )", " ", ""]
)
csplit = CharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap
)
```

```
text3 = """ The Martians seem to have calculated their descent with amazing subtlety--their mathematical learning is evidently far in excess of ours--and permitted it, we might have seen the gathering trouble far back in the nineteenth century. Men like Schiaparelli watched the red planet--it is odd, by-the-fluctuating appearances of the markings they mapped so well. All that time the Martians must have been getting ready. During the opposition of 1894 a great light was seen on the illuminated part of the disk, first at the Lick Observatory, then by Perrotin of Nice, and the inclined to think that this blaze may have been the casting of the huge gun, in the vast pit sunk into their planet, from which their shots were fired at two oppositions. The storm burst upon us six years ago now. As Mars approached opposition, Lavelle of Java set the wires of the astronomical exchange palpitating with the midnight of the twelfth; and the spectroscope, to which he had at once resorted, indicated a mass of flaming gas, chiefly hydrogen, moving with an enormous compared it to a colossal puff of flame suddenly and violently squirted out of the planet, as flaming gases rushed out of a gun. """
```

```
rsplit.split_text(text3)
```



```
['The Martians seem to have calculated their descent with amazing subtlety--their mathematical learning is evidently far in excess of ours--and to have carried out their preparations with a well-nigh perfect unanimity. Had our instruments \npermitted it, we might have seen the gathering trouble far back in the nineteenth century. Men like Schiaparelli watched the red planet--it is odd, by-the-bye, that for countless centuries Mars has been the star of war--but failed to interpret \nthe fluctuating appearances of the markings they mapped so well. All that time the Martians must have been getting ready.',
'During the opposition of 1894 a great light was seen on the illuminated part of the disk, first at the Lick Observatory, then by Perrotin of Nice, and then by other observers. English readers heard of it first in the issue of NATURE dated August 2. I am \ninclined to think that this blaze may have been the casting of the huge gun, in the vast pit sunk into their planet, from which their shots were fired at us. Peculiar markings, as yet unexplained, were seen near the site of that outbreak during the next \ntwo oppositions.',
'The storm burst upon us six years ago now. As Mars approached opposition, Lavelle of Java set the wires of the astronomical exchange palpitating with the amazing intelligence of a huge outbreak of incandescent gas upon the planet. It had occurred towards \nmidnight of the twelfth; and the spectroscope, to which he had at once resorted, indicated a mass of flaming gas, chiefly hydrogen, moving with an enormous velocity towards this earth. This jet of fire had become invisible about a quarter past twelve. He \ncompared it to a colossal puff of flame suddenly and violently squirted out of the planet, as flaming gases rushed out of a gun.']
```

LLamaIndex and LangChain - Splitters

```
: chunks=rsplit.split_text(text3)
for i, _ in enumerate(chunks):
    print(f"chunk # {i}, size: {len(chunks[i])}")
```

```
chunk # 0, size: 610
chunk # 1, size: 526
chunk # 2, size: 642
```

```
: print(chunks[0])
```

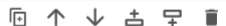
The Martians seem to have calculated their descent with amazing subtlety--their mathematical learning is evidently far in excess of ours--and to have carried out their preparations with a well-nigh perfect unanimity. Had our instruments permitted it, we might have seen the gathering trouble far back in the nineteenth century. Men like Schiaparelli watched the red planet--it is odd, by-the-by, that for countless centuries Mars has been the star of war--but failed to interpret the fluctuating appearances of the markings they mapped so well. All that time the Martians must have been getting ready.

```
: # What happens if the split is smaller:
chunk_size = 300
chunk_overlap = 5
rsplit = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    separators=["\n\n", "\n", "(?<=\. )", " ", ""]
)
```

```
: chunks=rsplit.split_text(text3)
for i, _ in enumerate(chunks):
    print(f"chunk # {i}, size: {len(chunks[i])}")
```

```
chunk # 0, size: 240
chunk # 1, size: 245
chunk # 2, size: 121
chunk # 3, size: 253
chunk # 4, size: 271
chunk # 5, size: 256
chunk # 6, size: 254
chunk # 7, size: 128
```

```
: print(chunks[0])
```



The Martians seem to have calculated their descent with amazing subtlety--their mathematical learning is evidently far in excess of ours--and to have carried out their preparations with a well-nigh perfect unanimity. Had our instruments

LLamaIndex and LangChain - Splitters

```
: # What happens if the split is smaller:  
chunk_size = 100  
chunk_overlap = 5  
rsplit = RecursiveCharacterTextSplitter(  
    chunk_size=chunk_size,  
    chunk_overlap=chunk_overlap,  
    separators=["\n\n", "\n", "(?<=\. )", " ", ""]  
)
```

```
: chunks=rsplit.split_text(text3)  
for i, _ in enumerate(chunks):  
    print(f"chunk # {i}, size: {len(chunks[i])}")
```

```
chunk # 0, size: 92  
chunk # 1, size: 96  
chunk # 2, size: 52  
chunk # 3, size: 99  
chunk # 4, size: 95  
chunk # 5, size: 54  
chunk # 6, size: 96  
chunk # 7, size: 29  
chunk # 8, size: 98  
chunk # 9, size: 95  
chunk # 10, size: 61  
chunk # 11, size: 97  
chunk # 12, size: 90  
chunk # 13, size: 73  
chunk # 14, size: 16  
chunk # 15, size: 97  
chunk # 16, size: 92  
chunk # 17, size: 74  
chunk # 18, size: 97  
chunk # 19, size: 96  
chunk # 20, size: 69  
chunk # 21, size: 93  
chunk # 22, size: 37
```

```
: print(chunks[0])
```

The Martians seem to have calculated their descent with amazing subtlety—their mathematical

LLamaIndex and LangChain - Splitters

```
# LlamaIndex has different splitters, including sentence splitter, that makes splits based on sentences (period).

from llama_index.core.node_parser import SentenceSplitter

from llama_index.core.node_parser import SentenceSplitter

# Initialize the SentenceSplitter with your chunk size and overlap
ssplit = SentenceSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap
)

# Apply the splitter on your text
split_texts = ssplit.split_text(text1)

# Display the split chunks
for idx, chunk in enumerate(split_texts):
    print(f"Chunk {idx + 1}:")
    print(chunk)
    print("\n" + "-" * 50 + "\n")
```

Chunk 1:
abcdefghijklmnopqrstuvwxyz1234567890

Both frameworks have splitters with similar goals, including for LangChain (beyond CharacterSplitter and RecursiveCharacterSplitter) HTML splitter (HTMLHeaderTextSplitter, to chop HTML pages while following the page structure), Code splitter (RecursiveCharacterSplitter with the option e.g. language=language.PYTHON), recursive JSON splitter (RecursiveJSONSplitter), Semantic splitter (SemanticChunker, splits in sentences and uses LLM to try to find semantic structures to group chunks that form coherent semantic ensembles) and tokens, and for LlamaIndex (beyond SentenceSplitter and TokenTextSplitter), HTML (HTMLNodeParser), JSON (JSONNodeParser), Markdown (MarkdownNodeParser), Code (CodeSplitter with the option e.g. language="python"), Hierarchical splitting (HierarchicalNodeParser, to attempt the best hierarchy of splits based on semantic meaning) and semantic splitting (SemanticSplitterNodeParser).

LLamaIndex and LangChain – Similarity Search

```
# Let's define a directory where we'll store the database beyond this notebook execution (and let's make sure it is empty, as I run this notebook often :  
persist_directory = 'docs/chroma/'  
embeddings = OllamaEmbeddings(model="nomic-embed-text")  
!rm -rf ./docs/chroma # remove old database files if any
```

```
text4 = [  
    """The alien spaceships looked like flying saucers.""",  
    """The alien spaceships were round in shape.""",  
    """The spaceships were destroying everything."""  
]
```

```
tempdb = Chroma.from_texts(text4, embedding=embeddings)
```

```
question = "What can you tell me about the alien spaceships?"
```

```
tempdb.similarity_search(question, k=2)
```

```
[Document(metadata={}, page_content='The alien spaceships were round in shape.'),  
 Document(metadata={}, page_content='The alien spaceships were round in shape.')]
```

Similarity search points to the documents that are closest semantically to the question, which may include a lot of redundant information, and miss some key points, for example that the alien spaceships were destroying everything. Max Marginal Relevance (MMR) search improves Similarity Search by picking the top k as Similarity Search does, but returning the vectors that are farthest from each other (in this top k list), so as to maximize the diversity of information returned.

```
tempdb.max_marginal_relevance_search(question, k=2, fetch_k=3)
```

```
[Document(metadata={}, page_content='The alien spaceships were round in shape.'),  
 Document(metadata={}, page_content='The alien spaceships looked like flying saucers.')]
```

Type Markdown and LaTeX: α^2

LangChain Chains and Tools

```
from langchain.prompts import ChatPromptTemplate
from langchain.schema.output_parser import StrOutputParser
```

```
from langchain.prompts import ChatPromptTemplate
from langchain.llms import Ollama
```

```
# Define a prompt template, used to build a prompt from various elements
prompt = ChatPromptTemplate.from_template(
    "tell me a short useful fact about {topic}"
)
```

```
# Use Ollama with Llama3 model
model = Ollama(model="llama3")
```

```
# Define the output parser, which simply takes the LLM output and displays it as a string
output_parser = StrOutputParser()
```

```
# With LangChain you can define a chain of elements, here the prompt (output) we built is redirected into the model (as input), the model output is redirected to the output_parser
chain = prompt | model | output_parser
```

```
# Let's call the chain
chain.invoke({"topic": "Paris"})
```

Bonjour!\n\nHere's a short and useful fact about Paris:\n\nThe city of Paris has an underground tunnel system called the Catacombs de Paris, which stretches for over 150 miles (240 km) beneath the city. While not all of it is open to the public, parts of the catacombs are accessible on guided tours, offering a unique glimpse into Paris' macabre history and a chance to explore a hidden world beneath the City of Light!"

LangChain Chains and Tools

```
# The same logic can be used in RAG. Suppose that we have two chunks.
```

```
#!pip install docarray
```

```
from langchain.embeddings import OllamaEmbeddings
```

```
from langchain.vectorstores import DocArrayInMemorySearch
```

```
# Initialize the Ollama embedding model
```

```
embedding = OllamaEmbeddings(model="llama3")
```

```
# Create the vector store with two chunks of text (for simplicity. Feel free to load a full pdf or video transcript as we did above in in the previous le
```

```
vectorstore = DocArrayInMemorySearch.from_texts(
```

```
    ["The Martians landed in the UK", "The river that flows in Paris is La Seine"],
```

```
    embedding=embedding
```

```
)
```

```
# Use the vector store as a retriever
```

```
retriever = vectorstore.as_retriever()
```

```
# Let's see what the retriever does when we ask a simple question - it should retrieve the available chunks, here both chunks irrespective of the question
```

```
retriever.get_relevant_documents("where did the Martians land?")
```

```
[Document(metadata={}, page_content='The river that flows in Paris is La Seine'),
```

```
 Document(metadata={}, page_content='The Martians landed in the UK')]
```

```
# Now let's create a prompt template as above, that tells the LLM to use the context (i.e. the best chunk) to answer the question
```

```
template = """Answer the question based only on the following context:
```

```
{context}
```

```
Question: {question}
```

```
"""
```

```
prompt = ChatPromptTemplate.from_template(template)
```

LangChain Chains and Tools

```
# We want to pass the context (the 2 sentences retrieved from the vector store) and the question to the LLM. We use RunnableMap to create a dictionary with
from langchain.schema.runnable import RunnableMap
inputs = RunnableMap({
    "context": lambda x: retriever.get_relevant_documents(x["question"]), # the context is retrieved by sending the question to the retriever, as we did in the previous code
    "question": lambda x: x["question"] # the second element of the dictionary is the question itself
})
```

```
# Let's see what the dictionary looks like
inputs.invoke({"question": "where did the Martians land?"})
```

```
{'context': [Document(metadata={}, page_content='The river that flows in Paris is La Seine'),
Document(metadata={}, page_content='The Martians landed in the UK')],
'question': 'where did the Martians land?'}
```

```
# We then want to pass the content of the dictionary to the prompt template (making a prompt of it), that is sent to the model, which output is sent to the parser.
# Let's define the dictionary again, this time piping it into the prompt, then to the model, then to the parser, and we call the whole chain 'chain'
chain = RunnableMap({
    "context": lambda x: retriever.get_relevant_documents(x["question"]),
    "question": lambda x: x["question"]
}) | prompt | model | output_parser
```

```
# What happens when we call the chain?
chain.invoke({"question": "where did the Martians land?"})
```

'According to the provided documents, the Martians landed in the UK.'

Section 6: Developing LLM Apps



Lillac / Shutterstock

- Building the Interface

ChatBot Principles

You need the right API calls, good model inputs, and an attractive UI

Step 1: Model choice

Each model is a compromise between task (specialization), size and precision.

Step 2: Model feed

Use APIs to point the model(s) to the right data, before concatenating the results

Step 3: Model wrapper

The UI can be simple, or include safeguards and filters.

Simple UI in Python

```
# Let's wrap the RAG into a simple UI.
import gradio as gr
import ollama
from bs4 import BeautifulSoup as bs
from langchain_community.embeddings import OllamaEmbeddings

# Create Ollama embeddings and vector store
#embeddings = OllamaEmbeddings(model="nomic-embed-text")
#vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)

# Define the function to call the Ollama Llama3 model
def ollama_llm(question, context):
    # Explicitly create a new conversation with only the current prompt
    formatted_prompt = f"Question: {question}\n\nContext: {context}"
    response = ollama.chat(model='llama3', messages=[{'role': 'user', 'content': formatted_prompt}])
    return response['message']['content']

# Define the RAG setup
retriever = vectordb.as_retriever()

def rag_chain(question):
    retrieved_docs = retriever.invoke(question)
    formatted_context = "\n\n".join(doc.page_content for doc in retrieved_docs)
    return ollama_llm(question, formatted_context)

# Define the Gradio interface
def get_important_facts(question):
    return rag_chain(question)

# Create a Gradio app interface
iface = gr.Interface(
    fn=get_important_facts,
    inputs=gr.Textbox(lines=2, placeholder="Enter your question here..."),
    outputs="text",
    title="RAG with Llama3",
    description="Ask questions",
)

# Launch the Gradio app
iface.launch()
```

Simple UI in Python

RAG with Llama3

Ask questions

question

Did the Martians land on Venus?

Clear Submit

output

No, the Martians did not land on Venus in this context. The passage suggests that the Martians are now watching over a settlement on another planet, possibly referring to a hypothetical colonization of Venus. The narrator mentions that "on the planet Venus they have found a securer settlement" and also notes that there will be no relaxation of the scrutiny of the Martian disk for many years to come, implying that the Martians may have established a new base or colony on Venus.

Flag

Use via API  · Built with Gradio 

Simple UI in Python – Adding Weather Module

```
# Let's define a simple function that returns the weather for a location
import sys
import os
import openai
from dotenv import load_dotenv

# Load environment variables from the .env file
load_dotenv()

# Retrieve the OpenWeatherMap API key from the environment variables
OPENWEATHER_API_KEY = os.environ.get('OPENWEATHER_API_KEY') # set a keys.env file in the parent directory, and set there your openweather API key

from langchain.agents import tool
import requests
from pydantic import BaseModel, Field

# Define the input schema
class CityInput(BaseModel):
    city: str = Field(..., description="City name to fetch weather data for")

# Tool to get the current weather
@tool(args_schema=CityInput)
def get_current_weather(city: str) -> dict:
    """Fetch current weather for a given city."""

    BASE_URL = "http://api.openweathermap.org/data/2.5/weather"

    # Parameters for the weather request
    params = {
        'q': city,
        'appid': OPENWEATHER_API_KEY,
        'units': 'metric', # To get the temperature in Celsius
    }

    # Make the request
    response = requests.get(BASE_URL, params=params)
```


Simple UI in Python – Adding Weather Module

```
if response.status_code == 200:
    results = response.json()
    weather_data = {
        "City": city,
        "Temperature": f"{results['main']['temp']}°C",
        "Weather Description": results['weather'][0]['description'],
        "Humidity": f"{results['main']['humidity']}%",
        "Wind Speed": f"{results['wind']['speed']} m/s",
        "Pressure": f"{results['main']['pressure']} hPa"
    }
else:
    raise Exception(f"Weather API Request failed with status code: {response.status_code}")

return weather_data
```

```
: get_current_weather("Richmond")
```

```
: {'City': 'Richmond',
  'Temperature': '23.22°C',
  'Weather Description': 'broken clouds',
  'Humidity': '73%',
  'Wind Speed': '4.12 m/s',
  'Pressure': '1015 hPa'}
```

Simple UI in Python – With Chain & Weather

```
: #Integrating the weather in the full code

import gradio as gr
import ollama
from bs4 import BeautifulSoup as bs
from langchain_community.embeddings import OllamaEmbeddings
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain
from langchain_core.runnables import Runnable
import requests
from pydantic import BaseModel, Field

# Weather tool function
def get_current_weather(city: str) -> str:
    """Fetch current weather for a given city."""

    BASE_URL = "http://api.openweathermap.org/data/2.5/weather"

    params = {
        'q': city,
        'appid': OPENWEATHER_API_KEY,
        'units': 'metric', # To get the temperature in Celsius
    }

    response = requests.get(BASE_URL, params=params)
```

Simple UI in Python – With Chain & Weather

```
if response.status_code == 200:
    results = response.json()
    temperature = results['main']['temp']
    weather_description = results['weather'][0]['description']
    humidity = results['main']['humidity']
    wind_speed = results['wind']['speed']
    pressure = results['main']['pressure']
else:
    raise Exception(f"Weather API Request failed with status code: {response.status_code}")

return (
    f"The current temperature in {city} is {temperature}°C with {weather_description}.\n"
    f"Humidity: {humidity}%, Wind Speed: {wind_speed} m/s, Pressure: {pressure} hPa."
)

# Define the prompt template
template = """Use the following pieces of context to answer the question at the end.
If you don't know the answer, just say that you don't know, don't try to make up an answer.
Use three sentences maximum. Keep the answer as concise as possible.
Always say "thanks for asking!" at the end of the answer.

{context}

Question: {question}
Helpful Answer: """
QA_CHAIN_PROMPT = PromptTemplate.from_template(template)
```

Simple UI in Python – With Chain & Weather

```
# Custom Runnable LLM class with debugging
class OllamaLLM(Runnable):
    def __init__(self, llm_fn):
        self.llm_fn = llm_fn

    def invoke(self, input, config=None, **kwargs):
        question = str(input)
        context = kwargs.get("context", "")
        print(f"Question passed to LLM: {question}")
        print(f"Context passed to LLM: {context}")
        stop = kwargs.get("stop", None)
        response = self.llm_fn(question, context)
        print(f"Response from LLM: {response}")
        return response

    def predict(self, input, **kwargs):
        return self.invoke(input, **kwargs)

    def __call__(self, *args, **kwargs):
        return self.invoke(*args, **kwargs)

# Instantiate the custom LLM class
ollama_llm_instance = OllamaLLM(ollama_llm)

# Define the conversational retrieval chain
qa_chain = ConversationalRetrievalChain.from_llm(
    llm=ollama_llm_instance,
    retriever=retriever,
    memory=memory # add the memory module to pass the previous exchange to the LLM as well
)
```

Simple UI in Python – With Chain & Weather

```
# Function to detect if the question is about the weather in a city
def is_weather_question(question: str) -> bool:
    return "weather" in question.lower() and "in" in question.lower()

# Extract city name from the weather question
def extract_city_from_question(question: str) -> str:
    # Simple heuristic to extract city name
    if "weather in" in question.lower():
        return question.lower().split("weather in")[1].strip().split()[0].capitalize()
    return ""

# Define the function to get important facts with debugging
def get_important_facts(question):
    # Check if the question is about the weather in a city
    if is_weather_question(question):
        city = extract_city_from_question(question)
        if city:
            return get_current_weather(city)
        else:
            return "I couldn't determine the city you're asking about. Please specify the city."

    # Otherwise, use the LLM-based chain
    print(f"Question passed to retriever: {question}")
    response = qa_chain.run({"question": question})
    print(f"Memory state: {memory.buffer}")
    return response

# Create a Gradio app interface
iface = gr.Interface(
    fn=get_important_facts,
    inputs=gr.Textbox(lines=2, placeholder="Enter your question here..."),
    outputs="text",
    title="RAG with Llama3",
    description="Ask questions",
)

# Launch the Gradio app
```

Simple UI in Python – With Chain & Weather

RAG with Llama3

Ask questions

question

In which city does the book take place?

ClearSubmit

output

The context for answering this question is provided in the text. Specifically, it refers to the conversation between two characters, one of whom mentions the destruction of the church and the rebuilding of it three years ago, as well as the mention of a city called Weybridge.

Helpful Answer:

Based on the conversation, we can infer that the story takes place in Weybridge.

Flag

Use via API  · Built with Gradio 

Simple UI in Python – With Chain & Weather

RAG with Llama3

Ask questions

question

And what is the weather in Weybridge, UK?

Clear

Submit

output

The current temperature in Weybridge, is 15.54°C with broken clouds.
Humidity: 85%, Wind Speed: 5.66 m/s, Pressure: 1003 hPa.

Flag

Use via API  · Built with Gradio 

Simple UI in Python – Improving the UI

```
: import gradio as gr
import ollama

from bs4 import BeautifulSoup as bs
from langchain_community.embeddings import OllamaEmbeddings
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain
from langchain_core.runnables import Runnable

# Define the prompt template
template = """Use the following pieces of context to answer the question at the end.
If you don't know the answer, just say that you don't know, don't try to make up an answer.
Use three sentences maximum. Keep the answer as concise as possible.

{context}

Question: {question}
Helpful Answer: """
QA_CHAIN_PROMPT = PromptTemplate.from_template(template)

# Define memory to store the previous exchanges
memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Custom Runnable LLM class with debugging
class OllamaLLM(Runnable):
    def __init__(self, llm_fn):
        self.llm_fn = llm_fn

    def invoke(self, input, config=None, **kwargs):
        question = str(input) # Convert the input to a string if necessary
        context = kwargs.get("context", "") # Retrieve context from kwargs if available

        # Debugging outputs
        print(f"Question passed to LLM: {question}")
        print(f"Context passed to LLM: {context}")
```


Simple UI in Python – Improving the UI

```
stop = kwargs.get("stop", None)
response = self.llm_fn(question, context)

# Debugging output
print(f"Response from LLM: {response}")

return response

def predict(self, input, **kwargs):
    return self.invoke(input, **kwargs)

def __call__(self, *args, **kwargs):
    return self.invoke(*args, **kwargs)

# Instantiate the custom LLM class
ollama_llm_instance = OllamaLLM(ollama_llm)

# Define the conversational retrieval chain
qa_chain = ConversationalRetrievalChain.from_llm(
    llm=ollama_llm_instance,
    retriever=retriever,
    memory=memory
)

# Define the function to get important facts with debugging
def get_important_facts(question, chat_history):
    print(f"Question passed to retriever: {question}")

    response = qa_chain.run({"question": question})
    print(f"Memory state: {memory.buffer}")

    # Append to chat history
    chat_history.append((question, response))
    return chat_history, chat_history
```

Simple UI in Python – Improving the UI

```
# Create a Gradio chat interface
with gr.Blocks() as iface:
    gr.Markdown("# RAG with Llama3")
    chatbot = gr.Chatbot(height=250) # Adjust the height here
    question_input = gr.Textbox(lines=2, placeholder="Enter your question here...", show_label=False)
    submit_btn = gr.Button("Submit")
    clear_btn = gr.Button("Clear Chat")

    # Set up the interaction
    submit_btn.click(get_important_facts, [question_input, chatbot], [chatbot, chatbot])
    question_input.submit(get_important_facts, [question_input, chatbot], [chatbot, chatbot])
    clear_btn.click(lambda: None, None, chatbot, queue=False)

# Launch the Gradio app
iface.launch()

# e.g. q: Tell me about the Martian invasion
```

Simple UI in Python – Improving the UI

RAG with Llama3

Chatbot

Based on the provided text, the context suggests that the story takes place after a catastrophic event where humanity has been wiped out by an alien invasion from Mars. The narrator is describing their experience and emotions as they navigate this new world.

What is the weather in London, UK?

The context for this question is that it appears in a conversation about a book taking place after a catastrophic event caused by an

What is the weather in London, UK?

Submit

Clear Chat

Improving Your Code – GenAI Helpers

- GitHub Copilot

- Gen AI to review/improve code (and help with GitHub functions)
- Includes ChatBot function (Chat about code and ask for code snippets or code review)
- Also includes inline queries

Your requests Copilot responses

```
// search for a product by name
app.get('/products/search/:name', (req, res) => {
  const products = productService.searchProductByName(req.params.name);
  // return an error if no products found
  if (!products) {
    res.status(404).send('Product not found');
  }
})
```

Your request

```
0 references
public static string GeneratePassword(int length = 8) {
  var random = new Random();
  change this to
}
```

⚠ Copilot generated code may be incorrect

Improving Your Code – GenAI Helpers

- AWS Q Developer
 - Gen AI to help optimize AWS resources usage, troubleshoot issues
 - Includes CodeWhisperer, plugin to Jupyter to generate code from comments

```
1 import os
2 import json
3 import boto3
4 import requests
5 import botocore.exceptions
6
7 s3_client = boto3.client("s3")
8 S3_BUCKET = os.getenv('BUCKET_NAME')
9
10 # 1. Function to get a file from url
11 def get_file_from_url(url):
12     response = requests.get(url)
13     return response.content
14
15 # 2. Function to upload image to S3
16
17
18 def handler(event, context):
19     url = event["queryStringParameters"]["url"]
20     name = event["queryStringParameters"]["name"]
21
22     # pass the output of method #1 as input to method #2
23
24     return {
25         'statusCode': 200,
26         'body': json.dumps('Successfully Uploaded Img!')
27     }
28
```

Filter files by name

Name	Last Modified
data.csv	18 hours ago
data.json	19 hours ago
example1a.py	36 minutes ago
example1b.py	34 minutes ago
example2a.py	33 minutes ago
example2b.py	17 hours ago
get_save_image.py	2 minutes ago

Launcher

get_save_image.py

Your requests

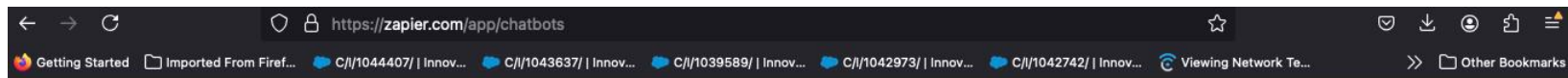
CodeWhisperer proposals

Using Integration Tools - Zapier

- Zapier builds tools for automation
- Integrates with >3000 applications
- Chatbot is one common integration interface

The screenshot displays the Zapier web application interface. At the top, there is a navigation bar with a search icon, a help icon, an 'Explore apps' button, a 'Contact Sales' button, and an 'Upgrade' button. A dark banner across the top right contains the text 'Last chance to save your seat at ZapConnect', the 'zapCONNECT' logo, the date 'Sept 28 | online', a 'Register for free' button, and a close icon. The left sidebar features a '+ Create' button and a list of navigation items: Home, Discover, Zaps, Tables, Interfaces, Chatbots (marked Beta), Canvas (marked Beta), Apps, Zap History, and More. The main content area begins with a 'Welcome to Zapier, Jerome!' message. Below this is a large box titled 'What would you like to automate?' with a 'Beta' tag. It contains a text input field with the placeholder 'The more details, the better.' and a right-pointing arrow. An example text below the field reads: 'Example: When I add a reaction to a Slack message, create a card in Trello.' Further down, there is a 'Recently updated' section with a dropdown menu set to 'All recently updated'. It lists 'Research Assistant' as the most recent update, with a note 'Updated 2 hours ago' and a disclaimer 'Data may be delayed by 5 minutes'. The 'Start with a template' section at the bottom offers five pre-built automation templates, each with a 'Zap' icon and a description: 1. 'Save new Gmail attachments to Google Drive' (Gmail and Google Drive icons). 2. 'Save new Gmail emails matching certain traits to a Google Spreadsheet' (Gmail and Google Sheets icons). 3. 'Create new Trello cards from new Google Calendar events' (Google Calendar and Trello icons). 4. 'Add new Facebook Lead Ads leads to rows on Google Sheets' (Facebook and Google Sheets icons). 5. 'Get email notifications or send follow-ups to new Typeform respondents' (Typeform and Gmail icons).

Zapier Chatbot Templates



+ Create

Home

Discover

Zaps

Tables

Interfaces

Chatbots Beta

Canvas Beta

Apps

Zap History

More

 Chatbots Beta

+ Build Your First Chatbot

Create a free, AI-powered chatbot that answers questions, resolves issues, and nurtures leads with the power of Zapier automation.

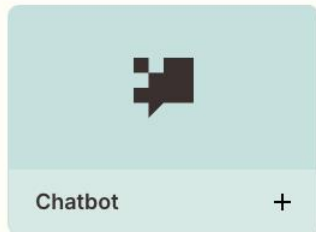
[Run Zaps](#) [Collect Info](#) [Embedding](#)



"Watch this demo on how to build a chatbot and embed it on your site in minutes!"

1:44

Start simple



Templates

All

Customer support

Lead management

Marketing campaigns

Project management

Sales pipeline

Zapier ChatBot Templates

Templates

[All](#)[Customer support](#)[Lead management](#)[Marketing campaigns](#)[Project management](#)[Sales pipeline](#)

AI Chat for Business

Use your business content to resolve support and lead questions with a customized chatbot.

👍 2,170

+ Use template

AI Chat for Lead Generation

Increase lead engagement with a conversational chat experience embedded on your website.

👍 1,754

+ Use template

ZapChat AI Chatbot

Build your own AI chatbot experience for your stakeholders powered by OpenAI.

👍 427

+ Use template

Social Media AI Chatbot

Generate posts and streamline your social media strategy with this chatbot, powered by OpenAI.

👍 401

+ Use template

Blog Generator AI Chatbot

Unleash your inner wordsmith with Zapier AI Chatbot's blog editing and writing tool, powered by OpenAI.

👍 374

+ Use template

AI Chat for Customer Service

Resolve customer questions and reduce support volume with a customized chatbot.

👍 368

+ Use template

↓ View more

Zapier Research Assistant ChatBot Setup

InstructionsKnowledgeNewLogicStyleConversationsAdvancedAnalytics

View Live ChatbotShareEmbed

Chatbot Instructions

Directive ⓘ

 Pop out

#Role:
You are a researcher of many topics, known for sharing accurate, well-cited information from journals, websites, and articles that provide multiple viewpoints on any given topic.

#Objective:
Your objectives are to ask for details whenever possible and provide well-cited, researched content including links to original content when

Greeting ⓘ

Static

I can help you research anything. Let me know what you'd like to learn about and then we can get started! Here are a couple of ideas for starting the conversation: 1. I'd like to learn about [topic]! 2. What are some

AI Model

Model

OpenAI GPT-4o mini

[Upgrade](#) to use other models available to you in OpenAI.

Restart Chat

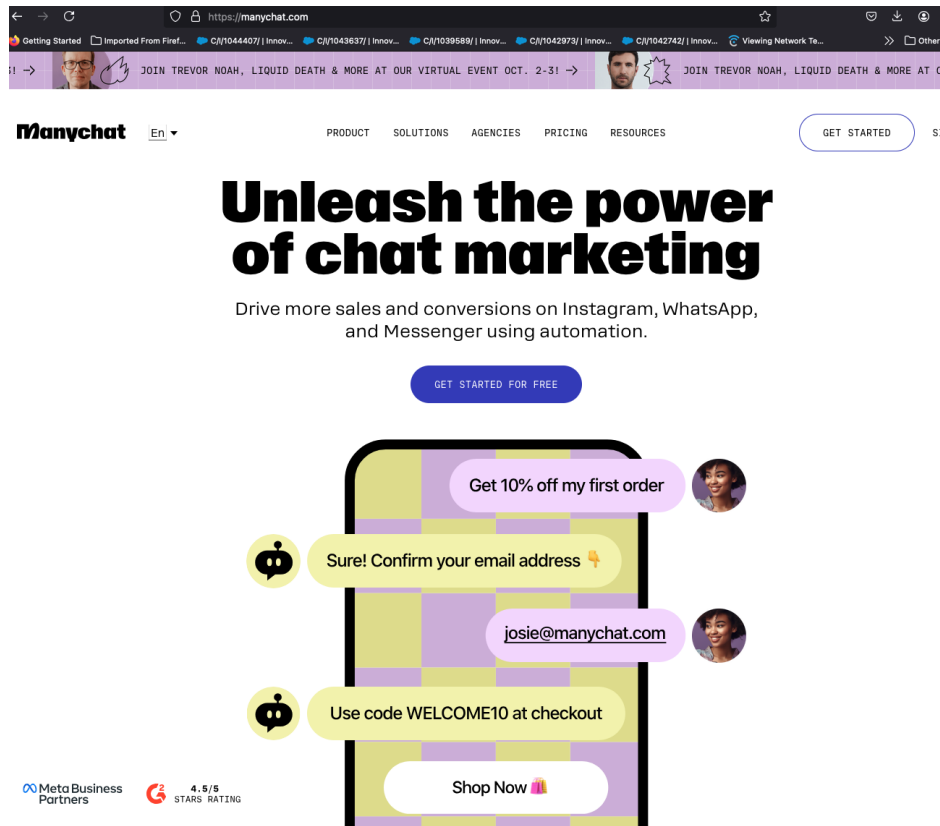


Research Assistant

I can help you research anything. Let me know what you'd like to learn about and then we can get started! Here are a couple of ideas for starting the conversation: 1. I'd like to learn about [topic]! 2. What are some reputable sources for information on [subject]? 3. Do you have any recommendations for research methodologies for studying [topic]?

This is an automated chatbot response. [Learn more](#)

Other UI Options



Trusted by 1M+ Businesses

